Frankestein Transformer: Unified Encoder-Decoder Library, CLI, and Research-Grounded Design Notes

Erick F. Merino M.
This work is not affiliated
erickfmm@gmail.com

February 2026

Abstract

Transformer architectures have become the dominant paradigm for sequence modeling across natural language processing, computer vision, and computational biology. Recent years have witnessed a rapid diversification of attention mechanisms and optimization strategies, with innovations spanning dense softmax variants, recurrent and retentive alternatives, sparse attention patterns, gated linear mechanisms, latent-space projections, and memory-augmented blocks. Concurrently, the optimizer landscape has expanded beyond classical AdamW to include variance-reduction methods, memory-efficient variants, schedule-free approaches, second-order preconditioners, and low-rank approximations. Despite this wealth of architectural and algorithmic advances, the research ecosystem remains fragmented: comparing sequence mixers and optimizers under consistent training conditions requires substantial engineering effort, hindering reproducibility and slowing scientific progress. In this work, we propose Frankestein Transformer, a configuration-driven CLI toolkit (`frankestein-transformer`) that unifies the definition, training, and deployment of transformer models through a strict YAML schema. The toolkit supports thirty-five sequence mixer variants across six families (dense, recurrent, sparse, gated, latent, and memory-augmented) and twenty-three optimizer families with prefixed per-parameter-group hyperparameter control. It provides subcommands for training (`train`), deployment (`deploy`), quantization (`quantize`), inference (`infer`), SBERT-style sentence-embedding training and inference (`sbert-train`, `sbert-infer`), HuggingFace-compatible export (`transformers-export`), BitNet-aware GGUF export (`bitnet-gguf`), and a web-based configuration builder (`web-server`). The system enforces reproducibility through a strict schema contract with `additionalProperties: false` constraints, offers a Streamlit-based web interface for schema-driven form rendering with real-time validation and CLI command generation, and provides end-to-end workflows for quantized deployment via ternary weight packing and sentence-embedding pipelines inspired by SBERT. A comprehensive automated testing suite with unit tests, YAML preset validation, and continuous integration ensures reliability across supported Python versions.

Contents

1 Introduction

1.1 Motivation and Problem Statement

Transformer architectures have fundamentally transformed the landscape of sequence modeling across natural language processing [79], computer vision, and computational biology. The success of models such as BERT [20], GPT, and their variants has established the transformer as the de facto standard for representation learning in deep learning. However, the rapid proliferation of architectural innovations presents significant practical challenges for researchers and practitioners.

Recent years have witnessed an explosion of alternative attention and sequence-mixing mechanisms, each addressing specific limitations of standard softmax attention: quadratic computational complexity [15], memory-efficient inference requirements [76, 31], content-based selective state management [7], and hardware-aware optimizations [67]. Simultaneously, the optimization literature has diversified beyond classical AdamW, introducing variance-reduction techniques [84, 60, 90], memory-efficient variants [72, 95], schedule-free approaches [18], and second-order preconditioning methods [33, 80].

The practical consequence is a fragmented research ecosystem where experimental comparison across architectures and optimizers requires significant engineering effort. Researchers must implement and debug multiple variants from scratch, ensure consistent training pipelines, and manage complex hyperparameter spaces. This fragmentation hampers reproducibility, slows scientific progress, and increases the barrier to entry for new researchers.

This work addresses these challenges through a unified, configuration-driven experimentation toolkit available at `https://github.com/erickfmm/frankestein-transformer` that provides:

1. **Schema-First Design**: A strict, validated configuration contract that enforces reproducibility while supporting thirty-five sequence mixer variants across six families and twenty-three optimizer families.

2. **Architecture Agnostic Training**: Common training infrastructure supporting dense attention baselines (standard, sigmoid), recurrent alternatives (RetNet, Mamba, ODE-style blocks), adaptive depth routing (Mixture-of-Depths) [98], conditional memory blocks (Engram) [14], sparse attention patterns (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA), and gated mechanisms (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax).

3. **Optimizer Routing Framework**: Prefixed hyperparameter groups enabling fine-grained control over embeddings, normalization layers, recurrent blocks, attention blocks, and other parameter subsets across diverse optimizers, including the APOLLO family (Apollo, Apollo-Mini, Q-Apollo) [96].

4. **End-to-End Workflows**: Integrated deployment via quantization (ternary weight packing, INT8 activations) and sentence-embedding capabilities inspired by SBERT [68].

5. **Interactive Configuration**: Web-based interface providing schema-driven form rendering, real-time validation, and CLI command generation.

6. **Reliability Tooling**: Comprehensive automated testing with unit-test suites, YAML preset validation, and CI automation across supported Python versions.

The project command surface is:

```
frankestein-transformer
```

with subcommands for encoder and decoder workflows: `train`, `deploy`, `quantize`, `infer`, `sbert-train`, `sbert-infer`, `transformers-export`, `bitnet-gguf`, `web-server`.

The `web-server` command launches a Streamlit-based configuration builder that provides:

- Schema-driven form fields with parameter titles and detailed descriptions
- Real-time tooltips and help text for each configuration option
- Live YAML preview and download functionality
- Generated CLI commands for training, deployment, inference, and SBERT workflows

This interactive interface serves as an alternative to manual YAML editing, improving usability for users exploring available configuration options and understanding their impact on model behavior and training dynamics.

1.2 Contributions

This work makes the following primary contributions:

1. **Unified Configuration Schema**: A YAML-based schema contract with strict validation that supports thirty-five sequence mixer variants across six families (dense, recurrent, sparse, gated, latent, and memory-augmented), alongside adaptive depth and conditional memory controls, while enforcing reproducibility through `additionalProperties: false` constraints.

2. **Comprehensive Architecture Support**: Implementation of modern transformer variants including standard softmax attention [79], sigmoid attention [67], RetNet [76], Mamba [31], ODE-style continuous transformers [93], Titans memory-augmented attention [7], Engram conditional memory layers [14], Mixture-of-Depths token routing [98], sparse attention mechanisms [15, 8, 91, 52, 89, 94, 82], and gated attention architectures [85, 86, 87, 34, 64, 47, 65]. System supports both encoder-mode training with bidirectional attention for masked language modeling (MLM) and decoder-mode training with causal masking for autoregressive next-token prediction.

3. **Optimizer Routing Framework**: Prefixed hyperparameter system enabling per-parameter-group control across twenty-three optimizers, including the adaptivity-tunable Anon [3], variance-reduction methods (MARS, Adan, AdEMAMix, Cautious AdamW), memory-efficient variants (Adafactor, GaLore, Lion), schedule-free approaches (Schedule-Free AdamW), curvature-aware methods (Sophia), second-order preconditioners (Shampoo, SOAP), orthogonality-oriented optimizers (Muon, Turbo-Muon), large-batch scaling (LAMB), and the APOLLO family (Apollo, Apollo-Mini, Q-Apollo) [96].

4. **Quantization and Deployment**: Integrated deployment pipeline supporting ternary weight packing and INT8 activation quantization with size estimates following 1.58-bit storage approximation.

5. **Sentence-Embedding Workflows**: SBERT-inspired training and inference pipelines supporting similarity scoring, retrieval, clustering, and persistent embedding export.

6. **Interactive Configuration Interface**: Streamlit-based web server providing schema-driven form generation, real-time validation, inline documentation, and CLI command generation.

7. **Automated Validation Pipeline**: Continuous integration and expanded unit testing that verify model training paths, optimizer/schema integration, and YAML example compatibility.

1.3 Reading Guide

This document is organized as a technical reference addressing four operational concerns:

1. **Prerequisites**: Appendix 1 provides an accessible introduction to transformers and attention mechanisms for readers new to the field.

2. **Configuration Contract**: Section 3.1 describes the YAML schema that enforces valid experiments and Section 3.2 explains validation rules.

3. **Architecture Selection**: Section 4.1 provides comprehensive comparison of sequence mixer families; Appendices 5, 6, 7, and 8 synthesize supporting literature; Appendix 9 details normalization variants, BitNet quantization, Mixture-of-Depths routing, and Engram conditional memory.

4. **Optimization Dynamics**: Section 5 details optimizer routing and training dynamics; Appendix 4 provides comprehensive optimizer family analysis.

5. **Deployment and Inference**: Sections 6 and 7 describe quantized deployment and sentence-embedding workflows.

1.4 Web-Based Configuration Builder

In addition to direct YAML editing, this project provides a Streamlit-based web interface (accessed via `web-server` command) that improves configuration accessibility and discoverability. The interface dynamically generates schema-driven forms with inline parameter documentation, real-time YAML preview, and automatic CLI command generation for training, deployment, inference, and SBERT workflows. Schema metadata (titles and descriptions) are rendered systematically across all configuration sections, including model architecture, training runtime, optimizer hyperparameters, deployment, and SBERT-specific parameters. This approach reduces the need to memorize YAML structure, prevents typos through schema validation, and serves as both a configuration tool and a learning resource for users exploring novel architectures.

2 Related Work

This work sits at the intersection of deep learning frameworks, training tooling, and configuration-driven experimentation. While Sections 4.1 and 5 survey the architectural and optimization literature in depth, this section focuses on the software ecosystem for building, training, and configuring transformer models.

2.1 Base Deep Learning Frameworks

The modern transformer research ecosystem rests on several foundational frameworks that provide automatic differentiation, GPU acceleration, and high-level model-building APIs.

PyTorch [61] has become the dominant framework for transformer research. Its imperative programming model and dynamic computation graphs enable flexible debugging and rapid prototyping of novel architectures. The `torch.nn.Module` abstraction, automatic mixed-precision training via `torch.cuda.amp`, and the `torch.compile` JIT compiler form the backbone of most contemporary transformer implementations. Frankestein Transformer is built entirely on PyTorch, leveraging its module system for composable architecture definitions.

TensorFlow [1] pioneered production-oriented deep learning with its static computation graph paradigm and the Keras high-level API. While its influence in transformer research has waned relative to PyTorch, TensorFlow's serving infrastructure (`TF Serving`, `TF Lite`) and the `tensorflow-text` library remain relevant for deployment scenarios.

JAX [10] offers a functional programming model built around composable transformations (`jit`, `vmap`, `grad`) and XLA compilation. Its design enables clean implementations of complex algorithms and has been adopted by several recent transformer libraries (e.g., Flax, Haiku). JAX's stateless design and automatic vectorization are particularly well-suited for research on novel attention mechanisms and optimization algorithms.

scikit-learn [62] provides classical machine learning algorithms and utilities (preprocessing, metrics, model selection) that serve as baselines and complementary tools in transformer workflows. While not designed for deep learning, its consistent API and extensive documentation make it a standard reference point for comparing transformer-based approaches against traditional methods.

2.2 Out-of-the-Box Training Frameworks

Several libraries provide complete training pipelines that abstract away the complexity of distributed training, data loading, and hyperparameter management.

Hugging Face Transformers [83] is the de facto standard for accessing, fine-tuning, and sharing pretrained transformer models. Its `Trainer` API, model hub with over 500,000 models, and tight integration with the `datasets` and `tokenizers` libraries have made it the primary entry point for transformer-based NLP. However, its architecture is optimized for fine-tuning existing pretrained models rather than experimenting with novel sequence mixer designs or training from scratch with custom optimization strategies.

Oumi (https://github.com/oumi-ai/oumi, 2025) is an open-source end-to-end platform for training, fine-tuning, evaluating, and deploying foundation models. It provides a unified interface across multiple cloud providers and supports both supervised fine-tuning and preference optimization (DPO, RLHF). Oumi emphasizes production readiness with built-in monitoring, checkpointing, and deployment tooling.

LLaMA Factory (https://github.com/hiyouga/LLaMA-Factory, 2024) offers a unified framework for efficient fine-tuning of over 100 large language models. It provides both a web-based UI and a CLI, supporting LoRA, QLoRA, full-parameter fine-tuning, and various alignment techniques. Its focus is on adapting existing pretrained LLMs to downstream tasks rather than architectural experimentation.

Axolotl (https://github.com/axolotl-ai-cloud/axolotl, 2024) is a streamlined fine-tuning tool supporting LoRA, QLoRA, DeepSpeed, and FSDP. It emphasizes configuration-driven workflows via YAML files and has become popular in the open-source LLM fine-tuning community. Like LLaMA Factory, it targets fine-tuning of pretrained models rather than training novel architectures from scratch.

2.3 Low-Code and Efficiency-Oriented Tools

A growing category of tools prioritizes accessibility and computational efficiency, lowering the barrier to transformer training.

Unsloth (https://github.com/unslothai/unsloth, 2024) provides low-code fine-tuning with 2–5× speedup and significantly reduced VRAM requirements through hand-written CUDA kernels and optimized Triton operations. It supports GGUF export for quantized deployment and integrates with the Hugging Face ecosystem. Unsloth's focus is on making fine-tuning faster and more memory-efficient, not on architectural exploration.

dashAI (https://www.dash-ai.com, 2024) is a desktop application for no-code machine learning training with a schema-driven UI. It allows users to configure datasets, models, and training parameters through a graphical interface without writing code. While dashAI demonstrates the value of schema-driven configuration for accessibility, it targets classical ML workflows rather than transformer architecture experimentation.

2.4 Configuration-Driven Experimentation

A common thread across the tools surveyed above is their focus on fine-tuning existing pretrained models. Hugging Face Transformers, LLaMA Factory, Axolotl, and Unsloth all assume a pretrained checkpoint as the starting point. Oumi supports training from scratch but targets standard architectures. None of these tools provide a systematic framework for experimenting with novel sequence mixer architectures, comparing diverse optimization algorithms under controlled conditions, or exploring the interaction between architectural choices and training dynamics.

Frankestein Transformer fills this gap by providing a schema-first configuration system purpose-built for architecture experimentation. Its key differentiators include:

- **35+ sequence mixers** spanning dense attention, recurrent/retentive blocks, sparse attention patterns, gated mechanisms, adaptive depth routing, and conditional memory layers, all selectable via a single YAML field.
- **23 optimizer families** with a prefixed hyperparameter routing system that enables per-parameter-group control across embeddings, normalization layers, attention blocks, and feed-forward networks.
- **Dual training modes** supporting both encoder-style masked language modeling (MLM) and decoder-style autoregressive (AR) next-token prediction, with specialized fine-tuning workflows for each.
- **Strict schema validation** via `additionalProperties: false` constraints that prevent configuration errors before training begins, ensuring reproducibility across experiments.
- **End-to-end workflows** spanning quantized deployment (ternary weight packing, INT8 activations) and sentence-embedding training inspired by SBERT [68].
- **Web-based configuration interface** providing schema-driven form rendering with inline documentation, real-time validation, and CLI command generation.

By decoupling architecture definition from training infrastructure through a validated configuration contract, Frankestein Transformer enables systematic comparison of sequence mixer families and optimization strategies under identical training conditions—a capability not provided by any existing tool in the ecosystem.

3 System Design and Architecture

3.1 Configuration-Centric Architecture

The core design choice in this repository is that experimentation is *schema first*. Instead of exposing a large number of loosely checked flags, the project forces model topology, optimizer family, training limits, and telemetry options through a single validated configuration document. This reduces ambiguity when reproducing results and makes it possible to compare many architectures under a consistent operational interface.

The authoritative contract is `configs/schema.yaml`. It enforces three top-level objects:

- `model_class`
- `model`
- `training`

Model Class Selection. The `model_class` field determines the architectural variant instantiated by the training pipeline. Three options are supported:

- **frankenstein**: Mixed-architecture encoder models supporting diverse attention mechanisms (standard, sigmoid, retentive, state-space, sparse, and gated mixers) with MoE (Mixture of Experts) and advanced features. Optimized for bidirectional encoder-style training with masked language modeling (MLM) objectives.
- **mini**: Simplified encoder variant designed for smaller-scale training scenarios. Provides reduced parameter overhead and faster iteration for experimentation and prototyping.
- **frankesteindecoder**: Autoregressive causal decoder for LLM-style next-token generation. Enables causal attention masking for sequential text generation tasks. When this class is selected, runtime enforces `mode='decoder'`.

Training Mode Selection. The `model.mode` field controls attention masking behavior across the model:

- **encoder**: Uses bidirectional attention where all tokens attend to all other tokens in the sequence. Suitable for masked language modeling (MLM) pre-training tasks where the model learns to predict randomly masked tokens based on full context.
- **decoder**: Uses causal masking where each token can only attend to previous tokens in the sequence. Required for autoregressive (AR) next-token prediction tasks such as language modeling and text generation. When `model_class='frankesteindecoder'`, the system automatically forces `mode='decoder'` at runtime.

This dual architecture support enables the system to handle both encoder-style pre-training (MLM on bidirectional contexts) and decoder-style generation (causal autoregressive prediction) through a unified configuration interface.

Sequence Mixer Pattern Selection: The `layer_pattern` field accepts an ordered list of mixer identifiers spanning six families: dense baselines (`standard_attn`, `sigmoid_attn`, `gqa`), recurrent/retentive (`retnet`, `retnet_attn`, `mamba`, `ode`, `titan_attn`), sparse attention (`sparse_transformer_attn`, `longformer_attn`, `bigbird_attn`, `sparsek_attn`, `nsa_attn`, `sparge_attn`, `fasa_attn`, `msa_attn`, `sparda_attn`), gated mechanisms (`gla_attn`, `deltanet_attn`, `gated_deltanet_attn`, `gated_deltanet2_attn`, `hgrn2_attn`, `fox_attn`, `gated_softmax_attn`, `kda_attn`), latent compression (`mla_attn`, `gqla_attn`, `mlra_attn`, `tucker_attn`, `iha_attn`, `gta_attn`, `mtla_attn`, `cca_attn`, `ccgqa_attn`), and conditional memory (`engram_attn`). The complete taxonomy with references is provided in Section 4 and Appendix 3.

The `training.optimizer.optimizer_class` supports a broad optimizer family: `sgd_momentum`, `adamw`, `adafactor`, `galore_adamw`, `prodigy`, `lion`, `sophia`, `muon`, `turbo_muon`, `radam`, `adan`, `adopt`, `ademamix`, `mars_adamw`, `cautious_adamw`, `lamb`, `schedulefree_adamw`, `shampoo`, `soap`, `anon`, `apollo`, `apollo_mini`, and `q_apollo`.

3.2 Schema Scope and Validation Rules

The schema is strict: top-level and nested objects set `additionalProperties: false`. This guarantees that unknown keys fail fast instead of being silently ignored. The `training.optimizer.parameters` object is additionally constrained by optimizer-specific prefix rules through `allOf`+`if/then` pattern checks.

Normalization values currently accepted by schema are:

$$\texttt{norm\_type} \in \{\texttt{layer\_norm}, \texttt{dynamic\_tanh}, \texttt{derf}, \texttt{rms\_norm}, \texttt{prms\_norm}\}$$

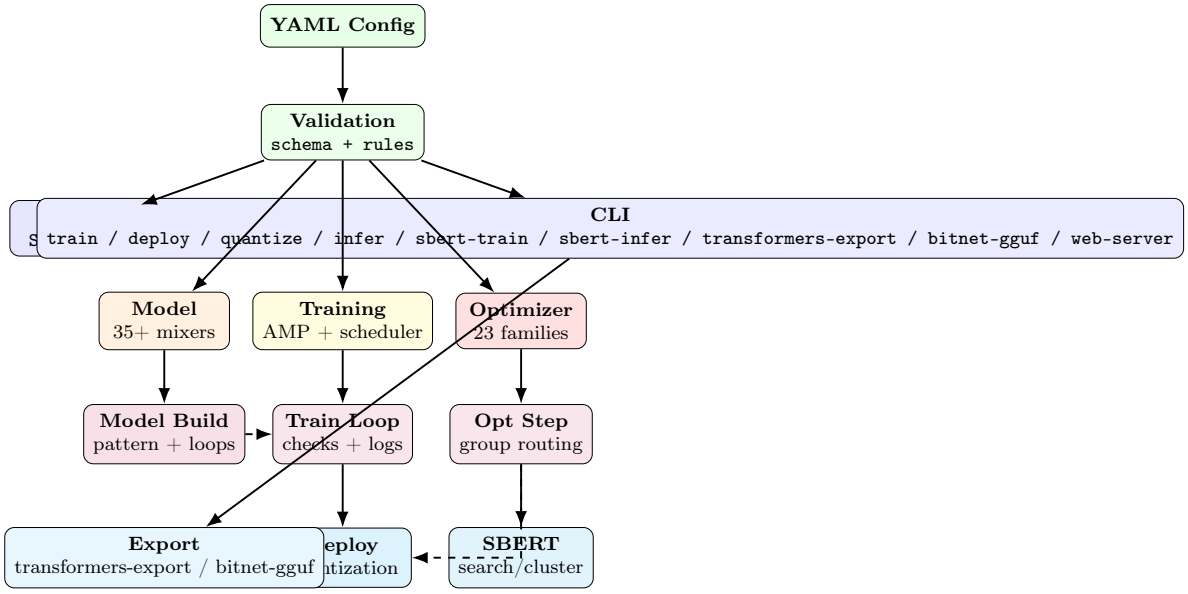

Figure 1: System architecture: configuration flows through validation, partitions into model/training/optimizer, executes runtime, produces deployment/SBERT artifacts.

3.3 Configuration Schema

The complete model and training configuration schema, including all fields, types, ranges, and validation rules, is documented in Appendix 2. The schema enforces `additionalProperties: false` at all levels, guaranteeing that unknown keys fail fast.

Looped depth induced by schema is:

$$L_{\text{logical}} = \texttt{num\_layers} \times \texttt{num\_loops}$$

which is the configuration-level definition of looped blocks.

3.4 Training Task Types

The `training.task` field determines the training objective, working in conjunction with `model.mode` to define how the model learns:

Masked Language Modeling (MLM).

- **Mode**: Requires `mode='encoder'` for bidirectional attention.
- **Objective**: Randomly mask tokens in input sequence (typically 15%) and train model to predict masked tokens based on full bidirectional context.
- **Use Case**: Pre-training encoders for representation learning, following BERT-style methodology [20]. Model learns bidirectional representations capturing context from both left and right.
- **Configuration**: Uses `mlm_probability` parameter to control masking fraction.

Autoregressive (AR) Next-Token Prediction.

- **Mode**: Requires `mode='decoder'` for causal masking.
- **Objective**: Train model to predict next token in sequence given all previous tokens only.

- **Use Case**: Language generation and LLM-style tasks following GPT methodology. Model learns sequential dependencies with causal attention where each token can only attend to preceding tokens.
- **Model Class**: Typically uses `model_class='frankesteindecoder'` for autoregressive decoder architectures.

This dual task support enables unified experimentation across both encoder-style pre-training (MLM for bidirectional understanding) and decoder-style generation (AR for sequential text production) within the same codebase.

3.5 Optimizer Configuration

The `training.optimizer` object selects among 23 optimizer families via `optimizer_class` and provides per-parameter-group hyperparameter control through a prefixed naming convention. The complete optimizer prefix contract and supported families are documented in Appendix 2.

3.6 Training Safety and Runtime Semantics

Schema-level safety features include accumulation, clipping, post-clip explosion checks, and NaN retries:

$$g_{\text{acc}} = \frac{1}{K}\sum_{i=1}^{K} g_i, \quad K = \texttt{gradient\_accumulation\_steps}$$

$$g_{\text{clip}} = g_{\text{acc}} \cdot \min\left(1, \frac{\tau}{\|g_{\text{acc}}\|_2 + \epsilon}\right), \quad \tau = \texttt{grad\_clip\_max\_norm}$$

then overflow guards use `inf_post_clip_threshold` and retry logic bounded by `max_nan_retries`.

The complete pseudocode for the schema-driven training step—including gradient accumulation, global-norm clipping, post-clip explosion guards, bounded NaN/Inf retries, scheduler dispatch, and rolling/best checkpoint rotation—is deferred to Appendix 9.

3.7 Normalization Variants

Normalization controls activation scale across depth and is also a schema compatibility question, since the accepted `norm_type` values are `layer_norm`, `dynamic_tanh`, `derf`, `rms_norm`, and `prms_norm` (partial RMSNorm). The mathematical formulations of LayerNorm, RMSNorm, pRMSNorm, Dynamic Tanh (DyT) [97], and Dynamic Erf (Derf) [12], together with a comparison table, are detailed in Appendix 9.

4 Architecture Taxonomy and Implementation

4.1 Attention and Sequence-Mixer Families

This system implements thirty-five sequence mixer variants organized into six functional categories reflecting research trends in sequence modeling design. The taxonomical organization reflects evolving understanding of how to balance expressivity, computational efficiency, and memory constraints.

1. **Dense Attention Baselines** (3): Standard softmax attention, sigmoid attention, and Grouped-Query Attention (GQA) provide full global contextualization at quadratic computational cost, serving as reference baselines for comparison with more efficient alternatives.

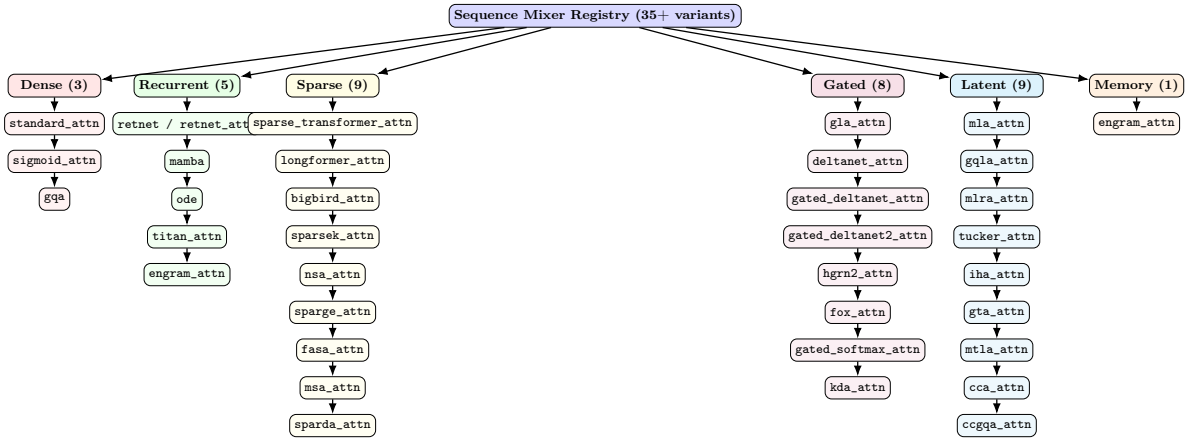

Figure 2: Comprehensive taxonomy of thirty-five sequence mixer variants across six functional categories. Dense baselines provide full global routing at quadratic cost. Recurrent architectures enable constant-time inference through state compression. Sparse variants reduce complexity through structured patterns or token selection. Gated mechanisms introduce data-dependent control over memory retention and forgetting. Latent methods compress key–value representations into low-rank bottlenecks. Conditional memory provides test-time read/write over an external memory bank.

2. **Recurrent and Retentive Architectures** (5): RetNet, Mamba, ODE-style blocks, Titans, and Engram maintain state representations enabling $\mathcal{O}(1)$ inference cost while preserving expressivity through recurrent dynamics, selective parameters, or test-time memory adaptation.

3. **Sparse Attention Patterns** (9): Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA, MSA, and SparDA reduce quadratic complexity through structured sparsity, token selection, or training-free pruning strategies.

4. **Gated Memory Mechanisms** (8): GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax, and KDA introduce data-dependent control over memory retention, forgetting, and update strength.

5. **Latent and Low-Rank Compression** (9): MLA, GQLA, MLRA, Tucker, IHA, GTA, MTLA, CCA, and CCGQA compress key–value representations into low-rank latent bottlenecks, trading a small reconstruction error for substantial memory and compute savings.

6. **Conditional Memory** (1): Engram provides test-time memory through learned read/write operations over an external memory bank.

4.2 Dense and Recurrent Families

The dense baselines (standard softmax [79] and sigmoid [67]) and the recurrent/retentive family (RetNet [76], Mamba [31], ODE-style continuous blocks [93], and Titans test-time memory [7]) provide the foundational expressiveness-versus-efficiency tradeoff. Mathematical formulations, parallel and recurrent forms, algorithmic pseudocode, and an architectural comparison table are collected in Appendix 5. The pattern-driven mixer dispatch algorithm (including the training-free enforcement policy for `fasa_attn` and `sparge_attn`) is likewise detailed there.

4.3 Latent and Low-Rank Attention Family

A complementary family compresses the per-token key–value state into a low-rank latent: Multi-Head Latent Attention (MLA), Group-Query Latent Attention (GQLA), Multi-Head Low-Rank

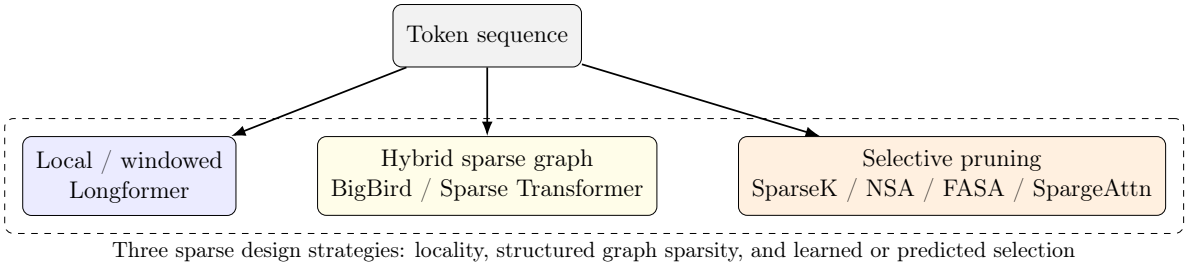

Figure 3: Conceptual map of sparse attention design choices used in the codebase. Different methods reduce cost by restricting neighborhoods, constructing sparse graphs, or selecting only high-value tokens/blocks.

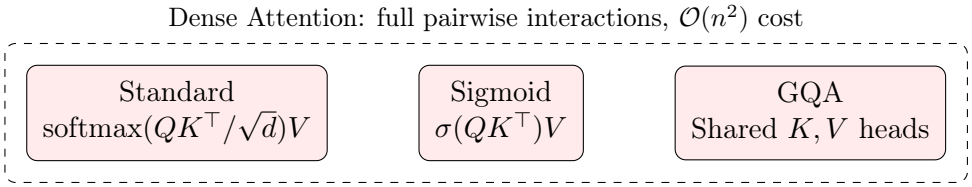

Figure 4: Three dense attention baselines. Standard softmax and sigmoid attention compute full $QK^\top$ matrices. Grouped-Query Attention (GQA) shares key–value heads across query groups, reducing memory bandwidth while preserving full attention expressivity.

Attention (MLRA), Tucker Attention, Interleaved Head Attention (IHA), Grouped-head laTenT Attention (GTA), and Multi-head Temporal Latent Attention (MTLA). These methods unify grouped-query, latent, factorised, and temporal-latent designs under a common low-rank bottleneck. Full formulations, pseudocode, and a comparison table are collected in Appendix 6.

4.4 Sparse Attention Extensions

The mixer registry includes nine sparse blocks: `sparse_transformer_attn`, `longformer_attn`, `bigbird_attn`, `sparsek_attn`, `nsa_attn`, `sparge_attn`, `fasa_attn`, `msa_attn`, and `sparda_attn` [15, 8, 91, 52, 89, 94, 82, 43, 26]. The implementation enforces an explicit execution policy for training-free sparse methods: `fasa_attn` and `sparge_attn` are eval/inference-only and raise runtime errors if used while the model is in training mode. Mathematical formulations, algorithmic pseudocode, per-block characteristics, and a comparison table are collected in Appendix 7.

4.5 Gated Attention Extensions

The mixer registry includes eight gated blocks: `gla_attn`, `deltanet_attn`, `gated_deltanet_attn`, `gated_deltanet2_attn`, `hgrn2_attn`, `fox_attn`, `gated_softmax_attn`, and `kda_attn` [85, 86, 87, 34, 64, 47, 65, 40]. The unifying idea is that gating controls *what information survives*: some gates act on recurrent state updates (GLA, DeltaNet variants, HGRN2), while others modify the full attention path itself (FoX and Gated Softmax). KDA introduces kernelized dynamic attention with learnable gating over the kernel bandwidth. This makes gating especially useful when the model must trade off recall, recency, and bounded memory. Mathematical formulations, algorithmic pseudocode, a generic gating template, and a comparison table are collected in Appendix 8.

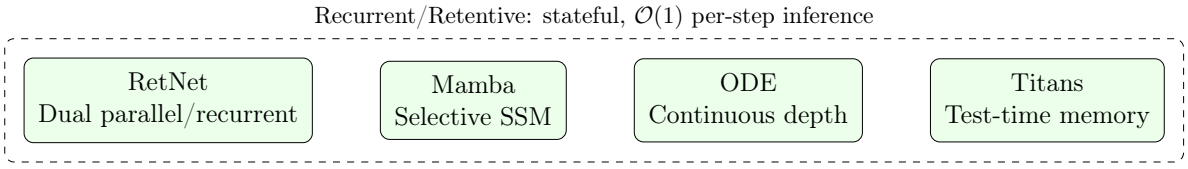

Figure 5: Recurrent and retentive architecture family. RetNet provides dual parallel and recurrent forms. Mamba uses selective state-space parameters. ODE blocks model continuous-depth dynamics. Titans introduce test-time memory adaptation. All achieve constant-time per-step inference.

5 Optimizer Families and Training Dynamics

Optimization of highly parameterized transformer architectures presents significant challenges due to non-convex loss landscapes, saddle points, and block heterogeneity across parameter groups. This system addresses these challenges through a unified framework supporting twenty-three optimizer families spanning six algorithmic categories: (1) classical baselines (SGD, AdamW), (2) advanced momentum and variance reduction (Adan, ADOPT, AdEMAMix, MARS, Cautious), (3) memory-efficient variants (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO), (4) schedule-free and parameter-free methods (Schedule-Free AdamW, Prodigy) plus large-batch scaling through LAMB, (5) curvature-aware and second-order (Sophia, Shampoo, SOAP), (6) geometry-oriented (Muon, Turbo-Muon), and (7) adaptivity-tunable optimizers (Anon). Detailed algorithmic descriptions, pseudocode, and a memory/complexity comparison table for all optimizers are provided in Appendix 4.

6 Quantization and Deployment

The deploy stack uses ternary weight packing plus INT8 activation quantization for efficient artifacts.

6.1 Ternary Quantization and Activation Scaling

The deploy-stage transforms a trained checkpoint into a compact artifact via BitNet-style ternary weight packing [81], INT8 activation scaling, and 1.58-bit storage approximation aligned with lightweight deployment goals [70, 11]. The mathematical formulations of the ternary weight scaling, the INT8 activation quantization/dequantization, and the FP32/FP16/1.58-bit size estimates are deferred to Appendix 9.

7 SBERT Downstream Tasks

Sentence embedding is built on Siamese-style training [68]. For sentence pair (s_1, s_2) with embeddings (e_1, e_2):

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\|\|e_2\|}$$

and regression-style cosine loss:

$$\mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

with $y \in [-1, 1]$ in this pipeline.

Supported downstream modes:

- **Similarity**: pairwise score between two sentences.

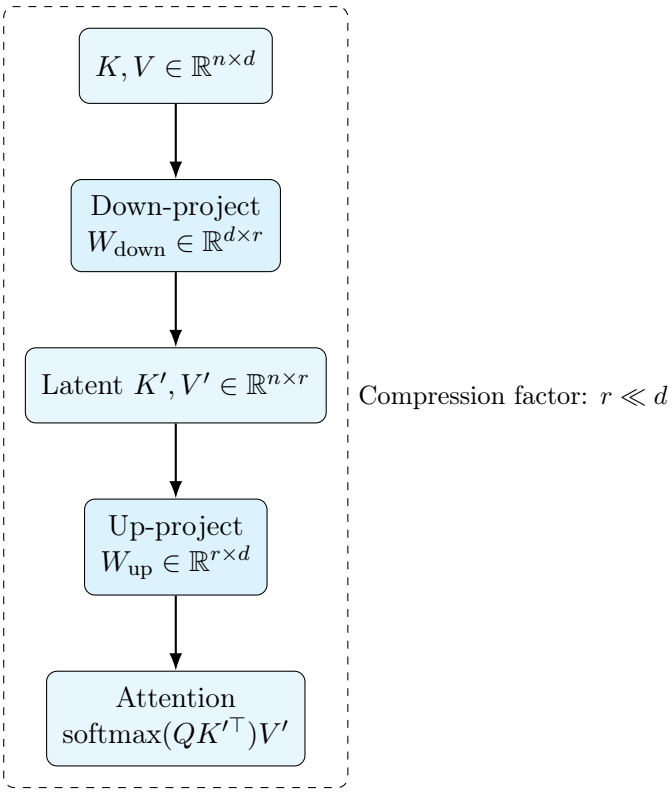

Figure 6: Latent and low-rank attention concept. Key–value pairs are compressed through a down-projection into a low-rank latent space ($r \ll d$), where attention is computed at reduced cost, then up-projected back. Variants differ in factorization structure (MLA, GQLA, MLRA, Tucker), head interleaving (IHA, GTA), temporal compression (MTLA), and cross-covariance formulations (CCA, CCGQA).

- **Search**: top-k nearest neighbors over a corpus.
- **Cluster**: grouping embeddings (e.g., k-means).
- **Encode**: persistent embedding export for later retrieval.

The pseudocode for the SBERT inference mode router—dispatching among similarity, search, cluster, and encode modes over a shared encoder—is deferred to Appendix 9.

8 Discussion

The design choices in Transformer Encoder Frankenstein reflect several engineering and research tensions in modern deep learning tooling.

8.1 Schema-Driven Design Trade-offs

The schema-first approach provides significant reproducibility benefits by enforcing explicit contracts and failing fast on invalid configurations. However, this approach also introduces rigidity: adding new architectures or optimizers requires schema extensions rather than loose command-line arguments. The prefixed hyperparameter system enables fine-grained control but increases configuration complexity for users accustomed to simpler interfaces.

The decision to enforce `additionalProperties: false` at all schema levels eliminates silent parameter swallowing that has plagued earlier configuration systems, but this strictness

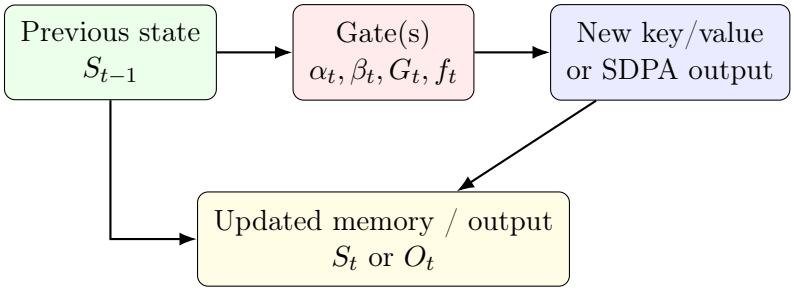

Figure 7: Generic gating template. A gate can decay existing memory, regulate write strength, or modulate dense attention outputs, depending on the block family.

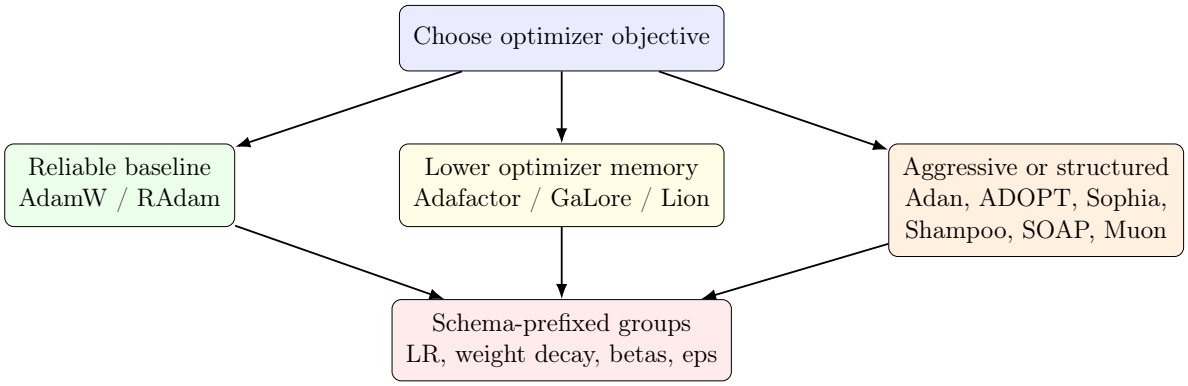

Figure 8: Optimizer selection in practice: the class determines the update rule, while the schema controls how hyperparameters are applied across embeddings, norms, recurrent blocks, attention blocks, and other parameters.

requires careful schema maintenance when extending system capabilities. Each new attention mechanism or optimizer variant must be properly integrated into the validation framework, including schema field definitions with appropriate types and constraints, prefixed hyperparameter mapping for optimizer-specific groups, default values aligned with research best practices, and documentation strings for web interface rendering.

8.2 Architectural Coverage and Gaps

The seventeen implemented mixer architectures span major research directions in sequence modeling, but certain gaps remain. The system lacks recent hybrid architectures such as Griffin [74] and Jamba [32], which combine gating with state-space models. MoE (Mixture of Experts) routing is implemented for FFN layers but not for attention computation, where recent work has shown benefits [45].

The sparse attention coverage is comprehensive, but implementation of training-free methods (FASA, SpargeAttn) raises runtime errors during training, reflecting architectural constraints: these methods require pretrained checkpoints from full-attention models or specific fine-tuning procedures that are not currently automated.

The gated mechanism coverage is strong across major categories (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax).

8.3 Optimizer Landscape Fragmentation

The support for twenty-two optimizer families across six algorithmic categories demonstrates comprehensiveness but also highlights the fragmented state of optimization research. Users face significant decision complexity when choosing among variance-reduction methods (Adan,

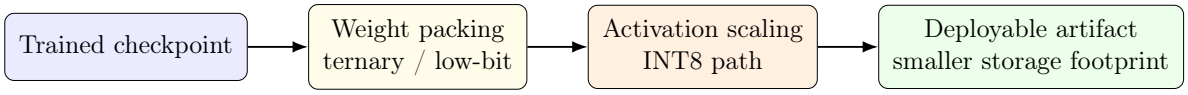

Figure 9: Deployment path from a trained checkpoint to a compact artifact. The codebase treats quantization as a deploy-stage transformation rather than a separate model family.

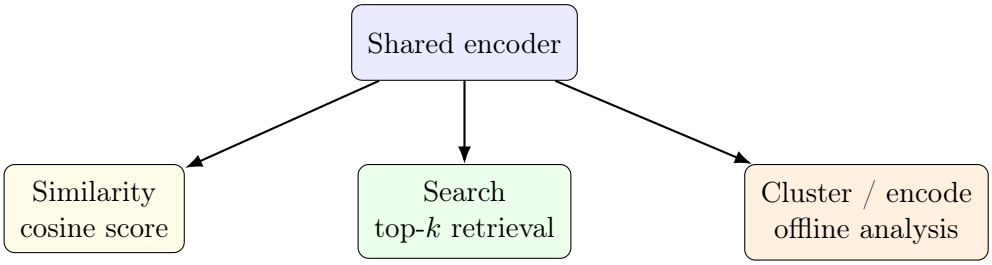

Figure 10: SBERT workflow reuse. A single encoder supports online pair scoring, corpus retrieval, clustering, and persistent embedding export.

MARS), memory-efficient variants (GaLore, Adafactor, APOLLO), and curvature-aware approaches (Sophia, Shampoo). The prefixed hyperparameter system, while powerful, requires understanding of which parameters are relevant for each optimizer class.

The implementation quality varies across optimizers: classical methods (AdamW, SGD with momentum) are highly optimized in PyTorch, while newer methods (Muon, Turbo-Muon, SOAP) may require custom implementations that affect numerical stability and performance characteristics.

8.4 Deployment and Production Considerations

The quantization pipeline demonstrates practical deployment concerns but makes specific engineering trade-offs. Ternary weight packing reduces storage to approximately 1.58 bits per parameter, but this aggressive compression may degrade performance, especially for smaller models where quantization error is more significant. The current implementation applies quantization uniformly across parameter types.

The SBERT workflows provide practical utility for semantic similarity and retrieval tasks, but implementation assumes standard pooling strategies (CLS token, mean pooling). Recent advances such as Matryoshka embeddings [58] and contrastive learning refinements [39] are not yet incorporated.

8.5 Integration and Extensibility Challenges

The current codebase structure, while functional, presents maintenance challenges as architecture and optimizer families expand. The dispatcher pattern for mixer selection and optimizer routing handles extensibility but risks becoming a "kitchen sink" of conditional logic. Future versions would benefit from plugin-based architectures where new mixers and optimizers could be registered declaratively rather than modifying core dispatch logic.

The web configuration interface provides significant usability improvements but introduces deployment complexity: running Streamlit alongside training jobs requires additional resources and infrastructure considerations that may not be appropriate for all environments, particularly HPC clusters without web access.

9 Conclusion

Transformer Encoder Frankenstein presents a unified, configuration-driven experimentation platform addressing critical challenges in modern deep learning research: architectural fragmentation across dense attention, recurrent models, sparse patterns, and gated mechanisms; optimizer landscape complexity spanning classical baselines, variance-reduction methods, memory-efficient variants, schedule-free approaches, curvature-aware algorithms, and geometry-oriented methods; and end-to-end deployment workflows spanning quantization and sentence embedding applications.

The system's primary contributions are:

1. **Schema-First Design**: A strict YAML-based configuration contract with validation and prefixed hyperparameter routing enabling reproducible experiments across seventeen mixer architectures and twenty-three optimizer families.

2. **Comprehensive Architecture Support**: Implementation spanning major research categories including dense baselines (standard, sigmoid attention), recurrent alternatives (RetNet, Mamba, ODE-style, Titans), sparse attention (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA), and gated mechanisms (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax).

3. **Unified Optimizer Framework**: Prefixed hyperparameter groups enabling fine-grained control over embeddings, normalization layers, recurrent blocks, attention weights, and FFN parameters across classical baselines (SGD+Momentum, AdamW), variance-reduction (Adan, ADOPT, AdEMAMix, MARS, Cautious), memory-efficient (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO), large-batch and schedule simplification (LAMB, Schedule-Free AdamW, Prodigy), curvature-aware (Sophia), second-order (Shampoo, SOAP), and geometry-oriented (Muon, Turbo-Muon) optimizers.

4. **End-to-End Workflows**: Integrated deployment pipeline supporting ternary weight packing and INT8 activation quantization; SBERT-inspired training and inference for semantic similarity, retrieval, and clustering tasks.

5. **Interactive Configuration**: Streamlit-based web interface providing schema-driven form generation, real-time validation, inline documentation, and CLI command synthesis.

This system enables rapid experimental iteration while maintaining reproducibility through strict configuration contracts. By consolidating diverse research contributions into a unified toolkit, it lowers barriers to exploring novel architectures and optimization strategies, particularly for researchers who may lack resources to implement and validate each variant independently.

9.1 Limitations and Future Directions

Several limitations and promising directions for future work emerge from this system's design and implementation:

1. **Architecture Integration**: Recent hybrid architectures (Griffin, Jamba, Mamba-X) demonstrate benefits of combining multiple mechanisms into unified blocks. Future versions should integrate these architectures and explore systematic composition patterns.

2. **Advanced Quantization**: Current implementation uses uniform ternary packing across all parameters. Research on layer-wise, channel-wise, and importance-aware quantization suggests more sophisticated strategies could improve quality-efficiency trade-offs.

3. **Plugin-Based Extensibility**: The current dispatch pattern becomes increasingly complex with each new addition. A plugin architecture allowing declarative registration of new mixers, optimizers, and normalization methods would improve maintainability and reduce risk of bugs in core dispatch logic.

4. **Automated Hyperparameter Optimization**: The schema supports extensive hyperparameter spaces, but users must manually explore these spaces. Integration with Bayesian optimization, multi-armed bandit strategies, or gradient-based hyperparameter tuning could automate effective configuration discovery.

5. **Production Deployment**: The web interface improves usability but may not be appropriate for all deployment environments. Headless configuration modes, API-based configuration management, or improved CLI ergonomics could serve HPC and production workflows.

6. **Evaluation Benchmarking**: While the system enables training with diverse architectures, comprehensive benchmarking comparing performance across mixers and optimizers on standardized tasks would provide valuable guidance for configuration selection.

7. **Training Stability Guarantees**: Current implementation includes NaN/Inf guards and gradient clipping, but formal analysis of stability conditions for different mixer-optimizer combinations, particularly with looped blocks and aggressive quantization, remains open.

8. **Multimodal and Task-Specific Extensions**: The current design focuses on sequence modeling. Extensions for vision-language models, multimodal architectures, and task-specific fine-tuning workflows (e.g., instruction tuning, RLHF) would broaden applicability.

The research trajectory of sequence modeling continues toward hybrid approaches that combine strengths of multiple paradigms—compression from recurrence, selectivity from attention, gating for memory management, and sparsity for efficiency. A unified experimentation platform like Transformer Encoder Frankenstein is increasingly valuable as this convergence accelerates, enabling researchers to systematically explore this expanding design space with reproducible, well-engineered infrastructure.

Bibliography

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 265–283, 2016.

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyansky, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL https://arxiv.org/abs/2305.13245.

[3] Anonymous authors. Anon: Extrapolating adaptivity beyond sgd and adam, 2026. URL https://arxiv.org/abs/2605.02317.

[4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL https://arxiv.org/abs/1607.06450.

[5] Jonathan T. Barron. Continuously differentiable exponential linear units. *arXiv preprint arXiv:1704.07483*, 2017.

[6] Mina Basirat and Peter M. Roth. The quest for a better ReLU: ELiSH and HardELiSH activation functions. *arXiv preprint arXiv:1905.10144*, 2019.

[7] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. URL https://arxiv.org/abs/2501.00663. Version Number: 1.

[8] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL https://arxiv.org/abs/2004.05150.

[9] Thibaut Boissin, Thomas Massena, Franck Mamalet, and Mathieu Serrurier. Turbo-muon: Accelerating orthogonality-based optimization with pre-conditioning. URL https://arxiv.org/abs/2512.04632. Version Number: 1.

[10] James Bradbury, Roy Frostig, Peter Hawkins, Matthew J. Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/jax-ml/jax.

[11] Riccardo Bravin, Massimo Pavan, Hazem Hesham Yousef Shalby, Fabrizio Pittorino, and Manuel Roveri. EmbBERT: Attention under 2 MB memory. URL http://arxiv.org/abs/2502.10001.

[12] Mingzhi Chen, Taiming Lu, Jiachen Zhu, Mingjie Sun, and Zhuang Liu. Stronger normalization-free transformers, . URL http://arxiv.org/abs/2512.10938.

[13] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms, . URL https://arxiv.org/abs/2302.06675. Version Number: 4.

[14] Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026. URL https://arxiv.org/abs/2601.07372.

[15] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019. URL https://arxiv.org/abs/1904.10509.

[16] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). *arXiv preprint arXiv:1511.07289*, 2016.

[17] Chang Dai, Hongyu Shan, Mingyang Song, and Di Liang. HoPE: Hyperbolic rotary positional encoding for stable long-range dependency modeling in large language models. URL http://arxiv.org/abs/2509.05218.

[18] Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. URL https://arxiv.org/abs/2405.15682. Version Number: 4.

[19] Keqi Deng and Philip C. Woodland. Multi-head temporal latent attention, 2025. URL https://arxiv.org/abs/2505.13544.

[20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. URL http://arxiv.org/abs/1810.04805.

[21] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, 453:1–24, 2021. doi: 10.1016/j.neucom.2021.03.063.

[22] Sai Surya Duvvuri, Chanakya Ekbote, Rachit Bansal, Rishabh Tiwari, Devvrit Khatri, David Brandfonbrener, Paul Liang, Inderjit Dhillon, and Manzil Zaheer. Interleaved head attention, 2026. URL https://arxiv.org/abs/2602.21371.

[23] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 106:300–312, 2018. doi: 10.1016/j.neunet.2018.07.012.

[24] Aiming Fang, Suhyune Lee, Nafise Sadat Moosavi, and Iryna Gurevych. Transformers with learnable activation functions. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 1736–1746, 2023. doi: 10.48550/arXiv.2208.14111.

[25] Tomas Figliola, Nicholas Alonso, Rishi Iyer, Quentin Anthony, and Beren Millidge. Compressed convolutional attention: Efficient attention in a compressed latent space, 2025. URL https://arxiv.org/abs/2510.04476.

[26] Yaosheng Fu, Guangxuan Xiao, Xin Dong, Song Han, and Oreste Villa. Sparda: Sparse decoupled attention for efficient long-context llm inference, 2026. URL https://arxiv.org/abs/2606.04511.

[27] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 315–323, 2011.

[28] Luke B. Godfrey and Michael S. Gashler. A continuum among logarithmic, linear, and exponential functions, and its potential to improve generalization in neural networks. In *Proceedings of the International Conference on Knowledge Discovery and Information Retrieval (KDIR)*, 2015.

[29] Ian J. Goodfellow, David Warde-Farley, Mehdi Mirza, Aaron Courville, and Yoshua Bengio. Maxout networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pages 1319–1327, 2013.

[30] Nils Graef, Filip Makraduli, Andrew Wasielewski, and Matthew Clapp. FlashNorm: Fast normalization for transformers. URL http://arxiv.org/abs/2407.09577.

[31] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. URL https://arxiv.org/abs/2312.00752. Version Number: 2.

[32] Albert Gu, AI21 Labs, et al. Jamba: A hybrid transformer-mamba language model, 2024. URL https://arxiv.org/abs/2403.19887.

[33] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. URL https://arxiv.org/abs/1802.09568. Version Number: 2.

[34] Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated deltanet-2: Decoupling erase and write in linear attention, 2026. URL https://arxiv.org/abs/2605.22791.

[35] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[36] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELU). In *arXiv preprint arXiv:1606.08415*, 2016.

[37] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1314–1325, 2019.

[38] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. In *arXiv preprint arXiv:1704.04861*, 2017.

[39] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning, 2020. URL https://arxiv.org/abs/2004.11362.

[40] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture, 2025. URL https://arxiv.org/abs/2510.26692.

[41] Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. Self-normalizing neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[42] Timon Klein, Jonas Kusch, Sebastian Sager, Stefan Schnake, and Steffen Schotthöfer. Tucker attention: A generalization of approximate attention mechanisms, 2026. URL https://arxiv.org/abs/2603.30033.

[43] Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, and Pengyu Zhao. Minimax sparse attention, 2026. URL https://arxiv.org/abs/2606.13392.

[44] Johannes Lederer. Activation functions in artificial neural networks: A systematic overview. *arXiv preprint arXiv:2101.09957*, 2021. doi: 10.48550/arXiv.2101.09957.

[45] Mike Lewis, Shruti Bhosale, Tim Dettmers, Douwe Kiela, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models, 2021. URL https://arxiv.org/abs/2103.16716.

[46] Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious optimizers: Improving training with one line of code. URL https://arxiv.org/abs/2411.16085. Version Number: 4.

[47] Zhixuan Lin, Ke Wang, et al. Forgetting transformer: Softmax attention with a forget gate, 2025. URL https://arxiv.org/abs/2503.02130.

[48] Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training, . URL https://arxiv.org/abs/2305.14342. Version Number: 4.

[49] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond, . URL https://arxiv.org/abs/1908.03265. Version Number: 4.

[50] Songtao Liu, Hongwu Peng, Zhiwei Zhang, Zhengyu Chen, and Yue Guo. Multi-head low-rank attention, 2026. URL https://arxiv.org/abs/2603.02188.

[51] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. URL https://arxiv.org/abs/1711.05101. Version Number: 3.

[52] Tianyu Lou, Zheyu Chen, Tao Yu, et al. Efficient sparse attention for long-range transformers, 2024. URL https://arxiv.org/abs/2406.16747.

[53] Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *Proc. ICML Workshop on Deep Learning for Audio, Speech and Language Processing*, 2013.

[54] Sushant Mehta, Raj Dandekar, Rajat Dandekar, and Sreedath Panat. Latent multi-head attention for small language models, 2025. URL https://arxiv.org/abs/2506.09342.

[55] Fanxu Meng. Gqla: Group-query latent attention for hardware-adaptive large language model decoding, 2026. URL https://arxiv.org/abs/2605.15250.

[56] Konstantin Mishchenko and Aaron Defazio. Prodigy: An expeditiously adaptive parameter-free learner. URL https://arxiv.org/abs/2306.06101. Version Number: 4.

[57] Diganta Misra. Mish: A self regularized non-monotonic activation function. *arXiv preprint arXiv:1908.08681*, 2019.

[58] Niklas Muennighoff et al. Matryoshka representation learning, 2022. URL https://arxiv.org/abs/2205.13147.

[59] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.

[60] Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix optimizer: Better, faster, older. URL https://arxiv.org/abs/2409.03137. Version Number: 2.

[61] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035, 2019.

[62] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[63] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi: 10.1016/0041-5553(64)90137-5.

[64] Zhen Qin, Xu Han, et al. Hgrn2: Gated linear rnns with state expansion, 2024. URL https://arxiv.org/abs/2404.07904.

[65] Yuxiang Qiu, Qwen Team, et al. Gated attention for large language models, 2025. URL https://arxiv.org/abs/2505.06708.

[66] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. In *arXiv preprint arXiv:1710.05941*, 2017.

[67] Jason Ramapuram, Federico Danieli, Eeshan Dhekane, Floris Weers, Dan Busbridge, Pierre Ablin, Tatiana Likhomanenko, Jagrit Digani, Zijin Gu, Amitis Shidani, and Russ Webb. Theory, analysis, and best practices for sigmoid self-attention. URL https://arxiv.org/abs/2409.04431. Version Number: 2.

[68] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. URL http://arxiv.org/abs/1908.10084.

[69] Susmita Roy, Souvik Manna, Subhash Bagui, et al. LiSHT: Non-parametric linearly scaled hyperbolic tangent activation function for neural networks. *arXiv preprint arXiv:1811.05870*, 2019.

[70] Hema Hariharan Samson. Lightweight transformer architectures for edge devices in real-time applications. URL http://arxiv.org/abs/2601.03290.

[71] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[72] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. URL https://arxiv.org/abs/1804.04235. Version Number: 1.

[73] Wei Shen, Ruichuan Huang, Minhui Huang, Cong Shen, and Jiawei Zhang. On the convergence analysis of muon. URL https://arxiv.org/abs/2505.23737. Version Number: 1.

[74] Rafael Soares et al. Griffin: Mixing gated linear recurrences with local attention for efficient sequence modeling, 2024. URL https://arxiv.org/abs/2402.19427.

[75] Luoyang Sun, Cheng Deng, Jiwen Jiang, Xinjian Wu, Haifeng Zhang, Lei Chen, Lionel Ni, and Jun Wang. Gta: Grouped-head latent attention, 2025. URL https://arxiv.org/abs/2506.17286.

[76] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. URL https://arxiv.org/abs/2307.08621. Version Number: 4.

[77] Shohei Taniguchi, Keno Harada, Gouki Minegishi, Yuta Oshima, Seong Cheol Jeong, Go Nagahara, Tomoshi Iiyama, Masahiro Suzuki, Yusuke Iwasawa, and Yutaka Matsuo. ADOPT: Modified adam can converge with any β_2 with the optimal rate. URL https://arxiv.org/abs/2411.02853. Version Number: 3.

[78] Ludovic Trottier, Philippe Giguère, and Brahim Chaib-draa. Parametric exponential linear unit for deep convolutional neural networks. In *Proceedings of the 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 207–214, 2017. doi: 10.1109/ICMLA.2017.0-133.

[79] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. URL https://arxiv.org/abs/1706.03762. Version Number: 7.

[80] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing shampoo using adam. URL https://arxiv.org/abs/2409.11321. Version Number: 2.

[81] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. URL http://arxiv.org/abs/2310.11453.

[82] Zhe Wang, Ming Liu, et al. Fasa: Frequency-aware sparse attention, 2026. URL https://arxiv.org/abs/2602.03152.

[83] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, pages 38–45, 2020.

[84] Xingyu Xie, Pan Zhou, Huan Li, Zhouchen Lin, and Shuicheng Yan. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. URL https://arxiv.org/abs/2208.06677. Version Number: 5.

[85] Songlin Yang, Bailin Wang, et al. Gated linear attention transformers with hardware-efficient training, 2023. URL https://arxiv.org/abs/2312.06635.

[86] Songlin Yang, Bailin Wang, et al. Parallelizing linear transformers with the delta rule over sequence length, 2024. URL https://arxiv.org/abs/2406.06484.

[87] Songlin Yang, Bailin Wang, et al. Gated delta networks: Improving mamba2 with delta rule, 2024. URL https://arxiv.org/abs/2412.06464.

[88] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, and Cho-Jui Hsieh. Large batch optimization for deep learning: Training BERT in 76 minutes. URL https://arxiv.org/abs/1904.00962. Version Number: 3.

[89] Han Yuan, DeepSeek-AI, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL https://arxiv.org/abs/2502.11089.

[90] Huizhuo Yuan, Yifeng Liu, Shuang Wu, Xun Zhou, and Quanquan Gu. MARS: Unleashing the power of variance reduction for training large models. URL https://arxiv.org/abs/2411.10438. Version Number: 4.

[91] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Pike Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. doi: 10.48550/ARXIV.2007.14062. URL https://arxiv.org/abs/2007.14062.

[92] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. URL http://arxiv.org/abs/1910.07467.

[93] Jing Zhang, Peng Zhang, Baiwen Kong, Junqiu Wei, and Xin Jiang. Continuous self-attention models with neural ODE networks. 35(16):14393–14401. ISSN 2374-3468, 2159-5399. doi: 10.1609/aaai.v35i16.17692. URL https://ojs.aaai.org/index.php/AAAI/article/view/17692.

[94] Yichi Zhang, Yizhong Wang, et al. Accurate and training-free sparse attention accelerating any model inference, 2025. URL https://arxiv.org/abs/2502.18137.

[95] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. URL https://arxiv.org/abs/2403.03507. Version Number: 2.

[96] Hanqing Zhu, Zhenyu Zhang, Wenyan Cong, Xi Liu, Sem Park, Vikas Chandra, Bo Long, David Z. Pan, Zhangyang Wang, and Jinwon Lee. Apollo: Sgd-like memory, adamw-level performance, 2025. URL https://arxiv.org/abs/2412.05270.

[97] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without normalization. URL http://arxiv.org/abs/2503.10622.

[98] Lianghui Zhu, Yuxin Fang, Bencheng Liao, Shijie Wang, Tianheng Cheng, Zilong Huang, Chen Chen, Lai Wei, Yutao Zeng, Ya Wang, Yi Lin, Yu Li, and Xinggang Wang. Mixture-of-depths attention, 2026. URL https://arxiv.org/abs/2603.15619.

1 Conceptual Introduction—Transformers and Attention for Beginners

This appendix provides an accessible introduction to the fundamental concepts of transformers and the attention mechanism, aimed at readers without deep prior knowledge in deep learning. Transformers are the engine behind modern artificial intelligence systems like ChatGPT, but their operation can be understood through real-world analogies.

1.1 What is a Transformer?

A transformer is a neural architecture that processes text or data sequences by interpreting the meaning of each word while considering all other words in context. Imagine you read a sentence:

"The bank was full of people waiting. Mary went to the bank."

What does "bank" mean in each case? In the first sentence, it refers to a financial institution (or a bench). In the second, it is less clear without context. A transformer solves this automatically by looking at all surrounding words. In the second sentence, seeing "Mary went", the model better understands that it probably is a riverbank (where she goes to swim) or a financial institution (where she goes to conduct a transaction).

This ability to understand the complete meaning of a word by considering the entire sentence is called *contextualization*, and it is the heart of how modern transformers work.

1.2 The Attention Mechanism: An Analogy

The attention mechanism is the "magic trick" that allows transformers to understand context. Think of it as participating in an important conversation in a noisy room:

1. **Lots of noise**: Someone is speaking and being very important to you.
2. **You ignore the rest**: Your brain automatically focuses attention on that person, ignoring other sounds.

3. **You understand better**: By concentrating, you catch every word clearly.

The neural attention mechanism works the same way: each word "asks" how important every other word is to understanding its meaning, then "focuses" its attention on the most relevant ones.

1.3 Practical Example Step-by-Step

Let's see how a transformer understands the word "eats" in two different contexts:

1.3.1 Context 1: "The cat eats fish"

The transformer, when processing "eats", internally asks:

- How relevant is "The"? Little (it's just an article). **Low attention**.
- How relevant is "cat"? Very relevant (it's the subject, who eats). **High attention**.
- How relevant is "fish"? Very relevant (it's what is eaten). **High attention**.

Thus, the mental representation of "eats" focuses primarily on "cat" and "fish".

1.3.2 Context 2: "The restaurant eats into profit margins"

Here the transformer asks in a different context:

- How relevant is "restaurant"? Very relevant (it's the subject). **High attention**.
- How relevant is "profit"? Very relevant (it's affected). **High attention**.
- How relevant is "margins"? Very relevant (it's what's being consumed). **High attention**.

The word "eats" gets a completely different representation because it "attends" to different words in different contexts.

1.4 How Attention Works Mathematically (Simple Version)

Behind the attention shift is mathematics. Here is the simplified version without too much technical detail:

1.4.1 Step 1: Queries, Keys, and Values

Each word in the sentence is transformed into three versions:

- **Query**: "What information do I need to know?"
- **Key**: "What information do I have?"
- **Value**: "Here is my important information."

Imagine in a library, each visitor is a "query", each book is a "key", and the content is the "value". The librarian (attention) matches queries with the most relevant keys to access the correct value.

1.4.2 Step 2: Compatibility

The system examines how *compatible* each query is with each key. Queries similar to keys get a *high* compatibility score. This is calculated by multiplying the query by the key (mathematically, the dot product).

1.4.3 Step 3: Focus

The compatibility scores are converted into "focus weights". High compatibilities mean "focus attention here", and low ones mean "ignore this". Mathematically, a function called `softmax` converts scores into percentages (like: 40% attention here, 35% here, 25% there).

1.4.4 Step 4: Combine

Finally, the system combines all values, weighted by the focus weights. If a word has 80% attention on the subject, 80% of the subject's information gets blended into that word's representation.

1.5 Multiple Attention Heads: Multiple Perspectives

A key feature is that transformers do not use a single attention but *multiple attention heads* in parallel. It's as if you had 8 people analyzing the sentence *simultaneously*, each paying attention to different aspects:

- **Person 1**: "I focus on subject-verb relationships."
- **Person 2**: "I search for the direct object."
- **Person 3**: "I track information about verb tense."
- **Person 4**: "I search for modifiers and adjectives."

Each "head" learns to focus on different language patterns. Together, they capture much richer understanding than a single head.

1.6 Stacking Layers

Transformers process information in *multiple layers* (usually 12 to 48 for practical models). Imagine editing an essay:

1. **First pass**: You fix spelling and basic grammar.
2. **Second pass**: You improve clarity and sentence structure.
3. **Third pass**: You ensure consistency and narrative flow.

Transformers work the same way: each layer refines text understanding. The first layer captures basic features (simple words, genders), while later layers understand complex concepts (word relationships, abstract meanings).

1.7 Why Does It Matter?

The attention mechanism was revolutionary because:

1. **Parallelism**: It finds context for *any word with any other word*, without processing them sequentially (unlike earlier systems). This makes it very fast.
2. **Flexibility**: It learns what patterns to look for automatically from data, without you needing to program rules manually.
3. **Scalability**: It works from small texts to contexts with millions of words.
4. **General Capabilities**: The same mechanism works for translation, summarization, question-answering, text generation, computer vision, and more.

1.8 Practical Challenges

Despite the extraordinary success of transformers, they present challenges:

1.8.1 Computational Complexity

If a sentence has 1,000 words, the standard attention mechanism must compare each word with all other 1,000 words. That's $1,000 \times 1,000 = 1,000,000$ comparisons. For long documents with millions of words, this becomes prohibitively expensive in time and memory.

1.8.2 Memory Requirements

Especially during inference (when using trained models), storing the entire attention matrix can consume enormous amounts of RAM or GPU memory, limiting the sequence lengths you can process.

1.8.3 Device Efficiency

Transformers were designed for powerful TPUs and GPUs. Running them on phones or embedded devices is challenging.

1.9 Modern Solutions

To solve these challenges, research has proposed many variants:

- **Sparse Attention**: Instead of comparing each word with ALL others, it compares only with a strategic subset (nearby neighbors, periodic patterns). Reduces complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$ or $\mathcal{O}(n)$.
- **Recurrent Models**: Like Mamba, they incorporate aspects of old recurrent neural networks but with better modern efficiency.
- **Gated Attention**: Mechanisms that selectively learn what information to pass forward, reducing storage needs.
- **Quantization**: Use smaller numbers (integers instead of decimals) to reduce memory without losing too much precision.

These innovations are what this toolkit (Frankenstein) allows you to experiment with easily.

1.10 Key Takeaways

- A **transformer** is a neural network that understands language context very well.
- The **attention mechanism** allows each word to "focus" on which other words are relevant to understanding its meaning.
- Attention works by computing **compatibility** between each word (query) and all others (keys), then combining information based on that compatibility (values).
- **Multiple heads** of attention explore different patterns in parallel.
- **Multiple layers** progressively refine understanding.
- The main challenge is that standard attention has quadratic complexity, which is expensive for long texts.
- Many **modern solutions** (sparse attention, recurrent models, quantization) address these challenges, and this toolkit lets you explore all of them.

2 Configuration Schema Structure

The Frankestein Transformer toolkit enforces a strict YAML configuration schema with `additionalProperties: false` at all levels. The schema is defined in `src/schema.yaml`, which references modular sub-schemas in `src/schema/`. Every configurable field must be declared in the schema; unrecognized keys are rejected at validation time. This appendix documents the complete schema structure, including all fields, their types, constraints, and cross-component validation rules.

2.1 Top-Level Structure

The root schema object has five top-level properties. All are optional except `training`, which is required:

- `base_model` (string) — HuggingFace model identifier or local path for continual pretraining or SBERT fine-tuning. When set, `model_class` and `model` are not required (the architecture is inferred from the checkpoint).
- `tokenizer` (object) — Tokenizer configuration with fields `name_or_path` (string), `use_fast` (bool), and `trust_remote_code` (bool). Required when using `base_model` with the `mlm` task.
- `model_class` (enum) — Model architecture variant: `frankenstein` (mixed-architecture encoder), `mini` (simplified encoder), or `frankesteindecoder` (autoregressive causal decoder). Required when `base_model` is not set.
- `model` (object) — Model architecture parameters (see §2.2). Required when `base_model` is not set.
- `training` (object, **required**) — Training configuration (see §2.3).

The schema enforces a mutual-exclusivity rule: either `base_model` must be present, or both `model_class` and `model` must be present.

2.2 Model Configuration

The `model` object defines all architectural hyperparameters. Required fields are `dims.vocab_size`, `dims.hidden_size`, `dims.num_layers`, `dims.num_heads`, and `dims.layer_pattern`. Table 1 lists all model fields with their types and descriptions.

Field	Type	Description
*model.dims.**		
`dims.mode`	enum (encoder, decoder)	Attention masking mode. `encoder` bidirectional; `decoder` = causal.
`dims.vocab_size`	int ≥ 1 (required)	Number of unique tokens in the voc match the tokenizer's vocab size.
`dims.hidden_size`	int ≥ 1 (required)	Hidden dimension. Must be divisibl `num_heads`. Typical: 512–2048.
`dims.num_layers`	int ≥ 1 (required)	Number of physical transformer lay 6–24.
`dims.num_loops`	int ≥ 1	Logical loop count. Each layer in `la` runs `num_loops` times. Default: 1.
`dims.num_heads`	int ≥ 1 (required)	Number of parallel attention heads. `hidden_size` evenly.
`dims.retention_heads`	int ≥ 1	Number of heads for RetNet layers. `hidden_size` evenly.

continu

Field	Type	Description
`dims.dropout`	float [0,1]	Global dropout rate. Typical: 0.1 (standard).
`dims.layer_pattern`	array of enum (required)	Ordered sequence of layer types. Le equal `num_layers`. 35+ mixer types Table 2.2.1).
*model.norm.**		
`norm.type`	enum	Normalization strategy: `layer_nor` `dynamic_tanh`, `derf`, `rms_norm`, `pr`
`norm.partial_ratio`	float (0,1]	Fraction of hidden dims for pRMSN estimation. Default: 0.0625.
*model.embedding.factorized.**		
`embedding.factorized.enabled`	bool	Enable embedding matrix factorizat parameters.
`embedding.factorized.dim`	int ≥ 1	Intermediate dimension for factoriz Typical: 64–256.
*model.embedding.conv.**		
`embedding.conv.enabled`	bool	Apply Conv1d over token embeddi local n-gram patterns.
`embedding.conv.kernel`	int ≥ 1	Kernel size for embedding Conv1d.
*model.attention.titan.**		
`attention.titan.positional_encoding`	enum (hope, rope)	Positional encoding method for `tit`
`attention.titan.use_hope`	bool	Legacy flag for HoPE in `titan_att` prefer `positional_encoding`.
`attention.titan.hope.base`	float ≥ 0	Base frequency scale for HoPE. Typ
`attention.titan.hope.damping`	float ≥ 0	Damping coefficient for HoPE atten Typical: 0.01.
`attention.titan.rope.base`	float ≥ 0	Base frequency for RoPE. Typical:
`attention.titan.rope.scaling`	float ≥ 0	Multiplier for RoPE position indice
*model.attention.mla.**		
`attention.mla.latent_rank`	int ≥ 1	Rank of joint KV latent for MLA. I `hidden_size // 2`.
*model.attention.gqla.**		
`attention.gqla.latent_rank`	int ≥ 1	Rank of joint KV latent for GQLA. `hidden_size // 2`.
`attention.gqla.num_groups`	int ≥ 1	Number of GQA groups for GQLA Must divide `num_heads`.
`attention.gqla.decode_path`	enum (mqa_absorb, gqa)	Decoding path for GQLA.
*model.attention.mlra.**		
`attention.mlra.latent_rank`	int ≥ 1	Total latent rank for MLRA. Defau `hidden_size // 2`.

continu

Field	Type	Description
attention.mlra.num_latent_heads	int ≥ 1	Number of disjoint latent sub-space Default: 4.
*model.attention.tucker.**		
attention.tucker.query_rank	int ≥ 1	Tucker rank for query weight tensor hidden_size.
attention.tucker.key_rank	int ≥ 1	Tucker rank for key weight tensor. hidden_size // 2.
attention.tucker.value_rank	int ≥ 1	Tucker rank for value weight tensor hidden_size // 2.
*model.attention.iha.**		
attention.iha.num_pseudo_heads	int ≥ 1	Number of pseudo-heads per head f Default: num_heads.
*model.attention.gta.**		
attention.gta.num_shared_groups	int ≥ 1	Number of head groups sharing an for GTA. Must divide num_heads.
attention.gta.value_latent_rank	int ≥ 1	Rank of value-cache latent for GTA hidden_size // 2.
*model.attention.mtla.**		
attention.mtla.latent_rank	int ≥ 1	Latent rank for MTLA. Default: hi 2.
attention.mtla.merge_factor	int ≥ 1	Number of consecutive KV entries m temporal slot. Default: 2.
attention.mtla.stride	int ≥ 1	Stride between merged temporal slo mtla_merge_factor.
*model.attention.cca.**		
attention.cca.latent_rank	int ≥ 1	Latent width for CCA. Default: hid 4. Must be divisible by num_heads.
attention.cca.num_conv_layers	enum (0, 1, 2)	Number of convolution layers in CC
attention.cca.conv_kernel_seq	int ≥ 1	Kernel size of depth-wise causal seq Default: 4.
attention.cca.conv_kernel_ch	int ≥ 1	Kernel size of head-wise grouped ch Default: 3.
attention.cca.qk_mean	bool	Enable q-k-mean bias in CCA. Defa
attention.cca.value_shift	bool	Enable value-shift in CCA. Default:
*model.attention.ccgqa.**		
attention.ccgqa.query_latent_rank	int ≥ 1	Query latent width for CCGQA. De hidden_size // 2.
attention.ccgqa.kv_latent_rank	int ≥ 1	KV latent width for CCGQA. Defa hidden_size // 8.
attention.ccgqa.num_kv_heads	int ≥ 1	Number of KV heads for CCGQA. num_heads.
attention.ccgqa.num_conv_layers	enum (0, 1, 2)	Number of convolution layers in CC 2.

continu

Field	Type	Description
`attention.ccgqa.conv_kernel_seq`	int ≥ 1	Kernel size of depth-wise causal seq Default: 4.
`attention.ccgqa.conv_kernel_ch`	int ≥ 1	Kernel size of head-wise grouped ch Default: 3.
`attention.ccgqa.qk_mean`	bool	Enable q-k-mean bias in CCGQA.
`attention.ccgqa.value_shift`	bool	Enable value-shift in CCGQA. Defa
*model.attention.msa.**		
`attention.msa.block_size`	int ≥ 1	Block size for MiniMax Sparse Atte 128.
`attention.msa.topk_blocks`	int ≥ 1	Number of blocks selected per GQA query. Default: 16.
`attention.msa.index_dim`	int ≥ 1	Dimensionality of Index Branch hea 64.
`attention.msa.kl_loss_weight`	float ≥ 0	Weight of KL alignment loss for MS branch. Default: 0.0.
*model.attention.sparda.**		
`attention.sparda.block_size`	int ≥ 1	KV block size for SparDA. Default:
`attention.sparda.topk_blocks`	int ≥ 1	Number of KV blocks selected per Default: 16.
`attention.sparda.forecast_dim`	int ≥ 1	Dimensionality of Forecast projectio
*model.attention.engram.**		
`attention.engram.max_ngram_size`	int ≥ 2	Maximum N-gram order for Engram module.
`attention.engram.n_heads_per_ngram`	int ≥ 1	Number of hash heads per N-gram
`attention.engram.embed_dim_per_head`	int ≥ 1	Embedding dimension per hash hea
`attention.engram.kernel_size`	int ≥ 1	Causal depthwise conv kernel width
`attention.engram.seed`	int	Random seed for N-gram hash mult
model. (flat top-level)*		
`use_moe`	bool	Enable Mixture of Experts in FFN
`num_experts`	int ≥ 1	Total number of expert networks in Requires `use_moe=true`.
`top_k_experts`	int ≥ 1	Number of experts activated per tol Must be $\leq$ `num_experts`.
`use_bitnet`	bool	Enable BitLinear ternary weight qu Default: `true`.
`bitnet_routers`	bool	Also quantize routing/scoring proje Requires `use_bitnet=true`. Default
`use_bitnet_conv`	bool	Quantize embedding Conv1d with Requires `use_bitnet=true` and `embedding.conv.enabled=true`. D
`use_mixture_of_depths`	bool	Enable Mixture-of-Depths token rou
`mixture_of_depths_capacity_ratio`	float (0,1]	Fraction of tokens updated per laye
`mixture_of_depths_router_aux_loss_weight`	float ≥ 0	Weight for MoD router auxiliary los
`ffn_hidden_size`	int ≥ 1	Intermediate dimension of feed-forw Typical: 4× `hidden_size`.

continu

Field	Type	Description
ffn_activation	enum (silu, gelu)	Non-linear activation for FFN layer
ffn_activation_config	object	Nested parameters for learnable act degrees/version/init, PReLU init, e
ode_solver	enum (rk4, euler)	Numerical integration method for C
ode_steps	int ≥ 1	Number of integration steps for OD Typical: 2–4.

Table 1: Complete model configuration fields. All fields are optional unless marked required.

Fields are grouped under hierarchical sub-objects (`dims`, `norm`, `embedding.factorized`, `embedding.conv`, `attention.titan`, `attention.mla`, `attention.gqla`, etc.); each sub-header row marks the prefix in effect for the rows that follow it. Fields listed under `model.*` remain flat top-level keys.

2.2.1 Layer Pattern Enumeration

The `model.dims.layer_pattern` array accepts the following 35 mixer types. Each element defines one physical layer in the model stack. Table 2 lists all available types with their training and inference support.

Type	Description	Train	Infer
standard_attn	Classic multi-head self-attention (BERT-style)	✓	✓
sigmoid_attn	Sigmoid-gated attention	✓	✓
gated_softmax_attn	Gated softmax attention with post-SDPA sigmoid	✓	✓
titan_attn	Titan attention with positional encoding (HoPE/RoPE)	✓	✓
retnet / retnet_attn	Retention Network (RNN+transformer hybrid)	✓	✓
mamba	Selective state-space model (SSM)	✓	✓
ode	Continuous-depth ODE layer	✓	✓
gla_attn	Gated Linear Attention with multiplicative decay	✓	✓
deltanet_attn / gated_deltanet_attn	DeltaNet with corrective delta rule	✓	✓
gated_deltanet2_attn	Gated DeltaNet-2 with decoupled channel-wise erase/write gates	✓	✓
hgrn2_attn	HGRN2 with hierarchical forget gates	✓	✓
fox_attn	Forgetting Transformer (FoX) with forget bias in logits	✓	✓
nsa_attn	Native Sparse Attention (three-branch)	✓	✓

continued on next page

Type	Description	Train	Infer
`engram_attn`	N-gram conditional memory lookup	✓	✓
`gqa_attn`	Grouped-Query Attention with configurable KV heads	✓	✓
`longformer_attn`	Longformer: sliding windows + global tokens	✓	✓
`bigbird_attn`	BigBird: local + random + global attention	✓	✓
`sparse_transformer_attn`	Sparse Transformer: factorized patterns	✓	✓
`sparsek_attn`	SparseK: differentiable top-k KV selection	✓	✓
`sparge_attn`	SpargeAttn — two-stage block pruning	×	✓
`fasa_attn`	FASA — frequency-aware token selection	×	✓
`msa_attn`	MiniMax Sparse Attention — block-sparse on GQA	✓	✓
`sparda_attn`	SparDA — decoupled sparse attention with Forecast projection	✓	✓
`kda_attn`	Kimi Delta Attention — channel-wise decay + scalar write gate	✓	✓
`mla_attn`	Multi-Head Latent Attention + RoPE	✓	✓
`gqla_attn`	Group-Query Latent Attention — two decoding paths	✓	✓
`mlra_attn`	Multi-Head Low-Rank Attention — partitionable latent	✓	✓
`tucker_attn`	Tucker Attention — generalised low-rank factorisation	✓	✓
`iha_attn`	Interleaved Head Attention — cross-head pseudo-heads	✓	✓
`gta_attn`	Grouped-head laTenT Attention — shared map + latent values	✓	✓
`mtla_attn`	Multi-head Temporal Latent Attention — temporal KV merge	✓	✓
`cca_attn`	Compressed Convolutional Attention — latent-space attention + convs	✓	✓
`ccgqa_attn`	Compressed Convolutional Grouped Query Attention — CCA + GQA in latent	✓	✓

Table 2: Complete `layer_pattern` enumeration. ✓ = supported, × = not supported.

2.3 Training Configuration

The `training` object controls the entire training pipeline. The only required field is `task`. Table 3 lists all training fields.

Field	Type	Description
`task`	enum (mlm, sbert) (required)	Training objective. `mlm` = Masked Langua Modeling; `sbert` = Sentence-BERT fine-t
`num_epochs`	int ≥ 1	Complete passes through the training dat (MLM only).
`batch_size`	int ≥ 1	Examples per optimizer step (before grad accumulation).
`dataloader_workers`	int ≥ 0	Parallel worker processes for data loading
`max_length`	int ≥ 1	Maximum token sequence length. Typical 128–2048.
`mlm_probability`	float [0,1]	Fraction of tokens masked for MLM predi Standard: 0.15.
`max_samples`	int ≥ 1	Total samples before stopping, independe epochs.
`dataset_batch_size`	int ≥ 1	Internal batch size for streaming dataset.
`num_workers`	int ≥ 0	Parallel workers for dataset streaming pip
`cache_dir`	string	Directory for caching processed datasets.
`local_parquet_dir`	string	Path to local parquet files for offline strea
`prefer_local_cache`	bool	Check local cache first before remote dow
`stream_local_parquet`	bool	Stream from local parquet when `local_parquet_dir` is set.
`join_temp_data_context_window`	int ≥ 0	If > 0, joins cached token chunks into seq this length.
`join_temp_data_min_remainder_tokens`	int ≥ 0	Minimum content tokens in last partial jo window.
`use_amp`	bool	Enable FP16 mixed precision.
`gradient_accumulation_steps`	int ≥ 1	Accumulate gradients over N mini-batche optimizer step.
`optimizer`	object	Optimizer configuration (see §2.4). Requi `task=mlm`.
`sbert`	object	SBERT training configuration (see §2.5). when `task=sbert`.
`scheduler_total_steps`	int ≥ 1	Total optimizer steps for LR schedule.
`scheduler_warmup_ratio`	float [0,1]	Fraction of `scheduler_total_steps` for l warmup.
`scheduler_type`	enum (cosine, constant, linear_warmup_then_constant)	LR decay strategy.
`grad_clip_max_norm`	float ≥ 0	Maximum allowed gradient norm. 0 = dis
`inf_post_clip_threshold`	float ≥ 0	Threshold for flagging post-clip gradient explosions.
`max_nan_retries`	int ≥ 0	Maximum retries on NaN/Inf gradients b stopping.
`checkpoint_every_n_steps`	int ≥ 1	Save rolling checkpoint every N steps.

continued on

Field	Type	Description
max_rolling_checkpoints	int ≥ 1	Maximum rolling checkpoints kept. Oldes when exceeded.
num_best_checkpoints	int ≥ 1	Number of best checkpoints kept permane validation loss).
nan_check_interval	int ≥ 1	Steps between NaN/Inf checks.
log_gradient_stats	bool	Log per-layer gradient norms, min/max.
gradient_log_interval	int ≥ 1	Steps between gradient stat logs.
csv_log_path	string	File path for step-level training metrics in format.
csv_rotate_on_schema_change	bool	Rotate CSV file when schema changes.
gpu_metrics_backend	enum (nvml, none)	GPU telemetry backend.
nvml_device_index	int ≥ 0	GPU index to monitor (0 = first GPU).
enable_block_grad_norms	bool	Log gradient norms per block (embedding attention, FFN, etc.).
telemetry_log_interval	int ≥ 1	Steps between heavy telemetry logs.
gpu_temp_guard_enabled	bool	Strict GPU temperature monitoring. Req gpu_metrics_backend=nvml.
switch_on_thermal	bool	Critical temp triggers GPU→CPU fallbac auto-resume on cooldown.
gpu_temp_pause_threshold_c	float	Temperature (°C) to pause training.
gpu_temp_resume_threshold_c	float	Temperature (°C) to resume training afte
gpu_temp_critical_threshold_c	float	Informational logging marker for severe t events.
gpu_temp_poll_interval_seconds	float	Seconds between temp checks during cool wait.
use_galore	bool	Enable Gradient Low-Rank Projection fo memory-efficient training.
galore_rank	int ≥ 1	Low-rank projection dimension for GaLor
galore_update_interval	int ≥ 1	Steps between projection matrix recompu
galore_scale	float ≥ 0	Scaling factor for projected gradients.
galore_max_dim	int ≥ 1	Maximum tensor dimension for GaLore p
hf_output_dir	string	Directory for saving final model via save_pretrained().

Table 3: Complete training configuration fields.

2.4 Optimizer Configuration

The training.optimizer object has two fields:

- optimizer_class (enum, required) — One of 23 optimizer algorithms: sgd_momentum, adamw, adafactor, galore_adamw, prodigy, lion, sophia, muon, turbo_muon, radam, adan, adopt, ademamix, mars_adamw, cautious_adamw, lamb, schedulefree_adamw, shampoo, soap, anon, apollo, apollo_mini, q_apollo.
- parameters (object) — Optimizer hyperparameters with prefixed keys in the format <optimizer>-<group>_<param>.

2.4.1 Shared Per-Group Suffixes

All optimizers support the following per-parameter-group suffixes, prefixed by the optimizer class name (e.g., adamw-lr_embeddings):

- **Learning rate groups:** `lr_embeddings`, `lr_norms`, `lr_attention`, `lr_other`
- **Weight decay groups:** `wd_embeddings`, `wd_norms`, `wd_attention`, `wd_other`
- **Beta groups:** `betas_embeddings`, `betas_norms`, `betas_attention`, `betas_other`
- **Epsilon groups:** `eps_embeddings`, `eps_norms`, `eps_attention`, `eps_other`

Note: parameters of ODE, RetNet, and Mamba mixers are routed into the `attention` group rather than having dedicated groups.

2.4.2 Optimizer-Specific Global Suffixes

Table 4 lists the additional global suffixes supported by each optimizer (prefixed by the optimizer name).

Optimizer	Specific Global Suffixes
`sgd_momentum`	`momentum`, `nesterov`
`adamw`	(none — shared suffixes only)
`adafactor`	`beta2_decay`, `clip_threshold`, `eps1`, `eps2`
`galore_adamw`	`rank`, `update_proj_gap`
`prodigy`	`d_coef`
`lion`	(none — shared suffixes only)
`sophia`	`rho`, `update_k`
`muon`	`momentum`, `nesterov`, `ns_steps`, `ns_eps`
`turbo_muon`	`momentum`, `nesterov`, `ns_steps`, `ns_eps`
`radam`	(none — shared suffixes only)
`adan`	(none — shared suffixes only)
`adopt`	(none — shared suffixes only)
`ademamix`	(none — shared suffixes only)
`mars_adamw`	(none — shared suffixes only)
`cautious_adamw`	`cautious_clip`
`lamb`	(none — shared suffixes only)
`schedulefree_adamw`	(none — shared suffixes only)
`shampoo`	(none — shared suffixes only)
`soap`	(none — shared suffixes only)
`anon`	`gamma`
`apollo`	`rank`, `update_proj_gap`, `scale`, `scale_type`, `proj_type`, `scale_front`, `disable_nl`
`apollo_mini`	`update_proj_gap`, `scale`, `proj_type`, `scale_front`, `disable_nl`
`q_apollo`	`rank`, `update_proj_gap`, `scale`, `scale_type`, `proj_type`, `scale_front`, `disable_nl`, `quant_bits`

Table 4: Optimizer-specific global parameter suffixes. All keys are prefixed with the optimizer class name (e.g., `sgd_momentum-momentum`).

2.5 SBERT Configuration

The `training.sbert` object configures Sentence-BERT fine-tuning. It is required when `training.task=sbert`. Key fields include:

- `dataset_name` (string) — HuggingFace dataset ID or local path.
- `dataset_type` (enum) — `paired_similarity`, `triplets`, or `qa`.
- `columns` (object) — Optional column name overrides for dataset fields.

- `query_prefix` / `document_prefix` (string) — Text prepended to query/document fields during tokenization.
- `output_dir` (string) — Directory for SBERT artifacts.
- `batch_size` (int) — Examples per batch. Typical: 16–64.
- `gradient_accumulation_steps` (int) — Accumulation steps for SBERT.
- `max_grad_norm` (float) — Gradient clipping threshold. Typical: 0.5–2.0.
- `epochs` (int) — Complete passes through SBERT dataset. Typical: 1–10.
- `warmup_steps` (int) — Steps for LR warmup.
- `evaluation_steps` (int) — Evaluate every N steps.
- `checkpoint_save_steps` (int) — Save rolling checkpoint every N steps.
- `resume_from_checkpoint` (bool) — Resume from latest checkpoint.
- `learning_rate` (float) — Initial LR. Typical: 1×10^{-5} to 1×10^{-4}.
- `max_train_samples` / `max_eval_samples` (int) — Limit training/evaluation samples.
- `max_seq_length` (int) — Max token length for input texts. Typical: 128–512.
- `pooling_mode` (enum) — `mean`, `cls`, or `max`.
- `trust_remote_code` (bool) — Allow custom code from model repo.
- `use_amp` (bool) — Enable FP16 mixed precision.
- `resample_balanced` (bool) — Resample to balanced distribution over similarity scores.
- `standardize_scores` (bool) — Z-score normalize similarity scores.
- `resample_std` (float) — Target std dev when `standardize_scores=true`.
- `optimizer` (object) — Custom optimizer for SBERT (same classes as MLM training).
- `wandb_project` (string) — Weights & Biases project name for telemetry.

2.6 Validation Rules

The schema enforces the following cross-component validation rules, implemented both in the JSON Schema (`_conditional_rules.yaml`) and at runtime in the configuration loader:

1. `additionalProperties: false` is enforced at all nesting levels. Any unrecognized key in any object triggers a validation error.
2. `model.dims.hidden_size` must be divisible by `model.dims.num_heads`. The per-head dimension is `hidden_size/num_heads`.
3. `model.dims.num_kv_heads` must divide `model.dims.num_heads` exactly when Grouped-Query Attention is used.
4. `vocab_size` must match the tokenizer's vocabulary size. Mismatch causes token ID corruption and unrecoverable training failure.
5. When `base_model` is set, `training.task` is required. If `task=mlm`, `tokenizer.name_or_path` is also required.
6. `task: mlm` requires `training.optimizer`; `task: sbert` requires `training.sbert`.
7. `bitnet_routers=true` requires `use_bitnet=true`. The flag has no effect otherwise.
8. `fasa_attn` and `sparge_attn` are evaluation-only layer types. They raise `RuntimeError` if used during training. They must not appear in `model.dims.layer_pattern` for training runs.
9. `model_class: frankesteindecoder` forces `model.dims.mode: decoder` at runtime. Setting `model.dims.mode: encoder` with this class is overridden.
10. Either `base_model` must be present, or both `model_class` and `model` must be present. The schema rejects configurations that provide neither or an incomplete combination.

3 Comprehensive Summary Tables

This appendix provides a complete reference of all components implemented in the Frankestein Transformer codebase: 35 sequence mixers across six families, 23 optimizers, 6 normalization

variants, 43 activation functions, and 4 embedding methods. Each table is self-contained and cross-references the bibliography.

3.1 Sequence Mixer Summary

Table 5 enumerates every sequence mixer supported by the `layer_pattern` schema field, organized by architectural family. Training and inference complexity are expressed in terms of sequence length n, hidden dimension d, and family-specific parameters.

Name	Family	Training	Inference	Key Characteristic	Reference
			Dense (3)		
standard_attn	Dense	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ cache	Full global context; KV bottleneck at long sequences	[79]
sigmoid_attn	Dense	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ cache	Element-wise gating; hardware-efficient replacement for softmax	[67]
gqa	Dense	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ cache	Tunable KV compression; $G\times$ cache reduction via shared heads	[2]
			Recurrent (5)		
retnet_attn	Recurrent	$\mathcal{O}(n^2 d)$ parallel	$\mathcal{O}(1)$	Multi-scale decay; triple computation mode (parallel/recurrent/chunkwise)	[76]
mamba	Recurrent	$\mathcal{O}(nd)$	$\mathcal{O}(1)$	Selective state-space model; hardware-aware parallel scan	[31]
ode	Recurrent	$\mathcal{O}(k \cdot n^2 d)$	$\mathcal{O}(kn)$	Continuous-depth RK4 integration; adaptive step solver	[93]
titan_attn	Recurrent	$\mathcal{O}(c^2 + nf)$	$\mathcal{O}(c^2 + 1)$	Test-time memory adaptation; surprise-driven write dynamics	[7]
engram_attn	Memory	$\mathcal{O}(n \cdot N \cdot H)$	$\mathcal{O}(1)$ lookup	Conditional N-gram memory; hash-based scalable retrieval	[14]
			Sparse (9)		
sparse_transformer_attn	Sparse	$\mathcal{O}(n\sqrt{n} \cdot d)$	$\mathcal{O}(n)$	Factorized strided + fixed masks; $\sqrt{n}$ sparsity pattern	[15]
longformer_attn	Sparse	$\mathcal{O}(n \cdot w \cdot d)$	$\mathcal{O}(n)$	Sliding window + dilation + global tokens; linear in n	[8]
bigbird_attn	Sparse	$\mathcal{O}(n)$	$\mathcal{O}(n)$	Random + local + global; Turing complete attention	[91]
sparsek_attn	Sparse	$\mathcal{O}(n \cdot d)$	$\mathcal{O}(k)$	Differentiable top-k selection; learned sparsity mask	[52]
nsa_attn	Sparse	$\mathcal{O}((t/d + n_l + w)d)$	$\mathcal{O}(n)$	Hardware-aligned 3-branch design; token/neighbor/window	[89]
sparge_attn	Sparse	$\mathcal{O}(n^2 \cdot s \cdot d)$	$\mathcal{O}(n)$	Two-stage block filtering; eval-only (no training)	[94]
fasa_attn	Sparse	$\mathcal{O}(t \cdot N_{\text{tip}} + N_{\text{fac}} \cdot d)$	$\mathcal{O}(n)$	RoPE frequency-aware sparse; eval-only	[82]
msa_attn	Sparse	$\mathcal{O}(H_{\text{kv}} \cdot d_{\text{idx}} \cdot N^2 + H_q \cdot d_h \cdot N \cdot k \cdot B_k)$	$\mathcal{O}(n)$	Index-branch block sparse; deployed at 109B scale	[43]
sparda_attn	Sparse	$\mathcal{O}(H_{\text{kv}} \cdot d_f \cdot N \cdot N_b + H_q \cdot d_h \cdot N \cdot k \cdot B_k)$	$\mathcal{O}(n)$	Forecast-guided lookahead prefetch; predictive sparsity	[26]

continued on next page

Name	Family	Training	Inference	Key Characteristic	Reference
			Gated (8)		
gla_attn	Gated	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Data-dependent diagonal decay; linear attention with gating	[85]
deltanet_attn	Gated	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Delta rule; error-correcting linear attention	[86]
gated_deltanet_attn	Gated	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Decay + write gating synthesis; unified delta framework	[87]
gated_deltanet2_attn	Gated	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Decoupled channel-wise erase/write; improved gating	[34]
hgrn2_attn	Gated	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Hierarchical bounded forget gates; multi-scale recurrence	[64]
fox_attn	Gated	$\mathcal{O}(L^2d)$	$\mathcal{O}(L)$ KV	Logit-space recency bias; FlashAttn compatible	[47]
gated_softmax_attn	Gated	$\mathcal{O}(L^2d)$	$\mathcal{O}(L)$ KV	Post-SDPA sigmoid gating; attention-sink mitigation	[65]
kda_attn	Gated	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Channel-wise decay + scalar write; Kimi Linear design	[40]
			Latent (9)		
mla_attn	Latent	$\mathcal{O}(n^2d)$	$\mathcal{O}(r_{\text{kv}} \cdot n)$	KV latent of rank $r_{\text{kv}} \approx d/2$; DeepSeek-style	[54]
gqla_attn	Latent	$\mathcal{O}(n^2d)$	$\mathcal{O}(r_{\text{kv}} \cdot n)$	Hardware-adaptive dual decode paths; GQA + latent fusion	[55]
mlra_attn	Latent	$\mathcal{O}(n^2d)$	$\mathcal{O}(r_{\text{kv}} \cdot n)$	L sub-heads; 4-way tensor-parallel compatible	[50]
tucker_attn	Latent	$\mathcal{O}(n^2d)$	$\mathcal{O}(k_{\text{rank}} \cdot n)$	Tucker factorization; unifies MHA/GQA/MLA	[42]
iha_attn	Latent	$\mathcal{O}(n^2H^2)$	$\mathcal{O}(H \cdot n \cdot d_h)$	Pseudo-heads; $\Theta(\sqrt{k})$ parameter scaling	[22]
gta_attn	Latent	$\mathcal{O}(n^2 \cdot G \cdot d_h)$	$\mathcal{O}(r_v \cdot n)$	Shared map + SiLU decoder; 62.5% FLOP reduction	[75]
mtla_attn	Latent	$\mathcal{O}(n^2d/m)$	$\mathcal{O}(r_{\text{kv}} \cdot n/m)$	Temporal merge; 5.3× decode speedup	[19]
cca_attn	Latent	$\mathcal{O}(n^2d/C)$	$\mathcal{O}(\tilde{e} \cdot n)$	Full attention in compressed latent; $C\times$ fewer FLOPs	[25]
ccgqa_attn	Latent	$\mathcal{O}(n^2d/C_1)$	$\mathcal{O}(\tilde{e}_{\text{kv}} \cdot n)$	CCA + decoupled GQA; best loss at 8× cache reduction	[25]

Table 5: Complete sequence mixer catalog. n = sequence length, d = hidden size, w = window size, r = low-rank dimension, k = sparsity parameter, C = compression ratio. Family-specific parameters are defined in the corresponding annex.

3.2 Optimizer Summary

Table 6 catalogs all 23 supported optimizers with their memory footprint, per-step computational cost, and key hyperparameters. State buffers are expressed as the number of optimizer-owned tensors per parameter; per-step cost focuses on the dominant term beyond gradient computation.

Optimizer	Family	State Buffers	Per-Step Cost	Key Hyperparams	Reference
Classical & Adaptive Baselines					
SGD+Momentum	Classical	1 (m)	$\mathcal{O}(n)$	lr, momentum, wd	[63]
AdamW	Adaptive	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[51]
RAdam	Adaptive	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[49]
Variance Reduction & Advanced Momentum					
Adan	Variance reduction	3 (m, v, s)	$\mathcal{O}(n)$	lr, β_1, β_2, β_3, eps, wd	[84]
ADOPT	Adam variant	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[77]
AdEMAMix	Multi-EMA	3 (m_1, m_2, v)	$\mathcal{O}(n)$	lr, β_1, β_2, β_3, eps, wd	[60]
MARS	Variance-reduced	3 (m, v, z)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd, γ	[90]
Cautious AdamW	Masked momentum	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[46]
Large-Batch, Memory-Efficient & Parameter-Free					
LAMB	Layer-wise adaptive	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[88]
Schedule-Free	Scheduler-free	3 (z, x, n)	$\mathcal{O}(n)$	lr, β_1, β_2, wd	[18]
Adafactor	Memory-efficient	1–2 (row/col)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd	[72]
GaLore	Low-rank projection	2 (m, v) + SVD	$\mathcal{O}(nr)$	lr, rank r, β_1, β_2, eps, wd	[95]
Prodigy	Parameter-free	3 (m, v, d)	$\mathcal{O}(n)$	β_1, β_2, eps, wd, d_0	[56]
Lion	Sign momentum	1 (m)	$\mathcal{O}(n)$	lr, β_1, β_2, wd	[13]
Second-Order, Geometric & Orthogonality					
Shampoo	Second-order	$2d$ (L_i, R_i)	$\mathcal{O}(n^{1+2/d})$	lr, eps, wd	[33]
SOAP	Second-order	$2d+2$ (m, v)	$\mathcal{O}(n^{1+1/d})$	lr, β_1, β_2, eps, wd	[80]
Sophia	Curvature-aware	3 (m, v, h)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd, k	[48]
Muon	Orthogonality	2 (m, v)	$\mathcal{O}(n \cdot k)$	lr, momentum, wd, NS steps k	[73]
Turbo-Muon	Orthogonality	2 (m, v)	$\mathcal{O}(n \cdot k)$	lr, momentum, wd, NS steps k	[9]
Low-Rank Adaptive & Tunable Adaptivity					
APOLLO	Low-rank adaptive	2 low-rank + proj	$\mathcal{O}(nr)$	lr, rank r, update gap, scale, betas, eps, wd	[96]
APOLLO-Mini	Low-rank adaptive	2 rank-1 + proj	$\mathcal{O}(n)$	lr, update gap, scale, betas, eps, wd	[96]
Q-APOLLO	Quantized low-rank	2 quantized + proj	$\mathcal{O}(nr)$	lr, rank r, update gap, scale, quant bits, betas, eps, wd	[96]
Anon	Adaptivity-tunable	3 (m, v, fixed_lr)	$\mathcal{O}(n)$	lr, β_1, β_2, eps, wd, γ	[3]

Table 6: Complete optimizer catalog. n = number of parameters, d = tensor dimensionality, r = low-rank dimension, k = Newton–Schulz iteration count. State buffers lists the number of optimizer state tensors maintained per parameter group.

3.3 Normalization Variants

Table 7 summarizes the six normalization methods available through the `norm_type` schema field, ranging from classical statistics-based approaches to normalization-free bounded transforms and

weightless RMSNorm with fused norm-then-project execution.

Method	Formula	Stats Needed	Reference
LayerNorm	$\gamma_i(x_i-\mu)/\sqrt{\sigma^2+\epsilon}+\beta_i$	mean + variance	[4]
RMSNorm	$\gamma_i x_i/\sqrt{\frac{1}{d}\sum_j x_j^2+\epsilon}$	RMS only	[92]
pRMSNorm	$\gamma_i x_i/\sqrt{\frac{1}{k}\sum_{j\le k} x_j^2+\epsilon}$	Partial RMS ($p\,\%$)	[92]
Dynamic Tanh (DyT)	$\tanh(\alpha x)$	none	[97]
Dynamic Erf (Derf)	$\mathrm{erf}(\alpha x+s)$	none	[12]
FlashNorm	$x_i/\sqrt{\frac{1}{d}\sum_j x_j^2+\epsilon}$	RMS only (weightless)	[30]

Table 7: Normalization variants. d = hidden dimension, $k=\lceil d\cdot p\rceil$ for partial ratio p, α and s are learnable parameters. FlashNorm is weightless (no learnable parameters); the per-dim scale is folded into the subsequent linear layer per Prop. 1 of [30].

3.4 Activation Functions

Table 8 catalogs the 43 feed-forward activation functions available through the `ffn_activation` schema field (40 elementwise plus 3 gated-FFN variants), grouped by family. "Learn." marks activations with trainable parameters; the full formulas are in Annex 8.

Name	Family	Formula	Learn.	Reference
Classical / Sigmoid–Tanh (12)				
silu / swish	Classical	$x\,\sigma(\beta x)$	no / β	[66]
gelu	Classical	$x\,\Phi(x)$	no	[36]
gelu_tanh	Classical	$\frac{x}{2}\left(1+\tanh\sqrt{\frac{2}{\pi}}(x+0.044715x^3)\right)$	no	[36]
relu	Classical	$\max(0,x)$	no	[59]
sigmoid	Classical	$1/(1+e^{-x})$	no	[44]
tanh	Classical	$(e^x-e^{-x})/(e^x+e^{-x})$	no	[44]
arctan	Classical	$\arctan(x)$	no	[44]
softsign / elliott	Classical	$x/(1+\lvert x\rvert)$	no	[44]
identity	Classical	x	no	(internal)
softplus	Classical	$\log(1+e^x)$	no	[27]
mish	Classical	$x\tanh(\log(1+e^x))$	no	[57]
Rectified family (12)				
leaky_relu	Rectified	$\max(0,x)+a\min(0,x)$, a=0.01	no	[53]
relu6	Rectified	$\min(\max(0,x),6)$	no	[37]
hardswish	Rectified	$x\,\mathrm{ReLU6}(x+3)/6$	no	[37]
prelu	Rectified	$\max(0,x)+\mathbf{p}\odot\min(0,x)$	**yes**	[35]
abs_relu	Rectified	$\max(0,x)-\max(0,-x)$	no	[21]
nl_relu	Rectified	$\beta\log(1+\max(0,x))$	no	[21]
brelu	Rectified	$\min(\max(0,x),t)$	no	[21]
vrelu	Rectified	$\lvert x\rvert$	no	[21]
hexpo	Rectified	$a\max(0,x)-c\max(0,-x)$	no	[21]
ptanh	Rectified	$\max(0,\tanh(x))$	no	[21]
dis_relu	Rectified	$\max(0,x-\delta)$	no	[21]
lisht	Rectified	$x\tanh(x)$	no	[69]
Exponential / ELU family (12)				
elu	Exponential	x if $x\ge 0$ else $\alpha(e^x-1)$	no	[16]
selu	Exponential	$\lambda\,\mathrm{ELU}_{\alpha'}(x)$	no	[41]
celu	Exponential	x if $x\ge 0$ else $\alpha(e^{x/\alpha}-1)$	no	[5]
pelu	Exponential	$\frac{\alpha}{\beta}x$ / $\alpha(e^{x/\beta}-1)$	**yes**	[21]

continued on next page

Name	Family	Formula	Learn.	Reference
mpelu	Exponential	ELU + learnable α, β	**yes**	[21]
felu / eelu / pdelu / preu	Exponential	ELU variants w/ learnable params	**yes**	[21]
softexp	Exponential	exp/linear/log interpolation	**yes**	[21]
elish / hardelish	Exponential	Swish $\cup$ ELU gated branches	no	[6]
Learnable / Adaptive (4)				
swish_trainable	Learnable	$x\,\sigma(\beta x)$, β trainable	**yes**	[66]
maxout	Learnable	$\max_i(W_i x + b_i)$	**yes**	[29]
raf	Learnable	$P(x)/Q(x)$ Padé$(5, 4)$, version A	**yes**	[24]
Gated FFN variants (3)				
swiglu	Gated	$\text{SiLU}(xW_g) \odot (xW_u)\,W_d$	**yes**	[71]
geglu	Gated	$\text{GELU}(xW_g) \odot (xW_u)\,W_d$	**yes**	[71]
reglu	Gated	$\text{ReLU}(xW_g) \odot (xW_u)\,W_d$	**yes**	[71]

Table 8: Activation function catalog. "Learn." indicates the presence of trainable parameters. RAF = Rational Activation Function (learnable Padé ratio). σ is the logistic sigmoid.

3.5 Embedding Variants

Table 9 catalogs the four embedding methods supported by the codebase: two positional encoding schemes and two structural embedding optimizations.

Method	Description	Reference
RoPE	Rotary Position Embedding; applies frequency-based rotation to query and key vectors, encoding relative position through inner product geometry	[**?**]
HoPE	Hyperbolic Rotary Positional Encoding; Lorentz-inspired rotation using hyperbolic functions; enforces monotonic attention decay with distance; RoPE is a special case	[17]
Factorized Embeddings	Decomposes $V \times H$ embedding matrix into $V \times R$ and $R \times H$ factors; reduces parameters from $\mathcal{O}(VH)$ to $\mathcal{O}(VR + RH)$; controlled by `factorized_embedding_dim`	(internal)
Embedding Convolution	Depthwise Conv1d applied over the embedding stream before the first transformer block; provides lightweight local-context filtering; controlled by `embedding_conv_kernel`	(internal)

Table 9: Embedding variants. V = vocabulary size, H = hidden size, R = factorized intermediate dimension.

4 Optimizer Families

4.1 The Evolution of Optimization in Neural Networks

The optimizer survey frames transformer optimization as a response to three structural pressures: non-convex loss landscapes, severe curvature heterogeneity across parameter blocks, and the memory cost of storing optimizer state for very large models. The report argues that the field has diverged into several trajectories: adaptive first-order baselines, variance-reduction methods, memory-efficient methods, structured second-order preconditioners, schedule-free methods, and orthogonality-oriented updates.

4.2 Standard Baseline and Adaptive Optimizers

SGD with Momentum. The classical update accumulates a momentum buffer and then applies a fixed learning rate. At each iteration t, the algorithm maintains an exponential moving average m_t of past gradients:

$$m_t = \beta m_{t-1} + g_t \tag{1}$$

$$\theta_{t+1} = \theta_t - \eta m_t \tag{2}$$

where $g_t = \nabla f(\theta_t)$, $\beta \in [0.8, 0.99]$ controls the momentum decay, and η is the fixed learning rate. The momentum term acts as velocity: it accelerates movement in consistent directions while dampening oscillations in variable directions. Its strengths are low memory overhead (single momentum buffer per parameter) and strong generalization when tuned carefully. Its main weakness in transformer workloads is poor robustness to heterogeneous curvature (different Hessian spectra across parameter groups) and strong dependence on learning-rate schedules.

Algorithm 1 SGD with Momentum

Require: Initial parameters θ_0, learning rate η, momentum coefficient β, weight decay λ
Ensure: Updated parameters θ_T

```
Initialize: momentum buffer m ← 0
for t = 0, 1, 2, ..., T − 1 do
    Compute gradient: g_t ← ∇f(θ_t)
    if weight_decay > 0 then
        g_t ← g_t + λθ_t                          ▷ L2 regularization
    end if
    Update momentum: m ← β · m + g_t
    Update parameters: θ_{t+1} ← θ_t − η · m
end forreturn θ_T
```

Adam and AdamW. Adam tracks exponential moving averages of both first moment (mean) and second moment (uncentered variance) to achieve element-wise adaptive learning rates. The update rule is:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \tag{3}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{4}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \quad \text{(bias correction)} \tag{5}$$

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right) \tag{6}$$

AdamW introduces a crucial modification: weight decay is applied directly to parameters (decoupled from gradients): $\theta_t \leftarrow \theta_t(1-\eta\lambda)$ rather than adding $\lambda\theta_t$ to the gradient. This decoupling prevents the adaptive scaling from interfering with regularization strength. The report treats AdamW as the practical baseline for transformer fine-tuning because it converges quickly and is relatively forgiving to hyperparameter variation. The tradeoff is memory cost: both moment tensors must be stored for every parameter, doubling optimizer-state memory relative to SGD.

Algorithm 2 AdamW (Adam with Decoupled Weight Decay)

Require: Initial parameters θ_0, learning rate η, exponential decay rates $\beta_1, \beta_2 \in [0, 1)$
Require: Momentum constant ϵ, weight decay λ
Ensure: Updated parameters θ_T

Initialize: first moment $m \leftarrow 0$, second moment $v \leftarrow 0$, step counter $t \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
    Compute gradient: $g_t \leftarrow \nabla f(\theta_{t-1})$
    Weight decay (decoupled): $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    Update moments: $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$
    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
    Bias correction: $\hat{m} \leftarrow m/(1 - \beta_1^t)$, $\hat{v} \leftarrow v/(1 - \beta_2^t)$
    Update parameters: $\theta_t \leftarrow \theta_{t-1} - \eta(\hat{m}/(\sqrt{\hat{v}} + \epsilon))$
end forreturn θ_T

RAdam (Rectified Adam). RAdam addresses Adam's instability in early training by dynamically rectifying the adaptive learning rate. The key observation is that the second moment v_t has very high variance in the first few steps, causing unreliable adaptive scaling. RAdam computes the effective simple moving average (SMA) window length:

$$\rho_t = \rho_\infty - \frac{2t\beta_2^t}{1 - \beta_2^t}, \quad \text{where} \quad \rho_\infty = \frac{2}{1 - \beta_2} - 1$$

When $\rho_t > 4$ (sufficient samples for variance estimation), RAdam applies adaptive scaling with a rectification term:

$$r_t = \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}}, \quad \theta_{t+1} = \theta_t - \eta r_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

When $\rho_t \leq 4$, RAdam falls back to SGD with momentum: $\theta_{t+1} = \theta_t - \eta\hat{m}_t$. This graceful transition eliminates the need for manual learning-rate warmup schedules and improves robustness.

Algorithm 3 RAdam (Rectified Adam)

Require: Initial parameters θ_0, learning rate η, β_1, β_2, weight decay λ
Ensure: Updated parameters θ_T

Initialize: $m \leftarrow 0$, $v \leftarrow 0$, $t \leftarrow 0$
Compute: $\rho_\infty \leftarrow 2/(1 - \beta_2) - 1$
for $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    if weight_decay > 0 **then**
        $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    end if
    $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$
    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
    $\hat{m} \leftarrow m/(1 - \beta_1^t)$, $\hat{v} \leftarrow v/(1 - \beta_2^t)$
    $\rho_t \leftarrow \rho_\infty - 2t\beta_2^t/(1 - \beta_2^t)$
    if $\rho_t > 4$ **then**
        $r_t \leftarrow \sqrt{((\rho_t - 4)(\rho_t - 2)\rho_\infty)/((\rho_\infty - 4)(\rho_\infty - 2)\rho_t)}$
        $\theta_t \leftarrow \theta_{t-1} - \eta r_t \hat{m}/(\sqrt{\hat{v}} + \epsilon)$
    else
        $\theta_t \leftarrow \theta_{t-1} - \eta\hat{m}$ ▷ SGD with momentum
    end if
end forreturn θ_T

4.3 Advanced Momentum and Variance Reduction (2024–2025)

Adan (Adaptive Nesterov Momentum). Adan reformulates Nesterov acceleration without the extra gradient computation required by classical Nesterov SGD. The algorithm maintains three momentum buffers:

$$m_t = (1-\beta_1)m_{t-1} + \beta_1 g_t \quad \text{(first moment)} \tag{7}$$

$$v_t = (1-\beta_2)v_{t-1} + \beta_2(g_t - g_{t-1}) \quad \text{(velocity/gradient difference)} \tag{8}$$

$$n_t = (1-\beta_3)n_{t-1} + \beta_3[g_t + (1-\beta_1)(g_t - g_{t-1})]^2 \quad \text{(Nesterov second moment)} \tag{9}$$

The **Nesterov Momentum Estimation** (NME) term $\bar{g}_t = g_t + (1-\beta_1)(g_t - g_{t-1})$ estimates the gradient at a future position without evaluating it. The update combines momentum with velocity:

$$\bar{m}_t = m_t + (1-\beta_1)v_t, \quad \theta_{t+1} = \theta_t - \eta\frac{\bar{m}_t}{\sqrt{n_t}+\epsilon}$$

Adan achieves fast convergence across diverse architectures (CNNs, GANs, Transformers) through this acceleration. The cost is maintaining three momentum-like buffers, increasing memory overhead relative to Adam.

Algorithm 4 Adan (Adaptive Nesterov Momentum)

Require: Initial parameters θ_0, learning rate η, $\beta_1, \beta_2, \beta_3 \in (0,1)$
Ensure: Updated parameters θ_T
Initialize: $m \leftarrow 0$, $v \leftarrow 0$, $n \leftarrow 0$, $g_{-1} \leftarrow 0$ (previous gradient)
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  $m \leftarrow (1-\beta_1)m + \beta_1 g_t$
  $\Delta g_t \leftarrow g_t - g_{t-1}$
  $v \leftarrow (1-\beta_2)v + \beta_2 \Delta g_t$
  Nesterov estimation: $\bar{g}_t \leftarrow g_t + (1-\beta_1)\Delta g_t$
  $n \leftarrow (1-\beta_3)n + \beta_3 \bar{g}_t^2$
  Combined momentum: $\bar{m} \leftarrow m + (1-\beta_1)v$
  $\theta_t \leftarrow \theta_{t-1} - \eta(\bar{m}/(\sqrt{n}+\epsilon))$
  $g_{t-1} \leftarrow g_t$
end forreturn θ_T

ADOPT (Adam with Optimal Pruned Tuning). ADOPT fixes a fundamental theoretical issue in Adam: the gradient appears in both the first moment and second moment estimates, creating circularity. ADOPT **decouples** by using the previous step's second moment for the denominator:

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2 \tag{10}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\frac{g_t}{\sqrt{v_{t-1}}+\epsilon} \quad \text{(uses } v_{t-1}\text{, not } v_t\text{)} \tag{11}$$

$$\theta_{t+1} = \theta_t - \eta m_t \tag{12}$$

This simple reordering achieves the optimal convergence rate $O(1/\sqrt{T})$ with **any** choice of $\beta_2 \in (0,1)$, without bounded-noise assumptions. The practical consequence is that ADOPT is a drop-in replacement for Adam with stronger theoretical guarantees and comparable or superior empirical performance across vision, NLP, RL, and generative modeling domains.

Algorithm 5 ADOPT (Adam with Optimal Pruned Tuning)

Require: Initial parameters θ_0, learning rate η, β_1, β_2, tolerance ϵ
Ensure: Updated parameters θ_T

Initialize: $m \leftarrow 0$, $v \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    if weight_decay > 0 **then**
        $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    end if
    Use **previous** second moment: denom $\leftarrow \sqrt{\max(v, \epsilon)} + \epsilon$
    Update first moment using previous variance: $m \leftarrow \beta_1 m + (1 - \beta_1)(g_t/\text{denom})$
    Update second moment with **current** gradient: $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
    $\theta_t \leftarrow \theta_{t-1} - \eta m$
end forreturn θ_T

AdEMAMix (Exponential Moving Average Mixture). AdEMAMix replaces Adam's single EMA of gradients with a **mixture of two EMAs**: one fast-decaying and one slow-decaying. This addresses the observation that gradients remain informative over tens of thousands of steps, not just hundreds:

$$m_{1,t} = \beta_1 m_{1,t-1} + (1 - \beta_1) g_t \quad \text{(fast EMA, } \beta_1 \approx 0.9\text{)} \tag{13}$$

$$m_{2,t} = \beta_3 m_{2,t-1} + (1 - \beta_3) g_t \quad \text{(slow EMA, } \beta_3 \approx 0.9999\text{)} \tag{14}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{15}$$

$$m_t = m_{1,t} + \alpha_t m_{2,t} \quad \text{(mixture with scheduled weight } \alpha_t\text{)} \tag{16}$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \tag{17}$$

The mixture weight α_t typically increases during training, allowing the fast EMA to provide immediate adaptation while the slow EMA accumulates long-range gradient correlations. Empirically, a 1.3B model on 101B tokens achieves similar loss to AdamW on 197B tokens (95% data efficiency gain), suggesting the slow EMA significantly reduces model forgetting.

Algorithm 6 AdEMAMix (Exponential Moving Average Mixture)

Require: Initial parameters θ_0, learning rate η, β_1 (fast), β_3 (slow), β_2 (second moment)
Require: Mixture weight α_t (typically increasing), tolerance ϵ
Ensure: Updated parameters θ_T

Initialize: $m_1 \leftarrow 0$ (fast EMA), $m_2 \leftarrow 0$ (slow EMA), $v \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    if weight_decay > 0 **then**
        $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    end if
    $m_1 \leftarrow \beta_1 m_1 + (1 - \beta_1) g_t$ ▷ Fast EMA
    $m_2 \leftarrow \beta_3 m_2 + (1 - \beta_3) g_t$ ▷ Slow EMA
    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
    $m_t \leftarrow m_1 + \alpha_t m_2$ ▷ Mixture
    $\theta_t \leftarrow \theta_{t-1} - \eta(m_t/(\sqrt{v} + \epsilon))$
end forreturn θ_T

MARS (Make Adaptive learning Rates Shine). MARS combines preconditioned gradient methods (e.g., AdamW) with **variance reduction** via scaled stochastic recursive momentum.

The core innovation is a variance-reduced gradient estimate:

$$c_t = g_t + \gamma(c_{t-1} - g_{t-1}) \quad \text{(SVRG-style recursive estimate)} \tag{18}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)c_t \tag{19}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)c_t^2 \tag{20}$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \tag{21}$$

where $\gamma \in [0.01, 0.1]$ controls how much historical gradient information is retained. The variance-reduced gradient c_t acts as an implicit noise filter, amplifying consistent signal directions and dampening contradictory noise. MARS-AdamW consistently outperforms bare AdamW by significant margins on GPT-2 pretraining, suggesting that variance reduction substantially improves convergence in mini-batch stochastic training.

Algorithm 7 MARS (Make Adaptive learning Rates Shine)

Require: Initial parameters θ_0, learning rate η, β_1, β_2, weight decay λ
Require: Variance reduction coefficient $\gamma \in [0.01, 0.1]$
Ensure: Updated parameters θ_T

Initialize: $m \leftarrow 0$, $v \leftarrow 0$, $c \leftarrow 0$ (variance-reduced gradient), $g_{-1} \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  Variance reduction: $c \leftarrow g_t + \gamma(c - g_{t-1})$
  $m \leftarrow \beta_1 m + (1-\beta_1)c$
  $v \leftarrow \beta_2 v + (1-\beta_2)c^2$
  if weight_decay > 0 **then**
    $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
  end if
  $\theta_t \leftarrow \theta_{t-1} - \eta(m/(\sqrt{v} + \epsilon))$
  $g_{t-1} \leftarrow g_t$
end forreturn θ_T

Cautious Optimizers (Cautious AdamW, Cautious Lion). The Cautious framework applies a **one-line modification** to any momentum-based optimizer: mask the update so that only dimensions where momentum and gradient directions agree are applied:

$$\text{mask}_i = \begin{cases} 1 & \text{if } m_t[i] \cdot g_t[i] > 0 \quad \text{(agreement)} \\ 0 & \text{otherwise} \end{cases} \tag{22}$$

$$u_t = \left(\frac{m_t}{\sqrt{v_t} + \epsilon} \right) \odot \text{mask} \quad \text{(element-wise masking)} \tag{23}$$

$$\theta_{t+1} = \theta_t - \eta u_t \tag{24}$$

The intuition: momentum m_t estimates the gradient direction from history; the current gradient g_t is instantaneous signal. When both agree, the optimizer is confident and should update aggressively. When they disagree, the optimizer is conflicted—the historical trend points toward a region that may have been good before, but current evidence contradicts it. By masking conflicted dimensions, Cautious becomes more conservative and avoids corrupted steps. Empirically, this simple masking achieves up to **1.47× speedup** on Llama and MAE pretraining while preserving convergence guarantees.

Algorithm 8 Cautious AdamW (Consensus-based Update Masking)

Require: Initial parameters θ_0, learning rate η, β_1, β_2, weight decay λ
Ensure: Updated parameters θ_T

Initialize: $m \leftarrow 0$, $v \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$
  $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
  Consensus mask: $\text{mask}_i \leftarrow 1$ if $m[i] \cdot g_t[i] > 0$, else 0 ▷ Agreement
  Base Adam update: $u \leftarrow m/(\sqrt{v} + \epsilon)$
  Apply mask: $u_{\text{masked}} \leftarrow u \odot \text{mask}$ ▷ Element-wise
  if weight_decay > 0 **then**
    $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
  end if
  $\theta_t \leftarrow \theta_{t-1} - \eta u_{\text{masked}}$
end forreturn θ_T

4.4 Large-Batch, Memory-Efficient, and Parameter-Free Optimizers

LAMB (Layer-wise Adaptive Moments optimizer for Batch training). LAMB extends Adam with **layer-wise adaptive rate scaling** (inspired by LARS), enabling stable training with extreme batch sizes (e.g., 64K on BERT):

$$m_t^L = \beta_1 m_{t-1}^L + (1 - \beta_1) g_t^L \quad \text{(layer-wise first moment)} \tag{25}$$

$$v_t^L = \beta_2 v_{t-1}^L + (1 - \beta_2)(g_t^L)^2 \tag{26}$$

$$u_{\text{adam}}^L = \frac{m_t^L}{\sqrt{v_t^L} + \epsilon} + \lambda \theta_{t-1}^L \quad \text{(base adaptive step with decay)} \tag{27}$$

$$\phi^L = \frac{\|\theta_{t-1}^L\|_2}{\|u_{\text{adam}}^L\|_2} \quad \text{(trust ratio: layer normalization)} \tag{28}$$

$$\theta_t^L = \theta_{t-1}^L - \eta \cdot \phi^L \cdot u_{\text{adam}}^L \tag{29}$$

The **trust ratio** $\phi^L = \|\theta^L\|_2 / \|u_{\text{adam}}^L\|_2$ normalizes the effective update step relative to weight magnitude. In very large batches, stochastic gradient noise can exceed signal, destabilizing layer-wise learning rates. By scaling updates proportionally to weight norms, LAMB ensures relative changes rather than absolute ones, preventing small confident updates in large vectors from dominating. LAMB preserves small-batch generalization benefits while enabling efficient large-batch training.

Algorithm 9 LAMB (Layer-wise Adaptive Moments optimizer for Batch training)

Require: Initial parameters θ_0, learning rate η, β_1, β_2, weight decay λ
Ensure: Updated parameters θ_T
Initialize: For each layer L: $m^L \leftarrow 0$, $v^L \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  for each layer L **do**
    $g_t^L \leftarrow \nabla f(\theta_t^L)$
    $m^L \leftarrow \beta_1 m^L + (1-\beta_1) g_t^L$
    $v^L \leftarrow \beta_2 v^L + (1-\beta_2)(g_t^L)^2$
    Base Adam: $u_{\text{adam}}^L \leftarrow m^L / (\sqrt{v^L} + \epsilon)$
    if weight_decay > 0 **then**
      $u_{\text{adam}}^L \leftarrow u_{\text{adam}}^L + \lambda \theta_{t-1}^L$
    end if
    Trust ratio: $\phi^L \leftarrow \|\theta_{t-1}^L\|_2 / \|u_{\text{adam}}^L\|_2$ if both nonzero, else 1
    $\theta_t^L \leftarrow \theta_{t-1}^L - \eta \phi^L u_{\text{adam}}^L$
  end for
end forreturn θ_T

Schedule-Free AdamW. Schedule-free methods remove explicit scheduler design from the optimization recipe. The algorithm maintains two parameter streams: z_t (exploration) and x_t (smooth average):

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{(variance)} \tag{30}$$

$$z_t = z_{t-1}(1-\eta\lambda) - \eta \frac{g_t}{\sqrt{v_t} + \epsilon} \quad \text{(adaptive step with decay)} \tag{31}$$

$$c_t = \frac{1}{t+1} \quad \text{(averaging coefficient, decreases over time)} \tag{32}$$

$$x_t = (1-c_t) x_{t-1} + c_t z_t \quad \text{(iterate averaging)} \tag{33}$$

$$y_t = (1-\beta) z_t + \beta x_t \quad \text{(interpolation for evaluation point)} \tag{34}$$

The averaging coefficient $c_t = 1/(t+1)$ implements an implicit learning-rate schedule without explicitly specifying total steps T. This framework unifies scheduling and iterate averaging: the algorithm explores via z_t while accumulating stable direction via x_t. The evaluation point y_t (used for gradient computation) interpolates between exploration and stability. Schedule-Free achieves state-of-the-art convergence across convex optimization, large-scale deep learning, and reinforcement learning, while removing a major hyperparameter.

Algorithm 10 Schedule-Free AdamW

Require: Initial parameters θ_0, learning rate η (fixed), β_1, β_2, weight decay λ
Ensure: Updated parameters θ_T or averaging x_T
Initialize: $z \leftarrow \theta_0$ (exploration), $x \leftarrow \theta_0$ (average), $v \leftarrow 0$, $t \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(y_{t-1})$ (gradient at interpolation point)
  $v \leftarrow \beta_2 v + (1-\beta_2) g_t^2$ ▷ Variance
  Weight decay on z: $z \leftarrow z(1-\eta\lambda)$
  $z \leftarrow z - \eta g_t / (\sqrt{v} + \epsilon)$ ▷ Adaptive step on raw
  $c_t \leftarrow 1/(t+1)$ ▷ Averaging coefficient
  $x \leftarrow (1-c_t) x + c_t z$ ▷ Iterate averaging
  $y_t \leftarrow (1-\beta_1) z + \beta_1 x$ ▷ Interpolation for next eval
end forreturn x_T or y_T

Adafactor. Adafactor reduces optimizer memory by **factorizing** second-moment statistics for matrix-shaped parameters, storing only row and column accumulators rather than dense variance states. For a gradient matrix $G_t \in \mathbb{R}^{m \times n}$:

$$R_t = \beta_2 R_{t-1} + (1 - \beta_2)(G_t^2)\mathbf{1}_n^T \quad \text{(row variance)} \tag{35}$$

$$C_t = \beta_2 C_{t-1} + (1 - \beta_2)\mathbf{1}_m^\top (G_t^2) \quad \text{(column variance)} \tag{36}$$

$$\hat{V}_t = \frac{R_t C_t}{\mathbf{1}_n^\top R_t} \quad \text{(reconstructed variance via outer product)} \tag{37}$$

$$U_t = \frac{G_t}{\sqrt{\hat{V}_t} + \epsilon} \quad \text{(normalized adaptive step)} \tag{38}$$

$$\hat{U}_t = \frac{U_t}{\max(1, \mathrm{RMS}(U_t))} \quad \text{(stability clipping)} \tag{39}$$

Instead of storing $m \times n$ second-moment values, Adafactor stores only $m + n$ accumulators, reducing memory from $O(n_{\text{params}})$ to $O(\sqrt{n_{\text{params}}})$. This is most attractive when VRAM is dominated by optimizer state rather than activations. The tradeoff is reduced optimization expressiveness and potential instability on some tasks.

Algorithm 11 Adafactor (Factorized Second-Moment Statistics)

Require: Gradient matrix $G_t \in \mathbb{R}^{m \times n}$, learning rate η, $\beta_2 \approx 0.999$
Ensure: Updated parameters

Initialize: Row accumulators $R \leftarrow \epsilon \mathbf{1}_m^T$, Column $C \leftarrow \epsilon \mathbf{1}_n$
for $t = 1, 2, \ldots, T$ **do**
  $G_t \leftarrow \nabla f(\theta_t)$ (matrix gradient)
  $R \leftarrow \beta_2 R + (1 - \beta_2)(G_t^2)\mathbf{1}_n^T$ ▷ Row inner products
  $C \leftarrow \beta_2 C + (1 - \beta_2)\mathbf{1}_m^\top (G_t^2)$ ▷ Column inner products
  Reconstructed variance: $\hat{V} \leftarrow (R \cdot C)/(\mathbf{1}_n^\top R)$ ▷ Via outer product
  Normalized adaptive step: $U_t \leftarrow G_t/(\sqrt{\hat{V}} + \epsilon)$
  Per-row RMS clipping: $\hat{U}_t \leftarrow U_t / \max(1, \mathrm{RMS}(U_t))$
  $\theta_{t+1} \leftarrow \theta_t - \eta \hat{U}_t$
end for

GaLore (Gradient Low-Rank Projection). GaLore projects 2D gradients into a low-rank subspace before optimization, reducing optimizer-state memory while maintaining adaptive step scaling. For a 2D parameter matrix $W \in \mathbb{R}^{m \times n}$:

$$U, S, V = \mathrm{SVD}(G_t) \quad \text{(compute singular decomposition)} \tag{40}$$

$$P \in \mathbb{R}^{n \times r} \quad \text{or} \quad P^\top \in \mathbb{R}^{r \times m} \quad \text{(select top-}r\text{ singular vectors)} \tag{41}$$

$$G_{\text{low}} = P^\top G_t \quad \text{(project to low-rank space)} \tag{42}$$

$$\Delta_{\text{low}} = \mathrm{Adam}(G_{\text{low}}) \quad \text{(optimize in compressed space)} \tag{43}$$

$$\Delta = P \Delta_{\text{low}} \quad \text{(reconstruct in original space)} \tag{44}$$

By projecting into a rank-r subspace (typically $r \ll \min(m, n)$), optimizer state is reduced from $O(mn)$ to $O(r(m + n))$ for 2D parameters. This complementary memory-saving approach is especially relevant when the model is too large for full-rank optimizer state. GaLore works synergistically with other techniques and has shown strong empirical results on billion-parameter models.

Algorithm 12 GaLore (Gradient Low-Rank Projection)

Require: 2D parameter matrix $W \in \mathbb{R}^{m\times n}$, learning rate η, rank $r \ll \min(m,n)$
Ensure: Updated parameters
Initialize: Adam state in low-rank space (if rank-2 variant)
for $t = 1, 2, \ldots, T$ **do**
  $G_t \leftarrow \nabla f(W_t)$
  if $t \mod K = 1$ **then** ▷ Periodic SVD
    $U_t, S_t, V_t^\top \leftarrow \mathrm{SVD}(G_t)$ (full or thin)
    Select projection: $P \leftarrow U_t[:, :r]$ or $P \leftarrow V_t[:, :r]$
  end if
  Project gradient: $G_{\text{low}} \leftarrow P^\top G_t$ (or $G_t P^\top$ for right projection)
  Run Adam on low-rank: $\Delta_{\text{low}} \leftarrow \mathrm{Adam}(G_{\text{low}})$
  Reconstruct in original space: $\Delta \leftarrow P\Delta_{\text{low}}$ (or $\Delta_{\text{low}} P^\top$)
  $W_t \leftarrow W_t - \eta\Delta$
end for

APOLLO, APOLLO-Mini, and Q-APOLLO. The APOLLO family [96] begins from the observation that AdamW's element-wise denominator can be **coarsened into a structured learning-rate update**. Rather than storing dense moments for every parameter entry, APOLLO projects a matrix gradient $G_t \in \mathbb{R}^{m\times n}$ into a compact random subspace and tracks Adam-style moments there:

$$R_t = P_t G_t \quad \text{or} \quad R_t = G_t P_t^\top \tag{45}$$

$$M_t^R = \beta_1 M_{t-1}^R + (1-\beta_1) R_t \tag{46}$$

$$V_t^R = \beta_2 V_{t-1}^R + (1-\beta_2) R_t^2 \tag{47}$$

$$\widetilde{R}_t = \frac{M_t^R}{\sqrt{V_t^R} + \epsilon} \tag{48}$$

The projected state is *not* expanded back into a dense low-rank update as in SVD-based methods. Instead, APOLLO estimates a structured scaling tensor S_t in the original space. In the standard APOLLO variant, that scaling is **channel-wise**: each row or channel receives its own norm ratio. In APOLLO-Mini, the scaling is reduced to a single **tensor-wise** scalar, corresponding to the rank-1 extreme described in the paper. The resulting parameter update is therefore Adam-like in adaptation but much closer to SGD in state cost:

$$W_t \leftarrow (1-\eta\lambda) W_{t-1} - \eta\,\alpha\,(G_t \odot S_t)$$

where α is the extra scale factor used to stabilize highly compressed variants.

The paper's practical contribution is twofold. First, APOLLO replaces expensive repeated SVD with periodically refreshed **Gaussian random projection**, so the systems burden is ordinary matrix multiplication rather than spectral decomposition. Second, the optimizer is unusually tolerant to extreme compression: even **APOLLO-Mini**, which keeps only rank-1 auxiliary state, remains competitive with or better than AdamW in the reported pre-training experiments while approaching SGD-level memory cost.

Algorithm 13 APOLLO / APOLLO-Mini Structured Gradient Scaling

Require: Weight matrix $W \in \mathbb{R}^{m \times n}$ with $m \leq n$, learning rate η, scale factor α, decay rates (β_1, β_2), weight decay λ, rank r, projection refresh interval T
Ensure: Updated parameters W_T
Initialize projected moments: $M^R \leftarrow 0$, $V^R \leftarrow 0$, step $t \leftarrow 0$
repeat
  Compute gradient: $G_t \leftarrow \nabla_W \phi(W_t)$
  if $t \bmod T = 0$ **then**
    Sample Gaussian projector $P_t \sim \mathcal{N}(0, 1/r)$ with a fresh seed
  end if
  Project gradient: $R_t \leftarrow P_t G_t$ ▷ or $G_t P_t^\top$ depending on layout
  Update projected AdamW moments: $M_t^R, V_t^R \leftarrow \text{AdamWState}(R_t; \beta_1, \beta_2)$
  Normalize projected state: $\widetilde{R}_t \leftarrow M_t^R / (\sqrt{V_t^R} + \epsilon)$
  if APOLLO **then**
    $S_t \leftarrow \text{diag}(s_1^R, \ldots, s_m^R)$, where $s_i^R = \|\widetilde{R}_t[i,:]\|_2 / \|R_t[i,:]\|_2$
  else
    $S_t \leftarrow s_t^R$, where $s_t^R = \|\widetilde{R}_t\|_2 / \|R_t\|_2$
  end if
  Update weights: $W_t \leftarrow (1 - \eta\lambda) W_{t-1} - \eta\, \alpha\, (G_t \odot S_t)$
  $t \leftarrow t + 1$
until convergence

APOLLO-Mini. APOLLO-Mini is the family member optimized for **extreme memory efficiency**. The paper motivates it by arguing that, in a rank-1 compact space, channel-wise scaling becomes too noisy, so the update is coarsened further to a single tensor-wise scale. The loss of granularity is partially offset by the explicit scaling factor α (the paper discusses values such as 128 in this regime), giving a useful point on the optimization Pareto frontier: very small state, no SVD overhead, and still strong pre-training behavior.

Q-APOLLO. The APOLLO paper also stresses that the family combines naturally with **quantization** for ultra-low-memory training. This repository turns that systems observation into a concrete optimizer variant, `q_apollo`. In the implementation, APOLLO's low-rank first and second moments are stored in quantized form together with per-tensor scales and offsets, and are dequantized only when needed for the next step. Q-APOLLO therefore preserves APOLLO's projected-gradient logic while reducing the precision of the remaining optimizer state, making it the most aggressive memory-saving member of the local optimizer stack.

Anon (Adaptivity Non-restricted Optimizer with Novel convergence technique). Anon [3] introduces continuously tunable adaptivity via a single parameter $\gamma \in \mathbb{R}$, bridging the gap between SGD-like ($\gamma = 0$) and Adam-like ($\gamma = 1$) behaviors, and extrapolating beyond both. The core innovation is the Incremental Delay Update (IDU) mechanism, which enables stable convergence across the entire real-valued adaptivity spectrum:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{(first moment)} \tag{49}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{(second moment)} \tag{50}$$

$$\bar{v}_t = \frac{v_t}{1-\beta_2^{\max(t,1)}} + \epsilon \quad \text{(bias-corrected second moment)} \tag{51}$$

$$\text{fixed\_lr}_k = \begin{cases} \bar{v}_k^{-\gamma/2} & \text{if } k = 0 \\ \sqrt{2 / \left(\frac{1}{\text{fixed\_lr}_{k-1}^2} + \bar{v}_k^{\gamma} \right)} & \text{if } k > 0 \end{cases} \tag{52}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{1-\beta_1^t} \cdot m_t \odot \text{fixed\_lr}_{\lfloor \log_2 t \rfloor} \tag{53}$$

Unlike standard adaptive optimizers, IDU updates the preconditioner only at powers of two ($t = 2^k$), accumulating gradient statistics between updates and aggregating them recursively. This soft, multi-scale accumulator avoids the hard max-tracking of AMSGrad while provably stabilizing convergence for any $\gamma \in \mathbb{R}$. The parameter γ directly controls adaptivity: $\gamma > 1$ accelerates escape from saddle points (favorable for complex architectures), $\gamma = 1$ recovers Adam-like behavior, $\gamma = 0$ yields SGD-like scaling, and $\gamma < 0$ biases toward flatter minima (beneficial for classical architectures like CNNs).

Anon maintains three state buffers (m, v, fixed_lr) with $\mathcal{O}(n)$ per-step cost, placing it in the same complexity class as AdamW but with the added flexibility of tunable adaptivity. The IDU technique provably converges at $\mathcal{O}(\sqrt{T})$ regret for convex problems and $\mathcal{O}(\ln T/\sqrt{T})$ for non-convex settings, matching the guarantees of mainstream adaptive optimizers while supporting the full real-valued adaptivity range.

Algorithm 14 Anon (Adaptivity Non-restricted Optimizer with IDU)

Require: Initial parameters θ_0, learning rate η, β_1, β_2, tolerance ϵ, adaptivity $\gamma \in \mathbb{R}$
Ensure: Updated parameters θ_T

Initialize: $m \leftarrow 0$, $v \leftarrow 0$, fixed_lr $\leftarrow 0$, step $t \leftarrow 0$, IDU counter $k \leftarrow -1$
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  $m \leftarrow \beta_1 m + (1-\beta_1) g_t$
  $v \leftarrow \beta_2 v + (1-\beta_2) g_t^2$
  if $t = 2^k$ **then**
    $k \leftarrow k + 1$
    $\bar{v} \leftarrow v / (1 - \beta_2^{\max(t/2,1)}) + \epsilon$
    if $k = 0$ **then**
      fixed_lr $\leftarrow \bar{v}^{-\gamma/2}$
    else
      fixed_lr $\leftarrow \sqrt{2/(1/\text{fixed\_lr}^2 + \bar{v}^{\gamma})}$
    end if
    $v \leftarrow 0$
  end if
  $\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{1-\beta_1^t} \cdot m \odot \text{fixed\_lr}$
end forreturn θ_T

Prodigy (Approximating the Distance Estimate). Prodigy adapts the effective step scale through a running distance-like statistic, eliminating the need for explicit learning-rate tuning.

The algorithm maintains a cumulative distance estimate:

$$u_t = \frac{g_t}{\sqrt{v_t} + \epsilon} \quad \text{(unnormalized adaptive step)} \tag{54}$$

$$s_t = s_{t-1} + \langle u_t, \theta_t - \theta_0 \rangle \quad \text{(cumulative signed distance)} \tag{55}$$

$$d_t = \max(d_{t-1}, d_0 + d_{\text{coef}} \cdot s_t) \quad \text{(distance estimate with lower bound)} \tag{56}$$

$$\theta_{t+1} = \theta_t - \eta \cdot d_t \cdot u_t \tag{57}$$

where d_0 is an initial bound and d_{coef} controls how aggressively the estimate adapts. The distance estimate d_t captures the combined magnitude of past gradients weighted by actual parameter displacement, implementing an implicit effective learning rate. This distance-aware scaling achieves robust convergence across problem scales and batch sizes without requiring a learning-rate schedule. Prodigy reduces hyperparameter sensitivity by estimating step sizes from optimization geometry rather than problem-specific prior knowledge.

Algorithm 15 Prodigy (Approximating the Distance Estimate)

Require: Initial parameters θ_0, learning rate η, coeff d_{coef}, initial dist $d_0 > 0$
Ensure: Updated parameters θ_T

Initialize: $s \leftarrow 0$ (cumulative signed distance), $d \leftarrow d_0$
for $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ ▷ Second moment
  $u_t \leftarrow g_t / (\sqrt{v_t} + \epsilon)$ ▷ Unnormalized adaptive
  Update distance: $s \leftarrow s + \langle u_t, \theta_t - \theta_0 \rangle$ ▷ Signed distance
  $d \leftarrow \max(d, d_0 + d_{\text{coef}} \cdot s)$ ▷ Lower-bounded distance
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot d \cdot u_t$ ▷ Distance-scaled update
end forreturn θ_T

4.5 Second-Order, Geometric, and Orthogonality Optimizers

Algorithm 16 Shampoo (Matrix Preconditioning via Kronecker Products)

Require: Initial parameters θ_0, learning rate η, eigendecomposition interval K
Ensure: Updated parameters θ_T

Initialize: $L \leftarrow \epsilon I_m$, $R \leftarrow \epsilon I_n$ (left and right Gram matrices)
for $t = 1, 2, \ldots, T$ **do**
  $G \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradient
  Update Gram matrices: $L \leftarrow L + GG^\top$, $R \leftarrow R + G^\top G$
  if $t \mod K == 0$ **then**
    $Q_L, \Lambda_L \leftarrow \text{eigh}(L)$ ▷ Eigendecomposition of L
    $Q_R, \Lambda_R \leftarrow \text{eigh}(R)$ ▷ Eigendecomposition of R
    $L^{-1/4} \leftarrow Q_L (\Lambda_L + \epsilon)^{-1/4} Q_L^\top$
    $R^{-1/4} \leftarrow Q_R (\Lambda_R + \epsilon)^{-1/4} Q_R^\top$
  end if
  $\Delta\theta \leftarrow L^{-1/4} G R^{-1/4}$ ▷ Preconditioned gradient
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$
end forreturn θ_T

Shampoo (Matrix Preconditioning via Kronecker Products). Shampoo is a **structured second-order method** that computes matrix preconditioners from Kronecker-structured

outer-product statistics. For a 2D parameter matrix $W \in \mathbb{R}^{m \times n}$:

$$L_t = L_{t-1} + G_t G_t^\top \quad \text{(left/row Gram matrix, } m \times m\text{)} \tag{58}$$

$$R_t = R_{t-1} + G_t^\top G_t \quad \text{(right/column Gram matrix, } n \times n\text{)} \tag{59}$$

$$L_t^{-1/4} = Q_L (\Lambda_L)^{-1/4} Q_L^\top \quad \text{(via eigendecomposition)} \tag{60}$$

$$R_t^{-1/4} = Q_R (\Lambda_R)^{-1/4} Q_R^\top \tag{61}$$

$$\Delta W_t = L_t^{-1/4} G_t R_t^{-1/4} \quad \text{(preconditioned gradient)} \tag{62}$$

Shampoo approximates the full-matrix Adagrad preconditioner $H^{-1/2}$ (where H is the Hessian) using Kronecker-factored structure. Instead of storing a $(mn) \times (mn)$ preconditioner, Shampoo maintains two smaller matrices ($m \times m$ and $n \times n$), capturing cross-parameter correlations within and across groups. The method has proven effective at scale (Google production systems) and is well-suited to transformer architectures with high-rank structure. Eigendecompositions are performed periodically (e.g., every $K = 10$ steps) to amortize cost. Tradeoff: higher per-step compute and periodic $O(m^3 + n^3)$ eigendecomposition versus improved conditioning and accelerated convergence.

SOAP (Shampoo with Adam in eigenbasis). SOAP is a simplified variant of Shampoo that decouples preconditioning from momentum tracking. Instead of complex matrix algebra on preconditioned gradients, SOAP runs standard Adam in the eigenbasis of Shampoo's preconditioners:

$$L_t = L_{t-1} + G_t G_t^\top, \quad R_t = R_{t-1} + G_t^\top G_t \quad \text{(Gram accumulation)} \tag{63}$$

$$Q_L, \Lambda_L = \text{eigh}(L_t), \quad Q_R, \Lambda_R = \text{eigh}(R_t) \quad \text{(periodic eigendecomposition)} \tag{64}$$

$$G_t^{\text{rot}} = Q_L^\top G_t Q_R \quad \text{(rotate gradient into eigenbasis)} \tag{65}$$

$$m_t^{\text{rot}} = \beta_1 m_{t-1}^{\text{rot}} + (1 - \beta_1) G_t^{\text{rot}} \tag{66}$$

$$v_t^{\text{rot}} = \beta_2 v_{t-1}^{\text{rot}} + (1 - \beta_2)(G_t^{\text{rot}})^2 \quad \text{(Adam in rotated space)} \tag{67}$$

$$U_t^{\text{rot}} = \frac{m_t^{\text{rot}}}{\sqrt{v_t^{\text{rot}}} + \epsilon}, \quad U_t = Q_L U_t^{\text{rot}} Q_R^\top \quad \text{(rotate back)} \tag{68}$$

The key insight: all momentum tracking occurs in the well-conditioned eigenbasis, simplifying numerical stability and theoretical analysis. SOAP combines benefits of Shampoo (explicit curvature structure) and Adam (proven momentum mechanics), while being cleaner and often more stable than full Shampoo. Eigendecompositions are recomputed every K steps, amortizing the cost.

Algorithm 17 SOAP (Shampoo with Adam in Eigenbasis)

Require: Initial parameters θ_0, learning rate η, β_1, β_2, eigendecomposition interval K
Ensure: Updated parameters θ_T
Initialize: $L \leftarrow \epsilon I_m$, $R \leftarrow \epsilon I_n$, $m \leftarrow 0$, $v \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  $G \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradient
  Update Gram: $L \leftarrow L + GG^\top$, $R \leftarrow R + G^\top G$
  if $t \mod K == 0$ **then**
    $Q_L, \Lambda_L \leftarrow \text{eigh}(L)$, $Q_R, \Lambda_R \leftarrow \text{eigh}(R)$
  end if
  $G^{\text{rot}} \leftarrow Q_L^\top G Q_R$ ▷ Rotate gradient into eigenbasis
  $m \leftarrow \beta_1 m + (1 - \beta_1) G^{\text{rot}}$ ▷ Exponential moving average (bias-correct outside)
  $v \leftarrow \beta_2 v + (1 - \beta_2)(G^{\text{rot}})^2$ ▷ Second moment (bias-correct outside)
  $u \leftarrow m/(\sqrt{v} + \epsilon)$ ▷ Adaptive step in eigenbasis
  $\Delta\theta \leftarrow Q_L u Q_R^\top$ ▷ Rotate back to parameter space
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$
end forreturn θ_T

Lion (EvoLved Sign Momentum). Lion achieves **minimal memory overhead** by using sign-based updates instead of full adaptive scaling:

$$c_t = \beta_1 m_t + (1 - \beta_1) g_t \quad \text{(momentum input)} \tag{69}$$

$$\theta_{t+1} = \theta_t - \eta \left(\text{sign}(c_t) + \lambda \theta_t\right) \quad \text{(sign operator update)} \tag{70}$$

$$m_{t+1} = \beta_2 m_t + (1 - \beta_2) g_t \quad \text{(momentum accumulation)} \tag{71}$$

All element-wise scaling operations are replaced with `sign()`, which outputs $\{-1, 0, +1\}$. This dramatically reduces memory compared to Adam-like methods: Lion stores only the momentum buffer, no second moment variance. The tradeoff: sign-based updates sacrifice the element-wise learning-rate adaptation that makes Adam effective. Lion is positioned not as a universally superior optimizer but as a specialized low-memory, high-throughput alternative for scenarios where activation memory dominates and some optimizer sophistication can be sacrificed. Works well in large-batch regimes and when hardware throughput is the primary constraint.

Algorithm 18 Lion (EvoLved Sign Momentum)

Require: Initial parameters θ_0, learning rate η, β_1, β_2, weight decay λ
Ensure: Updated parameters θ_T
Initialize: $m \leftarrow 0$ (momentum buffer)
for $t = 1, 2, \ldots, T$ **do**
  $g \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradient
  $c \leftarrow \beta_1 m + (1 - \beta_1) g$ ▷ Momentum input
  $\theta_t \leftarrow \theta_{t-1} - \eta(\text{sign}(c) + \lambda \theta_{t-1})$ ▷ Sign-based update with weight decay
  $m \leftarrow \beta_2 m + (1 - \beta_2) g$ ▷ Momentum accumulation (decoupled)
end forreturn θ_T

Sophia (Second-Order Hessian Information with Optimized Approximation). Sophia uses **diagonal Hessian estimates** for curvature-aware scaling without the memory and com-

pute cost of dense second-order methods.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{(first moment)} \tag{72}$$

$$h_t = \beta_2 h_{t-(k-1)} + (1-\beta_2)\hat{h}_t \quad \text{(diagonal Hessian, updated every } k \text{ steps)} \tag{73}$$

$$\text{clip}(x, C) = \min(\max(x, -C), C) \quad \text{(element-wise clipping)} \tag{74}$$

$$\theta_{t+1} = \theta_t - \eta \cdot \text{clip}\left(\frac{m_t}{\max(\gamma h_t, \epsilon)}, 1\right) \tag{75}$$

where $\hat{h}_t$ is a diagonal estimate (e.g., Hutchinson-trace estimator) and γ is a scaling factor. The diagonal Hessian h_t captures local curvature, allowing the optimizer to take smaller steps in sharp directions and larger steps in flat directions. The clipping operation clip(*cdot*, 1) prevents adaptive steps from exploding. Sophia belongs to the family of curvature-aware methods seeking better conditioning without the $O(n^2)$ or $O(n^3)$ cost of full second-order methods. Works particularly well in second-pass fine-tuning scenarios.

Algorithm 19 Sophia (Diagonal Hessian with Clipped Updates)

Require: Initial parameters θ_0, learning rate η, β_1, β_2, Hessian update freq k, clip threshold C
Ensure: Updated parameters θ_T

Initialize: $m \leftarrow 0$, $h \leftarrow \epsilon$ (diagonal Hessian estimate)
for $t = 1, 2, \ldots, T$ **do**
  $g \leftarrow \nabla f(\theta_{t-1})$
  $m \leftarrow \beta_1 m + (1-\beta_1) g$ ▷ First moment (bias-correct outside)
  if $t \mod k == 0$ **then**
    $\hat{h} \leftarrow \text{HutchinsonEstimate}(g)$ ▷ Diagonal Hessian via Hutchinson
    $h \leftarrow \beta_2 h + (1-\beta_2)\hat{h}$ ▷ Update Hessian estimate
  end if
  $u \leftarrow \frac{m}{\max(\gamma h, \epsilon)}$ ▷ Adaptive step (element-wise)
  $u \leftarrow \text{clip}(u, C)$ ▷ Element-wise clipping to $[-C, C]$
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot u$
end forreturn θ_T

Muon and Turbo-Muon (Orthogonality-Based Optimizers). Muon and Turbo-Muon are **orthogonality-oriented optimizers** that reshape update geometry using Newton-Schulz polynomial iterations for orthogonalization. For a 2D parameter matrix W with gradient G_t:

$$X_0 = \frac{G_t}{\|G_t\|_F + \epsilon} \quad \text{(normalized gradient)} \tag{76}$$

$$A_k = X_k X_k^\top \quad \text{(Gramian)} \tag{77}$$

$$B_k = bA_k + cA_k^2 \quad \text{(polynomial step, coefficients } b, c \text{ from Newton-Schulz)} \tag{78}$$

$$X_{k+1} = aX_k + B_k X_k \quad \text{(iteration } k = 0, 1, \ldots, 4) \tag{79}$$

$$W_{t+1} = W_t - \eta X_K \quad \text{(update with orthogonalized direction)} \tag{80}$$

Muon performs 5 Newton-Schulz iterations to produce an approximately orthogonal direction, ensuring updates respect geometric constraints. Turbo-Muon adds an **almost-orthogonal preconditioning** (AOL) step before iterations, reducing the number of required orthogonalization steps to 4. The rationale: orthogonal updates preserve norms during training, avoiding the norm creep observed in momentum-based methods. These optimizers show promise on large-scale training but require custom CUDA kernels for efficiency. Tradeoff: high per-step compute (matrix multiplications and polynomial iterations) versus fundamentally better-conditioned update geometry.

Algorithm 20 Muon (Newton-Schulz Orthogonalization)

Require: Initial parameters θ (matrices), learning rate η, Newton-Schulz iters K
Ensure: Updated parameters θ

for each 2D parameter matrix W **do**
  $G \leftarrow \nabla f(W)$ ▷ Gradient
  $X_0 \leftarrow \frac{G}{\|G\|_F+\epsilon}$ ▷ Normalize gradient by Frobenius norm
  for $k = 0, 1, \ldots, K-1$ **do**
    $A_k \leftarrow X_k X_k^\top$ ▷ Gramian (cost: $O(mn^2)$ or $O(m^2n)$ for tall/wide)
    $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$ ▷ Newton-Schulz coefficients
    $X_{k+1} \leftarrow B_k X_k$ ▷ Polynomial step
  end for
  $W \leftarrow W - \eta X_K$ ▷ Update with orthogonalized direction
end forreturn updated θ

Algorithm 21 Turbo-Muon (AOL-Preconditioned Orthogonalization)

Require: Initial parameters θ (matrices), learning rate η, Newton-Schulz iters K' (typically 4)
Ensure: Updated parameters θ

for each 2D parameter matrix W **do**
  $G \leftarrow \nabla f(W)$ ▷ Gradient
  $X_0 \leftarrow \frac{G}{\|G\|_F+\epsilon}$ ▷ Normalize gradient
  AOL (Almost-Orthogonal preconditioner): ▷ **Preconditioning step**
  $P \leftarrow (1.5I - 0.5X_0X_0^\top)$ ▷ **Preconditioning matrix**
  $X_0 \leftarrow PX_0$ ▷ **Preconditioned start**
  for $k = 0, 1, \ldots, K'-1$ **do**
    $A_k \leftarrow X_k X_k^\top$
    $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$
    $X_{k+1} \leftarrow B_k X_k$ ▷ **Reduced iterations due to AOL preconditioning**
  end for
  $W \leftarrow W - \eta X_{K'}$ ▷ **Update with preconditioned orthogonalized direction**
end forreturn updated θ

Group	Methods	Primary Goal	Interpretation from the Survey
Classical baseline	SGD, AdamW, RAdam	stability and reference baselines	These define the comparison floor for newer optimizer claims.
Momentum redesign	Adan, AdEMAMix, MARS, Cautious AdamW	faster or safer first-order adaptation	Best when convergence speed or noisy-gradient stability is the main concern.
Large-batch and schedule simplification	LAMB, Schedule-Free AdamW	operational robustness at scale	Reduce brittleness from batch-size growth or schedule engineering.
Memory-efficient	Adafactor, GaLore, APOLLO, APOLLO-Mini, Q-APOLLO, Lion	optimizer-state reduction	Most useful when VRAM is dominated by optimizer state rather than activations; APOLLO-family methods replace dense AdamW moments with projected or quantized structured scaling.

Group	Methods	Primary Goal	Interpretation from the Survey
Curvature-aware	Shampoo, SOAP, Sophia	better conditioning	Prefer when richer geometry is worth implementation and compute overhead.
Geometry-oriented	Muon, Turbo-Muon	orthogonalized update structure	Specialized options for matrix geometry and representation shaping.

5 Dense, Recurrent, and Memory-Augmented Transformers

This annex synthesizes the dense, recurrent, and memory-augmented transformer families. The field has evolved toward several primary paradigms: dense global attention with variants, key-value head compression via grouped-query attention, recurrent state compression with decay, selective state-space models, continuous-depth numerical integration, and memory-augmented architectures. Understanding this taxonomy illuminates fundamental tradeoffs between expressiveness, computational cost, memory footprint, and deployment simplicity.

5.1 Dense Attention Baselines: Standard and Sigmoid

5.1.1 Standard Softmax Attention

Standard multi-head self-attention [79] computes scaled dot-product similarity between token embeddings:

1: **Input:** Query matrix $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Key matrix $\mathbf{K} \in \mathbb{R}^{n \times d}$, Value matrix $\mathbf{V} \in \mathbb{R}^{n \times d}$
2: scores $\leftarrow \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \in \mathbb{R}^{n \times n}$ ▷ compute pairwise similarities
3: attention_weights $\leftarrow \texttt{softmax}(\text{scores}, \text{axis} = 1) \in \mathbb{R}^{n \times n}$ ▷ normalize across keys
4: **Output:** $\mathbf{Y} = \text{attention\_weights} \cdot \mathbf{V} \in \mathbb{R}^{n \times d}$ ▷ weighted value combination

For autoregressive generation, KV caching stores past keys and values to avoid $\mathcal{O}(n^2)$ recomputation. However, this linearly growing cache occupies $\sim n \cdot d_{\text{hidden}}$ bytes, which can consume hundreds of gigabytes for billion-parameter models. Standard attention achieves **perfect expressiveness** within the context window—any token can attend to any other token with learned weights—but pays the price of dense computation.

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ time, $\mathcal{O}(n^2)$ space (attention matrix materialization).
- **Inference complexity**: $\mathcal{O}(n)$ time per token, $\mathcal{O}(n)$ space (KV cache).
- **Strengths**: Unparalleled expressiveness; perfect history recall; highly parallelizable.
- **Weaknesses**: Quadratic bottleneck prohibits very long contexts; KV cache dominates memory during generation.

5.1.2 Sigmoid Attention

Sigmoid attention replaces row-wise softmax with element-wise sigmoid activation [67]:

1: **Input:** $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ as above, learnable bias $\mathbf{b} \in \mathbb{R}^{n \times n}$
2: logits $\leftarrow \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{b}$
3: attention_weights $\leftarrow \sigma(\text{logits})$ ▷ element-wise sigmoid, not softmax
4: **Output:** $\mathbf{Y} = \text{attention\_weights} \odot \mathbf{V}$

Unlike softmax, sigmoid does not enforce a probability distribution (weights need not sum to 1), enabling stronger token independence. Theoretical analysis via mixture-of-experts shows sigmoid achieves superior sample complexity: $\mathcal{O}(n^{-0.51})$ convergence for ReLU experts versus softmax's $\mathcal{O}(n^{-0.24})$. However, empirical training revealed gradient instabilities at scale. The remedy is **hybrid-norm**—adding normalization after the attention output—which stabilizes gradients without sacrificing the theoretical benefits of element-wise gating.

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ (identical to standard), but element-wise operations enable 17% inference speedup via FlashSigmoid.
- **Inference complexity**: $\mathcal{O}(n)$ per token with KV cache (asymptotically same, but lower constant factors).
- **Strengths**: Overcomes zero-sum competition; avoids row-wise synchronization; hardware-friendly implementation.
- **Weaknesses**: Requires careful stabilization (hybrid-norm); training instability at large scales without auxiliary loss.

5.1.3 Grouped-Query Attention (GQA)

Grouped-query attention [2] bridges multi-head attention (MHA) and multi-query attention (MQA) by partitioning query heads into groups, each sharing a single key-value head. If h is the total number of attention heads, h_k is the number of key-value heads, and $G = h/h_k$ is the group size, then every group of G query heads shares one KV head:

Input: $\mathbf{Q} \in \mathbb{R}^{n \times h \cdot d_h}$, $\mathbf{K} \in \mathbb{R}^{n \times h_k \cdot d_h}$, $\mathbf{V} \in \mathbb{R}^{n \times h_k \cdot d_h}$
$\mathbf{K}_{\text{full}} \leftarrow \texttt{repeat\_interleave}(\mathbf{K}, G)$ ▷ expand from h_k to h heads
$\mathbf{V}_{\text{full}} \leftarrow \texttt{repeat\_interleave}(\mathbf{V}, G)$
$\text{scores} \leftarrow \frac{\mathbf{Q}\mathbf{K}_{\text{full}}^\top}{\sqrt{d_h}} \in \mathbb{R}^{n \times n}$ ▷ standard dot product after expansion
$\text{attention\_weights} \leftarrow \texttt{softmax}(\text{scores}, \text{axis} = 1)$
Output: $\mathbf{Y} = \text{attention\_weights} \cdot \mathbf{V}_{\text{full}} \in \mathbb{R}^{n \times h \cdot d_h}$

GQA interpolates smoothly between MHA ($h_k = h$, $G = 1$) and MQA ($h_k = 1$, $G = h$). The key advantage is a tunable tradeoff between KV-cache footprint and representational capacity. Each additional KV head consumes $2 \cdot d_h \cdot n$ extra bytes in the cache, so reducing h_k yields proportional memory savings.

- **Training complexity**: $\mathcal{O}(n^2 \cdot h \cdot d_h)$ (identical to MHA after head expansion).
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(h_k \cdot n \cdot d_h)$ KV-cache space.
- **Strengths**: Flexible head-count tradeoff; proven at scale (Llama 2 & 3 use $h_k = 8$ KV heads with $h = 32$ query heads); reduces KV-cache by $G\times$; supports fast inference with FlashAttention.
- **Weaknesses**: Fewer KV heads can reduce representational capacity in small models; requiring h_k to divide h constrains architectural choices.

5.2 Recurrent and Retentive Architectures

5.2.1 Retentive Networks (RetNet)

RetNet [76] unifies three computation paradigms: parallel training, recurrent inference, and chunkwise deployment. Its core innovation is the **retention mechanism**, which uses a fixed exponential decay matrix to model temporal importance:

Parallel (training) form:
decay_matrix$[i,j] \leftarrow \gamma^{i-j}$ for $i \geq j$, else 0 ▷ causal exponential decay
decay_matrix$[i,j] \leftarrow 0$ for $i < j$ ▷ causal masking
$\mathbf{Y}_{\text{parallel}} \leftarrow (\mathbf{Q}\mathbf{K}^\top \odot \text{decay\_matrix})\mathbf{V}$ ▷ element-wise multiplication with decay
Recurrent (inference) form:
for $t = 1, \ldots, n$ **do**
    $\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$ ▷ state update with decay
    $\mathbf{y}_t \leftarrow \mathbf{q}_t \mathbf{s}_t$ ▷ output via query-state interaction
end for

The decay scalar $\gamma \in (0, 1)$ controls the temporal window. RetNet uses **multi-scale retention** with different γ values per head (e.g., $\gamma = 1 - 2^{-5}, 1 - 2^{-6}, \ldots$), allowing short-term and long-term dependencies simultaneously. Chunkwise recurrent mode divides sequences into chunks, processes each chunk in parallel, and threads a recurrent state between chunks.

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ (parallel), or $\mathcal{O}(n \cdot c \cdot d)$ (chunkwise recurrent with chunk size c).
- **Inference complexity**: $\mathcal{O}(1)$ per token, $\mathcal{O}(d^2)$ state space (fixed matrix).
- **Strengths**: Constant-time inference; triple computation paradigm; multi-scale consolidation.
- **Weaknesses**: Fixed decay imposes rigid inductive bias; may truncate learned long-range patterns.

5.2.2 Mamba: Selective State-Space Models

Mamba [31] frames recurrence as a continuous-time dynamical system with input-dependent parameters, achieving both linear training and constant-time inference:

Continuous dynamics: $h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}h(t)$
Discretization with step size Δ_t:
$\bar{\mathbf{A}}_t \leftarrow \exp(\Delta_t \mathbf{A})$
$\bar{\mathbf{B}}_t \leftarrow (\Delta_t \mathbf{A})^{-1}(\exp(\Delta_t \mathbf{A}) - \mathbf{I})\Delta_t \mathbf{B}_t$
Recurrent update:
for $t = 1, \ldots, n$ **do**
    $\Delta_t \leftarrow \text{softplus}(\text{Linear}(x_t))$ ▷ input-dependent step size
    $\mathbf{B}_t \leftarrow \text{Linear}(x_t), \quad \mathbf{C}_t \leftarrow \text{Linear}(x_t)$ ▷ input-dependent projection matrices
    $h_t \leftarrow \bar{\mathbf{A}}_t h_{t-1} + \bar{\mathbf{B}}_t x_t$ ▷ state transition
    $y_t \leftarrow \mathbf{C}_t h_t$ ▷ output projection
end for

The critical innovation is that $\mathbf{A}$ (the system matrix) is *not* input-dependent, but Δ_t, $\mathbf{B}_t$, and $\mathbf{C}_t$ are, making the system **time-varying**. This selectivity allows the model to *ignore* irrelevant information by setting Δ_t near zero, effectively creating a gate. A hardware-aware parallel scan algorithm implements the recurrence efficiently on GPUs by fusing computation within SRAM, avoiding expensive HBM bandwidth.

- **Training complexity**: $\mathcal{O}(n \cdot d)$ via hardware-aware scan (linear in sequence length).
- **Inference complexity**: $\mathcal{O}(1)$ per token, $\mathcal{O}(d)$ state (hidden vector).
- **Strengths**: Achieves linear training and constant-inference simultaneously; practical on long sequences (millions of tokens).
- **Weaknesses**: State vector compression can weaken exact copying and dense associative recall versus full attention.

5.3 Continuous-Depth Transformers: ODE Integration

5.3.1 ODE Transformer

The ODE Transformer [93] interprets network depth as numerical integration of a continuous dynamical system, using higher-order Runge-Kutta solvers to reduce truncation error:

1: **Continuous formulation:** $\frac{dh(t)}{dt} = f_\theta(h(t), t)$ where f_θ is the transformer sub-network
2: **Runge-Kutta-4 discrete approximation:**
3: $k_1 \leftarrow f_\theta(h_t, t)$
4: $k_2 \leftarrow f_\theta(h_t + \frac{1}{2}k_1, t + \frac{1}{2}\Delta t)$
5: $k_3 \leftarrow f_\theta(h_t + \frac{1}{2}k_2, t + \frac{1}{2}\Delta t)$
6: $k_4 \leftarrow f_\theta(h_t + k_3, t + \Delta t)$
7: $h_{t+1} \leftarrow h_t + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ ▷ weighted combination

Instead of simple Euler residual connections $h_{t+1} = h_t + f_\theta(h_t)$, the RK4 block computes four intermediate evaluations and combines them with classical RK4 weights. To avoid vanishing gradients, the architecture introduces **learned gating** that interpolates between intermediate approximations:

1: $g \leftarrow \sigma(\text{Linear}([k_1, k_2, k_3, k_4]))$ ▷ learnable gate
2: $h_{t+1} \leftarrow h_t + g \cdot k_1 + (1 - g) \cdot k_2$ ▷ interpolated refinement

This formulation reduces the effective number of parameters through weight sharing—the same f_θ is evaluated multiple times—while providing richer trajectory refinement.

- **Training complexity**: $\mathcal{O}(k \cdot n^2 \cdot d)$ where k is the RK order (4 for RK4).
- **Inference complexity**: $\mathcal{O}(k \cdot n)$ per token (higher constant overhead).
- **Strengths**: Significantly higher accuracy on generation tasks (state-of-the-art BLEU); parameter-efficient via weight sharing.
- **Weaknesses**: High per-step compute; inference latency increases by constant factor k; complex gating required for stability.

5.4 Test-Time Memory: Titans

The Titans architecture [7] introduces an orthogonal dimension: instead of static weights, the model maintains a learnable memory that is *updated during inference* based on a surprise-driven signal:

1: **Local short-term attention:** Apply standard or sparse attention within a fixed context window c
2: $y_t^{\text{local}} \leftarrow \text{Attention}(q_t, k_{[t-c:t]}, v_{[t-c:t]})$
3: **Memory update signal (surprise):**
4: $S_t \leftarrow \eta_t S_{t-1} - \theta_t \nabla_\ell(\mathcal{M}_{t-1}; x_t)$ ▷ surprise as gradient of loss w.r.t. memory
5: $\mathcal{M}_t \leftarrow (1 - \alpha_t)\mathcal{M}_{t-1} + S_t$ ▷ memory updated via EMA
6: **Long-term memory retrieval:**
7: $y_t^{\text{memory}} \leftarrow \mathcal{M}_t^*(q_t)$ ▷ query memory module for retrieved information
8: **Output combination:**
9: $y_t \leftarrow y_t^{\text{local}} + \text{gate}(y_t^{\text{memory}})$ ▷ combine local and retrieved memory

The memory module $\mathcal{M}$ is literally updated during the forward pass by computing gradients of an associative loss and applying SGD steps with momentum. The decay rates $\eta_t, \theta_t, \alpha_t$ are themselves input-dependent, allowing the model to switch memory paradigms when context shifts.

- **Training complexity**: Roughly $\mathcal{O}(c^2 + n \cdot f)$ where c is local window and f is memory update overhead.

Algorithm 22 Pattern-Driven Mixer Forward (Conceptual)

Require: Hidden states H, pattern P, layer index ℓ

$m \leftarrow P[\ell \bmod |P|]$
if $m \in \{\texttt{fasa\_attn}, \texttt{sparge\_attn}\}$ and model is in training mode **then**
    raise configuration/runtime error (training-free block in train mode)
else if $m = \texttt{standard\_attn}$ **then**
    $H \leftarrow$ softmax-attention(H)
else if $m = \texttt{sigmoid\_attn}$ **then**
    $H \leftarrow$ sigmoid-attention(H)
else if $m \in \{\texttt{retnet}, \texttt{retnet\_attn}\}$ **then**
    $H \leftarrow$ retention(H)
else if $m = \texttt{mamba}$ **then**
    $H \leftarrow$ selective-ssm(H)
else if $m = \texttt{ode}$ **then**
    $H \leftarrow$ rk-step(H)
else if m is a sparse attention key **then**
    $H \leftarrow$ sparse-attention-family(H)
else if m is a gated attention key **then**
    $H \leftarrow$ gated-attention-family(H)
else
    $H \leftarrow$ memory-augmented-attn(H)
end if
return H

- **Inference complexity**: $\mathcal{O}(c^2)$ local attention plus $\mathcal{O}(1)$ memory retrieval per token.
- **Strengths**: Handles extreme context lengths; enables true associative recall; memory adapts to input.
- **Weaknesses**: Significantly more complex; inference includes gradient computation; higher coordination overhead.

5.5 Pattern-Driven Mixer Dispatch

The codebase selects a mixer per block from the configured `layer_pattern`. The dispatch logic enforces the training-free policy for eval-only blocks and routes to the corresponding family implementation.

The field exhibits a clear progression along two axes: (i) **computational efficiency**, moving from $O(n^2)$ to $O(n)$ or $O(1)$, and (ii) **memory adaptivity**, shifting from static weights to dynamic, test-time updated models. Standard attention remains the expressiveness baseline; Mamba and RetNet represent the practical efficiency frontier; Titans introduces an orthogonal innovation axis (test-time learning). The choice of architecture reflects the fundamental engineering constraint: expressiveness versus deployment cost. The latent-attention family, which compresses the KV cache into a low-rank latent, is covered in Appendix 6.

6 Latent and Low-Rank Attention Families

This annex covers the latent / low-rank KV compression family of attention mechanisms. These architectures compress the per-token key–value state into a low-rank latent vector that is the only quantity materialised in the KV cache during decoding, then reconstruct full multi-head keys and values on the fly. The family unifies grouped-query, latent, factorised, and temporal-latent designs under a common low-rank bottleneck.

6.1 Multi-Head Latent Attention (MLA)

Multi-Head Latent Attention (MLA) [54] compresses the per-token key–value state into a single low-rank latent vector c_{KV} of rank r_{kv}, which is the only quantity materialised in the KV cache during decoding. A pair of up-projection matrices reconstructs the full multi-head keys and values on the fly, while RoPE is applied to the decompressed query and key tensors so that positional structure is preserved. The authors show that the Pareto-optimal latent width sits near $r_{kv} = d_{\text{hidden}}/2$, at which point MLA halves the KV cache yet matches multi-head attention (MHA) quality on small models.

6.1.1 Mathematical Formulation

$$c_{KV} = W_{DKV}\, x \in \mathbb{R}^{r_{kv}}, \qquad k = W_{UK}\, c_{KV}, \qquad v = W_{UV}\, c_{KV}, \qquad q = W_Q\, x.$$

$$\tilde{q} = \mathrm{RoPE}(q), \quad \tilde{k} = \mathrm{RoPE}(k), \qquad \text{attn} = \texttt{softmax}\left(\frac{\tilde{q}\,\tilde{k}^\top}{\sqrt{d_h}}\right) v.$$

6.1.2 Algorithmic Pseudocode

Input: hidden state $x \in \mathbb{R}^{n \times d}$, rank r_{kv}, projections $W_{DKV}, W_{UK}, W_{UV}, W_Q$
$c_{KV} \leftarrow W_{DKV}\, x$ ▷ down-project to latent of rank r_{kv} (cached at inference)
$k \leftarrow W_{UK}\, c_{KV}, \quad v \leftarrow W_{UV}\, c_{KV}$ ▷ up-project to full heads
$q \leftarrow W_Q\, x$
$\tilde{q} \leftarrow \mathrm{RoPE}(q), \quad \tilde{k} \leftarrow \mathrm{RoPE}(k)$ ▷ apply rotary embeddings
weights $\leftarrow \texttt{softmax}(\tilde{q}\,\tilde{k}^\top/\sqrt{d_h})$
Output: $\mathbf{Y} = \text{weights} \cdot v$

6.1.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ (full attention on decompressed heads).
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(r_{kv} \cdot n)$ KV-cache space (latent only).
- **Trainable**: fully trainable; r_{kv} is a hyperparameter (Pareto-optimal $\approx d/2$).
- **Strengths**: Halves KV cache versus MHA at matched quality; RoPE-compatible; clean latent bottleneck.
- **Limitations**: Up-projection adds decode FLOPs; latent rank is a fixed global budget shared across heads.

6.2 Group-Query Latent Attention (GQLA)

Group-Query Latent Attention (GQLA) [55] is a minimal modification of MLA that exposes two algebraically equivalent decoding paths over the same trained weights: an MQA-absorb path (fusing the up-projection into the query, optimal on H100-class hardware) and a GQA-expanded path (materialising full keys/values, optimal on H20 / multi-token-prediction hardware). The two paths are mathematically identical because the up-projection is linear, so $\texttt{softmax}(q_{\text{absorbed}} \cdot c_{KV}^\top) = \texttt{softmax}((q \cdot W_{UK}^\top) \cdot c_{KV}^\top)$, allowing hardware-adaptive dispatch with no retraining.

6.2.1 Mathematical Formulation

Same latent as MLA:

$$c_{KV} = W_{DKV}\, x, \qquad k = W_{UK}\, c_{KV}, \qquad v = W_{UV}\, c_{KV}, \qquad q = W_Q\, x.$$

$$\text{Path A (MQA-absorb): attn} = \texttt{softmax}\left(\frac{(q\,W_{UK}^{\top})\,c_{KV}^{\top}}{\sqrt{d_h}}\right)(W_{UV}\,c_{KV}).$$

$$\text{Path B (GQA-expanded): attn} = \texttt{softmax}\left(\frac{q\,k^{\top}}{\sqrt{d_h}}\right)v, \quad k = W_{UK}c_{KV},\ v = W_{UV}c_{KV}.$$

6.2.2 Algorithmic Pseudocode

Input: x, weights $W_{DKV}, W_{UK}, W_{UV}, W_Q$, hardware flag hw $\in \{\text{H100}, \text{H20}\}$
$c_{KV} \leftarrow W_{DKV}\,x$ ▷ shared latent, cached
$q \leftarrow W_Q\,x$
if hw == H100 **then** ▷ MQA-absorb path
  $q_{\text{abs}} \leftarrow q\,W_{UK}^{\top}$ ▷ absorb up-projection into query
  weights $\leftarrow \texttt{softmax}(q_{\text{abs}}\,c_{KV}^{\top}/\sqrt{d_h})$
  $\mathbf{Y} \leftarrow \text{weights} \cdot (W_{UV}\,c_{KV})$
else ▷ GQA-expanded path
  $k \leftarrow W_{UK}\,c_{KV}, \quad v \leftarrow W_{UV}\,c_{KV}$
  weights $\leftarrow \texttt{softmax}(q\,k^{\top}/\sqrt{d_h})$
  $\mathbf{Y} \leftarrow \text{weights} \cdot v$
end if
Output: Y

6.2.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$, identical to MLA.
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(r_{kv} \cdot n)$ latent state; path chosen by hardware.
- **Trainable**: trained once; the two decoding paths reuse the same weights.
- **Strengths**: Hardware-adaptive decode with no quality loss; algebraic equivalence guarantees consistency.
- **Limitations**: Same latent-rank tradeoffs as MLA; path selection is a deployment-time heuristic.

6.3 Multi-Head Low-Rank Attention (MLRA)

Multi-Head Low-Rank Attention (MLRA) [50] refactors the single MLA latent into L disjoint sub-heads each of rank r/L, so that the KV cache can be partitioned across L devices for 4-way tensor-parallel decoding (each device loads only $1/L$ of the cache). Per-sub-head up-projections reconstruct the corresponding slice of the full heads and are concatenated, yielding a 2.8× decode speedup over MLA while preserving the global latent budget.

6.3.1 Mathematical Formulation

$$c_{KV} = [c_1, c_2, \ldots, c_L], \quad c_i \in \mathbb{R}^{r/L}, \qquad c_{KV} = W_{DKV}\,x.$$

$$k_i = W_{UK}^{(i)}\,c_i, \quad v_i = W_{UV}^{(i)}\,c_i, \qquad k = [k_1; \ldots; k_L], \quad v = [v_1; \ldots; v_L].$$

$$\text{attn} = \texttt{softmax}\left(\frac{q\,k^{\top}}{\sqrt{d_h}}\right)v.$$

6.3.2 Algorithmic Pseudocode

Input: x, sub-head count L, per-sub-head projections $W_{UK}^{(i)}, W_{UV}^{(i)}$
$c_{KV} \leftarrow W_{DKV}\, x = [c_1, \ldots, c_L]$ ▷ partition latent into L blocks
for $i = 1, \ldots, L$ (parallel across L devices) **do**
    $k_i \leftarrow W_{UK}^{(i)}\, c_i, \quad v_i \leftarrow W_{UV}^{(i)}\, c_i$
end for
$k \leftarrow \texttt{concat}(k_1, \ldots, k_L), \quad v \leftarrow \texttt{concat}(v_1, \ldots, v_L)$
weights $\leftarrow \texttt{softmax}(q\, k^\top / \sqrt{d_h})$
Output: $\mathbf{Y} = \text{weights} \cdot v$

6.3.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ (full attention on concatenated heads).
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(r_{kv} \cdot n)$ state partitioned across L devices.
- **Trainable**: fully trainable; L controls partition granularity.
- **Strengths**: 2.8× decode speedup; clean 4-way tensor-parallel cache sharding; preserves total latent budget.
- **Limitations**: Requires L to divide r_{kv}; more projection matrices than MLA; inter-device all-reduce on output.

6.4 Tucker Attention

Tucker Attention [42] unifies MHA, GQA and MLA under a single Tucker-style factorisation of the query/key/value weight tensors with independent ranks $q_{\text{rank}}, k_{\text{rank}}, v_{\text{rank}}$. MHA is recovered when all ranks equal the hidden size, GQA when $k_{\text{rank}} = v_{\text{rank}} < d$, and MLA when $k_{\text{rank}} = v_{\text{rank}}$ equals the latent rank. Because the factorisation is a pure linear reparameterisation, the method is fully compatible with FlashAttention and RoPE, and yields an order of magnitude fewer parameters for comparable metrics.

6.4.1 Mathematical Formulation

$$\mathbf{Q} = W_{Q,\text{core}}\, W_{Q,\text{factor}}\, x, \qquad \mathbf{K} = W_{K,\text{core}}\, W_{K,\text{factor}}\, x, \qquad \mathbf{V} = W_{V,\text{core}}\, W_{V,\text{factor}}\, x.$$

$$\text{attn} = \texttt{softmax}\left(\frac{\mathbf{Q}\,\mathbf{K}^\top}{\sqrt{d_h}}\right)\mathbf{V}.$$

Special cases: MHA $\Leftrightarrow q_{\text{rank}} = k_{\text{rank}} = v_{\text{rank}} = d$; GQA $\Leftrightarrow k_{\text{rank}} = v_{\text{rank}} < d$; MLA $\Leftrightarrow k_{\text{rank}} = v_{\text{rank}} = r_{kv}$.

6.4.2 Algorithmic Pseudocode

Input: x, ranks $q_{\text{rank}}, k_{\text{rank}}, v_{\text{rank}}$, core/factor matrices
$\mathbf{Q} \leftarrow W_{Q,\text{core}}\, W_{Q,\text{factor}}\, x$ ▷ rank-q_{rank} factorised query
$\mathbf{K} \leftarrow W_{K,\text{core}}\, W_{K,\text{factor}}\, x$ ▷ rank-k_{rank} factorised key
$\mathbf{V} \leftarrow W_{V,\text{core}}\, W_{V,\text{factor}}\, x$ ▷ rank-v_{rank} factorised value
Apply RoPE to $\mathbf{Q}, \mathbf{K}$ if positional encoding is used
weights $\leftarrow \texttt{softmax}(\mathbf{Q}\,\mathbf{K}^\top / \sqrt{d_h})$ ▷ FlashAttention-compatible
Output: $\mathbf{Y} = \text{weights} \cdot \mathbf{V}$

6.4.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ on the reconstructed tensors.
- **Inference complexity**: $\mathcal{O}(n)$ per token; cache size governed by k_{rank}.
- **Trainable**: ranks are hyperparameters; the factorisation is a drop-in weight reparameterisation.
- **Strengths**: Unifies MHA/GQA/MLA; order-of-magnitude parameter reduction at matched metrics; FlashAttention/RoPE compatible.
- **Limitations**: Two matmuls per projection add latency; rank selection is a per-task tuning problem.

6.5 Interleaved Head Attention (IHA)

Interleaved Head Attention (IHA) [22] constructs P pseudo-heads per original head (typically $P = H$) where each pseudo query/key/value is a learned linear combination of all H original heads. This induces up to P^2 distinct attention patterns per head at an $\mathcal{O}(H^2 P)$ overhead. Theoretically, IHA needs only $\Theta(\sqrt{k}\, n^2)$ parameters versus $\Theta(k\, n^2)$ for MHA on the Polynomial task; empirically it delivers +10–20% on RULER multi-key retrieval, +5.8% on GSM8K and +2.8% on MATH-500.

6.5.1 Mathematical Formulation

$$q_{\text{pseudo}}[p] = \sum_{h=1}^{H} \text{mix}_q[p,h]\, q[h], \quad k_{\text{pseudo}}[p] = \sum_{h} \text{mix}_k[p,h]\, k[h], \quad v_{\text{pseudo}}[p] = \sum_{h} \text{mix}_v[p,h]\, v[h].$$

$$\text{attn}_p = \texttt{softmax}\left(\frac{q_{\text{pseudo}}[p]\, k_{\text{pseudo}}[p]^\top}{\sqrt{d_h}}\right) v_{\text{pseudo}}[p], \qquad \mathbf{Y}[h] = \frac{1}{P}\sum_{p} \text{attn}_p[h].$$

6.5.2 Algorithmic Pseudocode

Input: per-head $q[h], k[h], v[h]$, mixing matrices $\text{mix}_q, \text{mix}_k, \text{mix}_v \in \mathbb{R}^{P\times H}$
for $p = 1, \ldots, P$ **do**
  $q_p \leftarrow \sum_h \text{mix}_q[p,h]\, q[h]$ ▷ learned mix across heads
  $k_p \leftarrow \sum_h \text{mix}_k[p,h]\, k[h], \quad v_p \leftarrow \sum_h \text{mix}_v[p,h]\, v[h]$
  $\text{attn}_p \leftarrow \texttt{softmax}(q_p\, k_p^\top / \sqrt{d_h})\, v_p$
end for
Output: $\mathbf{Y}[h] \leftarrow \frac{1}{P}\sum_p \text{attn}_p[h]$ ▷ average pseudo-head outputs per head

6.5.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot H \cdot P)$ with $P \approx H$, i.e. $\mathcal{O}(n^2 \cdot H^2)$.
- **Inference complexity**: $\mathcal{O}(n)$ per token with KV cache; extra $\mathcal{O}(H^2 P)$ mixing overhead.
- **Trainable**: mixing matrices are learned; P is a hyperparameter.
- **Strengths**: $\Theta(\sqrt{k})$ parameter efficiency on Polynomial; large gains on retrieval and reasoning benchmarks.
- **Limitations**: Quadratic-in-heads mixing cost; more attention patterns to compute; added latency at large H.

6.6 Grouped-head laTenT Attention (GTA)

Grouped-head laTenT Attention (GTA) [75] combines two compressions: (1) a *shared attention map* — one attention-score tensor per group of heads, reused across all heads in the group, shrinking the key cache; and (2) a *nonlinear value decoder* — the value cache is compressed to a latent by a down-projection and reconstructed by a SiLU-gated up-projection before the output projection. GTA cuts attention FLOPs by up to 62.5% versus GQA, shrinks the KV cache by up to 70%, and yields a 2× end-to-end inference speedup.

6.6.1 Mathematical Formulation

$$q_g = \texttt{mean}(q[\text{group } g]), \quad k_g = \texttt{mean}(k[\text{group } g]), \quad \text{attn}_g = \texttt{softmax}\left(\frac{q_g\, k_g^\top}{\sqrt{d_h}}\right) \text{ (reused across the group).}$$

$$v_{\text{latent}} = W_{DV}\, x, \qquad v = \text{SiLU}(W_{UV}\, v_{\text{latent}}), \qquad \mathbf{Y} = \text{attn}_g \cdot v.$$

6.6.2 Algorithmic Pseudocode

Input: q, k per head, hidden x, group partition $\{g\}$
$v_{\text{latent}} \leftarrow W_{DV}\, x$ ▷ compress values to latent (cached)
$v \leftarrow \text{SiLU}(W_{UV}\, v_{\text{latent}})$ ▷ nonlinear value decode
for each group g **do**
    $q_g \leftarrow \texttt{mean}(q[g]), \quad k_g \leftarrow \texttt{mean}(k[g])$ ▷ shared per-group queries/keys
    $\text{attn}_g \leftarrow \texttt{softmax}(q_g\, k_g^\top / \sqrt{d_h})$
    $\mathbf{Y}[g] \leftarrow \text{attn}_g \cdot v[g]$ ▷ one map reused for all heads in g
end for
Output: $\mathbf{Y}$

6.6.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot G \cdot d_h)$ where G is the number of groups (vs H heads for GQA).
- **Inference complexity**: $\mathcal{O}(n)$ per token; value cache $\mathcal{O}(r_v \cdot n)$, key cache reduced by group sharing.
- **Trainable**: group assignment and latent rank are trainable/hyperparameters.
- **Strengths**: up to 62.5% attention-FLOP reduction, 70% KV-cache shrink, 2× end-to-end speedup.
- **Limitations**: Shared map reduces per-head diversity; nonlinear decoder adds latency; grouping must be chosen carefully.

6.7 Multi-head Temporal Latent Attention (MTLA)

Multi-head Temporal Latent Attention (MTLA) [19] extends MLA along the temporal axis: a hyper-network dynamically merges m temporally adjacent KV-cache vectors into a single slot, shrinking the temporal length by a factor of m. A stride-aware causal mask keeps parallel training consistent with autoregressive inference, yielding a 5.3× decode speedup and 8.3× GPU-memory reduction versus MHA on En–De speech translation.

6.7.1 Mathematical Formulation

$$c_{KV}^{(t)} = W_{DKV}\, x^{(t)} \in \mathbb{R}^{r_{kv}} \text{ (per-token latent).}$$

$$\bar{c}_{KV}^{(j)} = \sum_{i=0}^{m-1} \alpha_i\, c_{KV}^{(sj+i)}, \qquad \alpha_i = \texttt{sigmoid}(\text{HyperNet}(c_{KV}^{(sj+i)})),$$

where s is the stride and m the merge window. Queries attend over merged slots $\bar{c}_{KV}$ with a stride-aware causal mask M_{stride}:

$$\text{attn} = \texttt{softmax}\left(\frac{q\,\bar{k}^\top}{\sqrt{d_h}} + M_{\text{stride}}\right)\bar{v}, \quad \bar{k} = W_{UK}\,\bar{c}_{KV},\ \bar{v} = W_{UV}\,\bar{c}_{KV}.$$

6.7.2 Algorithmic Pseudocode

Input: per-token latents $c_{KV}^{(t)}$, merge window m, stride s
Merge phase (hyper-network):
for each merged slot $j = 0, 1, \ldots$ **do**
    $\alpha_i \leftarrow \texttt{sigmoid}(\text{HyperNet}(c_{KV}^{(sj+i)}))$ for $i = 0, \ldots, m-1$
    $\bar{c}_{KV}^{(j)} \leftarrow \sum_{i=0}^{m-1} \alpha_i\, c_{KV}^{(sj+i)}$ ▷ gated average over window
end for
$\bar{k} \leftarrow W_{UK}\,\bar{c}_{KV}, \quad \bar{v} \leftarrow W_{UV}\,\bar{c}_{KV}$
weights $\leftarrow \texttt{softmax}(q\,\bar{k}^\top/\sqrt{d_h} + M_{\text{stride}})$ ▷ stride-aware causal mask
Output: $\mathbf{Y} = \text{weights} \cdot \bar{v}$

6.7.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d/m)$ on the merged sequence (length $\lceil n/m \rceil$) plus hyper-network cost.
- **Inference complexity**: $\mathcal{O}(n/m)$ per token over merged slots; $\mathcal{O}(r_{kv} \cdot n/m)$ state.
- **Trainable**: hyper-network and merge window m are learned/chosen; stride-aware mask enforces consistency.
- **Strengths**: 5.3× decode speedup, 8.3× GPU-memory reduction vs MHA; temporal compression orthogonal to rank compression.
- **Limitations**: Merge window can blur fine temporal detail; hyper-network adds compute; mask design is non-trivial.

6.8 Architectural Comparison and Synthesis

6.9 Compressed Convolutional Attention (CCA)

Compressed Convolutional Attention (CCA) [25] takes a fundamentally different approach to latent attention: rather than compressing the KV-cache and then up-projecting to full width before attending (as MLA/GQLA/MLRA do), CCA down-projects queries, keys, and values into a shared latent and performs the *entire* attention operation inside it. There are no Q/K/V up-projection matrices—only a single output up-projection W_O maps the latent output back to the residual stream. This simultaneously reduces parameters, KV-cache size, **and** attention FLOPs by the compression factor $C = E/\tilde{e}$, whereas MLA only shrinks the cache.

To make attention in the fully compressed latent viable, CCA introduces three innovations: (1) two causal convolutions on the packed q/k tensor (a depth-wise sequence convolution and a head-wise grouped channel convolution), (2) q-k-mean (adds the pre-convolution mean of q

and k to the post-convolution values), and (3) value-shift (each head receives half its values from the current token and half from the previous token via two independent projections). After these steps, QK L2-normalisation and a learnable key temperature β are applied, RoPE is applied *directly in the latent* (no separate RoPE head needed), and standard softmax attention is computed.

6.9.1 Mathematical Formulation

$$\tilde{q}_{\text{pre}} = \tilde{W}_Q\, x, \qquad \tilde{k}_{\text{pre}} = \tilde{W}_K\, x, \qquad \tilde{e} = E/C.$$

$$\tilde{q} = \text{Conv}(\tilde{q}_{\text{pre}}) + \tfrac{1}{2}(\tilde{q}_{\text{pre}} + \tilde{k}_{\text{pre}}), \qquad \tilde{k} = \text{Conv}(\tilde{k}_{\text{pre}}) + \tfrac{1}{2}(\tilde{q}_{\text{pre}} + \tilde{k}_{\text{pre}}).$$

$$\tilde{v} = [\tilde{W}_{V_1}\, x_t \parallel \tilde{W}_{V_2}\, x_{t-1}], \qquad \hat{q} = \frac{\tilde{q}\sqrt{d_h}}{\|\tilde{q}\|}, \quad \hat{k} = \frac{\tilde{k}\sqrt{d_h}}{\|\tilde{k}\|} e^{\beta}.$$

$$\text{attn} = \texttt{softmax}\left(\frac{\text{RoPE}(\hat{q})\,\text{RoPE}(\hat{k})^\top}{\sqrt{d_h}}\right)\tilde{v}, \qquad \mathbf{Y} = W_O\,\text{attn}.$$

6.9.2 Algorithmic Pseudocode

Input: hidden state $x \in \mathbb{R}^{n\times E}$, compression factor C, projections $\tilde{W}_Q, \tilde{W}_K, \tilde{W}_{V_1}, \tilde{W}_{V_2}, W_O$
$\tilde{e} \leftarrow E/C$ ▷ latent width
$\tilde{q}_{\text{pre}} \leftarrow \tilde{W}_Q\, x, \quad \tilde{k}_{\text{pre}} \leftarrow \tilde{W}_K\, x$ ▷ down-project to latent
$\tilde{q}_{\text{conv}} \leftarrow \text{ConvSeq}(\tilde{q}_{\text{pre}}), \quad \tilde{k}_{\text{conv}} \leftarrow \text{ConvSeq}(\tilde{k}_{\text{pre}})$ ▷ causal depth-wise + grouped convs
$\text{qk\_mean} \leftarrow (\tilde{q}_{\text{pre}} + \tilde{k}_{\text{pre}})/2$ ▷ q-k-mean bias
$\tilde{q} \leftarrow \tilde{q}_{\text{conv}} + \text{qk\_mean}, \quad \tilde{k} \leftarrow \tilde{k}_{\text{conv}} + \text{qk\_mean}$
$\tilde{v}_1 \leftarrow \tilde{W}_{V_1}\, x_t, \quad \tilde{v}_2 \leftarrow \tilde{W}_{V_2}\, x_{t-1}$ ▷ value-shift
$\tilde{v} \leftarrow [\tilde{v}_1 \parallel \tilde{v}_2]$ ▷ concat half-heads
$\hat{q} \leftarrow \tilde{q}\,\sqrt{d_h}/\|\tilde{q}\|, \quad \hat{k} \leftarrow \tilde{k}\,\sqrt{d_h}/\|\tilde{k}\| \cdot e^{\beta}$ ▷ QK-norm + temperature
$\hat{q} \leftarrow \text{RoPE}(\hat{q}), \quad \hat{k} \leftarrow \text{RoPE}(\hat{k})$ ▷ RoPE in latent
$\text{weights} \leftarrow \texttt{softmax}(\hat{q}\,\hat{k}^\top/\sqrt{d_h})$
Output: $\mathbf{Y} = W_O\,(\text{weights} \cdot \tilde{v})$

6.9.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d/C)$ (attention in compressed latent, $C\times$ fewer FLOPs than MHA).
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(\tilde{e} \cdot n)$ KV-cache space.
- **Trainable**: C, conv kernel sizes, and all three tricks (convs, qk-mean, value-shift) are configurable.
- **Strengths**: Reduces params, cache, AND FLOPs simultaneously; RoPE integrates natively (no RoPE head); >2× fewer params than MLA at same compression.
- **Limitations**: Does not remove quadratic S^2 (only divides by C); conv/qk-mean/v-shift add inductive bias; fused kernel needed for theoretical speedup.

6.10 Compressed Convolutional Grouped Query Attention (CCGQA)

Compressed Convolutional Grouped Query Attention (CCGQA) [25] extends CCA with GQA-style key/value head sharing applied *inside* the compressed latent, and **decouples** the query and KV compression rates: queries are compressed by C_1 and keys/values by $C_2 \geq C_1$. The per-head latent dimension d_h must match between query and key heads, enforcing the constraint

$C_2/C_1 = n_h/n_{kv}$. KV heads are shared per group (replicated to match query heads), and the qk-mean bias uses broadcast/reduce operators B_{group} (replicate KV heads to query heads) and E_{group} (average query heads within each group). CCGQA achieves the best reported loss at 8× KV-cache compression with no quality drop versus MHA.

6.10.1 Mathematical Formulation

Same pipeline as CCA, with decoupled latents:

$$\tilde{q}_{\text{pre}} = \tilde{W}_Q\, x \in \mathbb{R}^{E/C_1}, \qquad \tilde{k}_{\text{pre}} = \tilde{W}_K\, x \in \mathbb{R}^{E/C_2}, \qquad C_2 \geq C_1.$$

$$\text{qk\_mean}_q = \tfrac{1}{2}(\tilde{q}_{\text{pre}} + B_{\text{group}}(\tilde{k}_{\text{pre}})), \qquad \text{qk\_mean}_k = E_{\text{group}}(\text{qk\_mean}_q).$$

$$\text{attn} = \texttt{softmax}\left(\frac{\text{RoPE}(\hat{q})\,\text{RoPE}(\hat{k})^\top}{\sqrt{d_h}}\right)\tilde{v}, \qquad \mathbf{Y} = W_O\,\text{attn}.$$

6.10.2 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d/C_1)$ (S^2 term scales by $1/C_1$; projection terms by $1/C_1 + 1/C_2$).
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(\tilde{e}_{kv} \cdot n)$ KV-cache space (C_2-compressed, group-shared).
- **Trainable**: C_1, C_2, n_{kv}, and all CCA tricks are configurable. Constraint: $C_2/C_1 = n_h/n_{kv}$.
- **Strengths**: Decoupled compression enables smooth Pareto frontier; same arithmetic intensity as GQA; best loss at 8× cache reduction.
- **Limitations**: Same quadratic S^2 (only reduced by C_1); fused kernel needed; two value projections add complexity.

The latent family shows a clear progression along two axes: (i) **rank compression**, moving from full heads (MHA) to a shared latent (MLA) to partitioned sub-head latents (MLRA), and (ii) **deployment adaptivity**, moving from a single decode path to hardware-adaptive dispatch (GQLA) and temporal cache merging (MTLA). Tucker Attention provides the unifying algebraic frame: MHA, GQA, and MLA are all special cases of a Tucker factorisation with appropriate rank choices. CCA and CCGQA take a fundamentally different approach: rather than compressing the KV-cache and then up-projecting for full-width attention (MLA/GQLA/MLRA), they perform attention *entirely inside* the compressed latent space, reducing not just the cache but also the attention FLOPs by the compression factor. CCGQA further decouples query and KV compression rates, achieving the best reported loss at 8× KV-cache compression.

7 Comprehensive Sparse Attention Mechanisms

7.1 Executive Summary

Sparse attention mechanisms address the prohibitive $\mathcal{O}(n^2)$ computational and memory complexity of standard scaled dot-product attention by restricting the set of attended key-value pairs. Modern sparse attention spans a rich design space distinguished by four orthogonal dimensions: (i) **sparsity pattern** (fixed geometric, data-dependent, or frequency-based), (ii) **trainability** (end-to-end trained or inference-only), (iii) **sparsity unit** (individual tokens, local windows, or blocks), and (iv) **mechanism** (masking, selection, filtering, or compression). This annex synthesizes seven major sparse attention families—Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, and FASA—providing mathematical formulations, algorithmic pseudocode, and comprehensive architectural comparisons.

7.2 Sparse Transformer: Factorized Strided and Fixed Patterns

7.2.1 Mathematical Formulation

The Sparse Transformer [15] reduces quadratic complexity to $\mathcal{O}(n\sqrt{n})$ by factorizing sparse attention into two complementary sparse heads. Let n be the sequence length and $l = \lfloor \sqrt{n} \rfloor$ be the stride.

Strided attention head : Each position i attends to every l-th previous position:

$$\mathcal{A}_i^{(\text{strided})} = \{j : j \leq i,\ (i-j) \bmod l = 0\}$$

Fixed attention head : Each position i attends to positions within its local block plus fixed summary columns:

$$\mathcal{A}_i^{(\text{fixed})} = \{j : \lfloor j/l \rfloor = \lfloor i/l \rfloor\} \cup \{j : j \bmod l \in \{l-c, \ldots, l-1\}\}$$

where c is a hyperparameter controlling the number of summary columns (typically $c = 1$ or $c = 2$).

The attention computation for each head h follows standard scaled dot-product rules restricted to the sparse set:

$$\text{Attn}_h(Q, K, V)_i = \text{softmax}\left(\frac{q_i^h {K^h_{\mathcal{A}_i}}^\top}{\sqrt{d_k}}\right) V^h_{\mathcal{A}_i}$$

The key insight is that with the two factorized heads operating in parallel across a depth of L transformer layers, every position can reach every other position through a path of length at most $L + 1$, preserving long-range reachability with a fraction of dense attention cost.

7.2.2 Algorithmic Pseudocode

Algorithm 23 Sparse Transformer Attention: Strided + Fixed Dual Heads

Require: Query $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Key $\mathbf{K} \in \mathbb{R}^{n \times d}$, Value $\mathbf{V} \in \mathbb{R}^{n \times d}$
Require: Stride $l = \lfloor \sqrt{n} \rfloor$, summary columns $c \in \{1, 2\}$
Ensure: Output $\mathbf{Y} \in \mathbb{R}^{n \times d}$

Head 1: Strided Attention
for $i = 1, \ldots, n$ **do**
  $\mathcal{A}_i \leftarrow \{j : j \leq i \text{ and } (i-j) \bmod l = 0\}$ ▷ Every l-th position
  $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$
  $\mathbf{y}_i^{(1)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
end for
Head 2: Fixed Block + Summary Attention
for $i = 1, \ldots, n$ **do**
  block_start $\leftarrow \lfloor i/l \rfloor \cdot l$, block_end $\leftarrow \min(i+1, \text{block\_start} + l)$
  $\mathcal{A}_i \leftarrow [\text{block\_start}, \text{block\_end}) \cup \{\text{cols from last } c \text{ columns}\}$
  $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$
  $\mathbf{y}_i^{(2)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
end for
Concatenate: $\mathbf{Y} = [\mathbf{y}^{(1)} \| \mathbf{y}^{(2)}]$ and project via output linear layer **return** $\mathbf{Y}$

7.2.3 Key Characteristics

- **Complexity**: $\mathcal{O}(n\sqrt{n} \cdot d)$ during training; $\mathcal{O}(n)$ per token during generation.
- **Trainable**: Yes; full backpropagation through selected positions.
- **Sparsity pattern**: Fixed and data-agnostic; patterns do not adapt to content.
- **Strength**: Structured reachability guarantees long-range dependencies; proven effective on long sequences (Enwik8, text images).
- **Limitation**: Fixed patterns may miss important but non-contiguous relationships; requires custom CUDA kernels for practical efficiency.

7.3 Longformer: Sliding Window, Dilation, and Global Tokens

7.3.1 Mathematical Formulation

Longformer [8] achieves linear $\mathcal{O}(n \cdot w)$ complexity by combining three complementary attention patterns.

Sliding window attention : Local neighborhood of size w:

$$\mathcal{A}_i^{(\text{window})} = \{j : |i - j| \leq w/2\}$$

Dilated sliding window : Windowed attention with dilation d, skipping stride $d + 1$:

$$\mathcal{A}_i^{(\text{dilated})} = \{j : |i - j| \leq (w/2)(d + 1) \text{ and } (i - j) \bmod (d + 1) = 0\}$$

Global attention : Designated global tokens (e.g., [CLS]) attend to all positions and are attended by all:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \ldots, n\} & \text{if } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{window})} \cup \mathcal{G} & \text{otherwise} \end{cases}$$

The three patterns are applied via separate projections. The local and global attention can use either the same projections or task-specific separate ones.

7.3.2 Algorithmic Pseudocode

Algorithm 24 Longformer: Sliding Window + Dilated + Global

Require: Query $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Key $\mathbf{K}$, Value $\mathbf{V}$, window size w, dilation d, global token indices $\mathcal{G}$
Ensure: Output $\mathbf{Y}$

for $i = 1, \ldots, n$ **do**
  if $i \in \mathcal{G}$ **then**
    $\mathcal{A}_i \leftarrow \{1, \ldots, n\}$ ▷ Global token attends to all
  else
    Local window: $\mathcal{A}^{(\text{local})} \leftarrow [i - w/2, i + w/2] \cap [1, n]$
    Dilated offsets: $\mathcal{A}^{(\text{dilated})} \leftarrow \{j : (i - j) \bmod (d + 1) = 0\} \cap \mathcal{A}^{(\text{dilated range})}$
    Combine: $\mathcal{A}_i \leftarrow \mathcal{A}^{(\text{local})} \cup \mathcal{A}^{(\text{dilated})} \cup \mathcal{G}$
  end if
  $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$
  $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
end forreturn $\mathbf{Y}$

7.3.3 Key Characteristics

- **Complexity**: $\mathcal{O}(n \cdot w)$ where w is the window size (linear when $w \ll n$).
- **Trainable**: Yes; drop-in replacement for standard attention.
- **Coverage**: With L layers and window size w, receptive field grows to $L \times w$, covering entire sequence with shallow networks.
- **Strength**: Practical for document-level tasks; dilation expands local receptive fields with minimal overhead; global tokens provide query-document correspondence.
- **Limitation**: Window size remains a design hyperparameter; suboptimal for tasks requiring dense attention patterns.

7.4 BigBird: Random, Local, and Global Sparse Graph

7.4.1 Mathematical Formulation

BigBird [91] preserves universal approximation and Turing completeness using a principled mix of three sparsity patterns.

Random connections : Each position connects to r randomly sampled positions:

$$\mathcal{A}_i^{(\text{random})} = \text{RandomSample}(\{1, \ldots, n\}, r)$$

Local window : Sliding window neighborhood:

$$\mathcal{A}_i^{(\text{local})} = \{j : |i - j| \leq w/2\}$$

Global tokens : Task-specific global anchors:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \ldots, n\} & \text{if } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{random})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{G} & \text{otherwise} \end{cases}$$

The composite sparse mask M is:

$$M_{ij} = \mathbf{1}[j \in \mathcal{A}_i^{(\text{random})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{A}_i^{(\text{global})}]$$

In practice, BigBird groups tokens into blocks of size b and applies the sparsity decision at the block level, enabling efficient block-sparse implementations.

7.4.2 Theoretical Guarantee

The authors prove that with $O(1)$ random connections and global tokens, the sparse graph maintains the same universal approximation and Turing completeness properties as dense attention. Formally, any computable function can be represented and any sequence of operations can be simulated within the BigBird connectivity graph.

7.4.3 Algorithmic Pseudocode

Algorithm 25 BigBird: Random + Local + Global Block-Sparse Attention

Require: Query $\mathbf{Q}$, Key $\mathbf{K}$, Value $\mathbf{V}$, block size b, random links per block r, global indices $\mathcal{G}$
Ensure: Output $\mathbf{Y}$

Reshape into blocks: $n_b = \lceil n/b \rceil$ blocks
for block $i = 0, \ldots, n_b - 1$ **do**
  for position j in block i **do**
    Local window: $\mathcal{A}_j^{(\text{local})}$ = nearby positions in block $\cup$ boundary positions
    Random block sampling: Sample r blocks uniformly at random, add all positions from those blocks
    Global: Add all global token positions
    $\mathcal{A}_j \leftarrow \mathcal{A}_j^{(\text{local})} \cup \mathcal{A}_j^{(\text{random})} \cup \mathcal{G}$
  end for
end for
for $i = 1, \ldots, n$ **do**
  $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$
  $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
end forreturn $\mathbf{Y}$

7.4.4 Key Characteristics

- **Complexity**: $\mathcal{O}(n)$ or near-linear in optimal block-sparse implementation.
- **Trainable**: Yes.
- **Theoretical grounding**: Provably maintains universal approximation and Turing-completeness with sparse connectivity.
- **Strength**: Strong long-document performance on question answering, summarization, and genomics tasks.
- **Limitation**: Random sampling introduces variance and non-determinism; tuning of r, w, g hyperparameters remains task-dependent.

7.5 SparseK Attention: Differentiable Top-k Selection

7.5.1 Mathematical Formulation

SparseK [52] enables learnable sparsity via a **differentiable top-k operator**. A scoring network ϕ_θ evaluates key-value pair importance:

$$u_j = \phi_\theta(\mathbf{k}_j, \mathbf{q}_i) \in \mathbb{R}$$

The **SparseK operator** selects the top-k scores while remaining differentiable. It computes a threshold $\tau(u)$ such that the sum of active scores equals k:

$$\text{SparseK}(u, k)_j = \max(u_j - \tau(u), 0), \quad \text{where} \quad \sum_j \max(u_j - \tau, 0) = k$$

The threshold τ is found via bisection. The attention becomes:

$$m_j = \text{SparseK}(u, k)_j, \quad \text{Attn}_i = \text{softmax}\left(\frac{\mathbf{q}_i K_{\text{sel}}^\top}{\sqrt{d_k}}\right) V_{\text{sel}}$$

where $K_{\text{sel}}, V_{\text{sel}}$ contain only the top-k entries (those with $m_j > 0$).

7.5.2 Algorithmic Pseudocode

Algorithm 26 SparseK: Differentiable Top-k Attention

Require: Query $\mathbf{q} \in \mathbb{R}^d$, Key matrix $\mathbf{K} \in \mathbb{R}^{n \times d}$, Value matrix $\mathbf{V} \in \mathbb{R}^{n \times d}$
Require: Scoring network ϕ_θ, target sparsity k
Ensure: Attention output $\mathbf{y}$

Compute importance scores:
for $j = 1, \ldots, n$ **do**
    $u_j \leftarrow \phi_\theta(\mathbf{k}_j, \mathbf{q})$
end for
Compute differentiable top-k via threshold:
$u_{\text{sorted}} \leftarrow \text{sort}(u, \text{descending} = \text{True})$
$\text{cumsum} \leftarrow \text{cumsum}(u_{\text{sorted}})$
Find $\rho = \max\{i : u_{\text{sorted}}[i] > 0 \text{ and } \text{cumsum}[i] \leq k\}$
$\tau \leftarrow (u_{\text{sorted}}[\rho] - k)/(\rho + 1)$ ▷ Threshold for k active elements
$m \leftarrow \max(u - \tau, 0)$ ▷ Differentiable selection mask
Apply mask and compute attention:
$K_{\text{sel}} \leftarrow K[m > 0]$, $V_{\text{sel}} \leftarrow V[m > 0]$
$\text{scores} \leftarrow \mathbf{q} K_{\text{sel}}^\top / \sqrt{d_k}$
$\text{attn} \leftarrow \text{softmax}(\text{scores})$
$\mathbf{y} \leftarrow \text{attn} \cdot V_{\text{sel}}$ **return y**

7.5.3 Key Characteristics

- **Complexity**: $\mathcal{O}(n \cdot d)$ during training (linear in sequence length); $\mathcal{O}(k)$ per token during generation.
- **Trainable**: Yes; end-to-end gradient flow through the SparseK operator.
- **Incremental generation**: Supports efficient constant-memory autoregressive generation.
- **Strength**: Seamlessly integrates into existing LLM architectures; minimal fine-tuning needed.
- **Limitation**: Scattered memory access from top-k selection may limit cache efficiency on some hardware; scoring network adds overhead.

7.6 NSA (Native Sparse Attention): Hardware-Aligned Hierarchical Branches

7.6.1 Mathematical Formulation

NSA [89] decomposes sparse attention into three parallel branches that are combined through learned gates.

Compression branch : A learnable MLP φ compresses key-value blocks:

$$\tilde{K}_t^{\text{cmp}} = [\varphi(k_{id+1:id+l})]_{1 \leq i \leq \lfloor (t-l)/d \rfloor} \quad , \quad \tilde{V}_t^{\text{cmp}} = [\varphi(v_{id+1:id+l})]$$

Selection branch : High-importance blocks are selected based on compressed scores:

$$p_t^{\text{cmp}} = \text{softmax}(q_t^\top \tilde{K}_t^{\text{cmp}} / \sqrt{d}), \quad I_t = \text{TopK}(p_t^{\text{cmp}}, n), \quad \tilde{K}_t^{\text{sel}} = \text{Gather}(K, I_t)$$

Sliding window branch : Fixed local context:

$$\tilde{K}_t^{\text{win}} = K_{t-w:t}, \quad \tilde{V}_t^{\text{win}} = V_{t-w:t}$$

Gated combination : The three attention outputs are combined via learned gates:

$$\alpha_t^{(\text{cmp})} = \sigma(\text{Linear}_{\text{cmp}}(q_t)), \quad \alpha_t^{(\text{sel})} = \sigma(\text{Linear}_{\text{sel}}(q_t)), \quad \alpha_t^{(\text{win})} = \sigma(\text{Linear}_{\text{win}}(q_t))$$

$$y_t = \alpha_t^{(\text{cmp})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{cmp}}, \tilde{V}^{\text{cmp}}) + \alpha_t^{(\text{sel})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{sel}}, \tilde{V}^{\text{sel}}) + \alpha_t^{(\text{win})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{win}}, \tilde{V}^{\text{win}})$$

7.6.2 Algorithmic Pseudocode

Algorithm 27 NSA: Compressed + Selected + Window Branches with Learned Gating

Require: Query $\mathbf{q}_t$, cached Key/Value sequences, block size l, stride d, selection count n, window w
Require: Compression MLP φ, gate networks $\text{Linear}_{\text{cmp}}, \text{Linear}_{\text{sel}}, \text{Linear}_{\text{win}}$
Ensure: Output $\mathbf{y}_t$

Compression branch:
for $i = 1$ to $\lfloor t/d \rfloor$ **do**
  $k_i^{\text{cmp}} \leftarrow \varphi(\mathbf{K}_{id+1:id+l})$ ▷ Compress block via MLP
  $v_i^{\text{cmp}} \leftarrow \varphi(\mathbf{V}_{id+1:id+l})$
end for
$\tilde{\mathbf{K}}^{\text{cmp}} \leftarrow [k_1^{\text{cmp}}, \ldots, k_{\lfloor t/d \rfloor}^{\text{cmp}}]$, $\tilde{\mathbf{V}}^{\text{cmp}} \leftarrow [v_1^{\text{cmp}}, \ldots, v_{\lfloor t/d \rfloor}^{\text{cmp}}]$
$\mathbf{y}_t^{(\text{cmp})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{cmp}}, \tilde{\mathbf{V}}^{\text{cmp}})$
Selection branch:
Compute scores on compressed: $\mathbf{s} \leftarrow \text{softmax}(\mathbf{q}_t \tilde{\mathbf{K}}^{\text{cmp}\top} / \sqrt{d})$
Select top-n important blocks: $I \leftarrow \text{TopK}(\mathbf{s}, n)$
Gather full blocks: $\tilde{\mathbf{K}}^{\text{sel}} \leftarrow [\mathbf{K}_{I_i d+1:I_i d+l}]_i$, $\tilde{\mathbf{V}}^{\text{sel}} \leftarrow [\mathbf{V}_{I_i d+1:I_i d+l}]_i$
$\mathbf{y}_t^{(\text{sel})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{sel}}, \tilde{\mathbf{V}}^{\text{sel}})$
Window branch:
$\mathbf{y}_t^{(\text{win})} \leftarrow \text{SDPA}(\mathbf{q}_t, \mathbf{K}_{t-w:t}, \mathbf{V}_{t-w:t})$
Gated fusion:
$\alpha^{(\text{cmp})} \leftarrow \sigma(\text{Linear}_{\text{cmp}}(\mathbf{q}_t))$, $\alpha^{(\text{sel})} \leftarrow \sigma(\text{Linear}_{\text{sel}}(\mathbf{q}_t))$, $\alpha^{(\text{win})} \leftarrow \sigma(\text{Linear}_{\text{win}}(\mathbf{q}_t))$
$\mathbf{y}_t \leftarrow \alpha^{(\text{cmp})} \mathbf{y}_t^{(\text{cmp})} + \alpha^{(\text{sel})} \mathbf{y}_t^{(\text{sel})} + \alpha^{(\text{win})} \mathbf{y}_t^{(\text{win})}$ **return** $\mathbf{y}_t$

7.6.3 Key Characteristics

- **Complexity**: $\mathcal{O}(t/d + n \cdot l + w)$ tokens attended per step (significantly reduced versus $\mathcal{O}(t)$ for dense).
- **Trainable**: Yes; direct training with all branches differentiable.
- **Hardware alignment**: Designed for efficient tensor core utilization; compression and selection reduce memory bandwidth pressure.
- **Strength**: 11.6× decoding speedup and 9× forward speedup on 64k sequences; outperforms full attention on many tasks.
- **Limitation**: Requires custom Triton/CUDA kernels; complex multi-branch architecture increases engineering overhead.

7.7 FASA: Frequency-Aware Sparse Attention

7.7.1 Mathematical Formulation

FASA [82] is a **training-free inference-time method** that exploits rotary position embedding (RoPE) structure. Under RoPE, the token embedding is rotated by $\theta_i = B^{-2(i-1)/d}$, decomposing the d-dimensional space into $d/2$ frequency chunks (FCs).

Key insight : Only a small subset ($< 1\%$) of FCs matter for contextual awareness; most encode positional patterns.

Dominant frequency chunk identification : For each layer l and head h, identify dominant FCs via contextual agreement (CA):

$$\mathrm{CA}^{l,h,i} = \frac{|\mathrm{TopK}(\alpha^{l,h}) \cap \mathrm{TopK}(\alpha^{l,h,i})|}{K}$$

where $\alpha^{l,h}$ is the full attention mask and $\alpha^{l,h,i}$ is the mask using only FC i. Dominant FCs have high CA—they contribute meaningfully to the final attention pattern.

Token importance prediction (TIP) stage : Using only dominant FCs, compute lightweight importance per token:

$$S_t^{l,h} = \sum_{i \in \mathcal{I}_{\mathrm{dom}}^{l,h}} \alpha^{l,h,i}(\mathbf{q}_t, \mathbf{K}_{1:t}), \quad \mathcal{T}_t = \text{TopK-Indices}(S_t, N_{\mathrm{fac}})$$

Focused attention computation (FAC) stage : Compute full-precision attention on selected tokens:

$$\hat{\alpha}_{\mathrm{FAC}} = \mathrm{softmax}\left(\frac{\mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^\top}{\sqrt{d}}\right), \quad \mathbf{o}_t = \hat{\alpha}_{\mathrm{FAC}} \mathbf{V}_{\mathcal{T}_t}$$

7.7.2 Algorithmic Pseudocode

Algorithm 28 FASA: Frequency-Aware Sparse Attention (Inference-Time)

Require: Current query $\mathbf{q}_t$, cached Key/Value $\mathbf{K}_{1:t}, \mathbf{V}_{1:t}$, RoPE base B, dominant FCs $\mathcal{I}_{\text{dom}}$
Require: TIP token budget N_{tip}, FAC token budget N_{fac}
Ensure: Output $\mathbf{o}_t$

Stage 1: Token Importance Prediction (TIP)
for layer l and head h **do**
  for position $i = 1$ to t **do**
    Compute importance from dominant FCs: $s_i \leftarrow 0$
    for $fc \in \mathcal{I}_{\text{dom}}^{l,h}$ **do**
      Rotate query and key into FC fc: $\mathbf{q}^{(fc)}, \mathbf{k}_i^{(fc)}$
      $s_i^{(fc)} \leftarrow \text{softmax}(\mathbf{q}^{(fc)} \mathbf{k}_i^{(fc)\top} / \sqrt{d})$
      $s_i \leftarrow s_i + s_i^{(fc)}$
    end for
  end for
  $\mathcal{T}_t \leftarrow \text{TopK} - \text{Indices}([s_1, \ldots, s_t], N_{\text{fac}})$ ▷ Select top tokens
end for
Stage 2: Focused Attention Computation (FAC)
$\text{scores}_{\text{fac}} \leftarrow \mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^\top / \sqrt{d}$ ▷ Full-precision dot product
$\alpha_{\text{fac}} \leftarrow \text{softmax}(\text{scores}_{\text{fac}})$ ▷ Full softmax
$\mathbf{o}_t \leftarrow \alpha_{\text{fac}} \mathbf{V}_{\mathcal{T}_t}$ ▷ Weighted value sum **return** $\mathbf{o}_t$

7.7.3 Key Characteristics

- **Complexity**: TIP is $\mathcal{O}(t \cdot N_{\text{tip}})$; FAC is $\mathcal{O}(N_{\text{fac}} \cdot d)$.
- **Trainable**: No; training-free inference optimization.
- **Applicability**: Requires RoPE-based models; extended to ALiBi and MLA with modifications.
- **Strength**: Near-oracle accuracy with ≤ 256 tokens out of millions; up to 2.56× decoding speedup; orthogonal to quantization.
- **Limitation**: Offline offline calibration required; dominant FC identification adds preprocessing overhead.

7.8 SpargeAttn: Two-Stage Block-Level Filtering

7.8.1 Mathematical Formulation

SpargeAttn [94] is a **universal training-free method** that predicts which blocks of the attention matrix will contain negligible values and skips computation for those blocks.

Stage 1 — Sparse prediction : Compute low-cost proxy importance $\hat{s}_{ij}$ for block (i, j):

$$\hat{s}_{ij} = f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j; \text{hyperparams}), \quad \text{skip if } \hat{s}_{ij} < \epsilon_1$$

Common predictors include block-mean similarity or self-similarity statistics.

Stage 2 — Softmax-aware filtering : After computing $\tilde{P}_{ij} = \text{softmax}(Q_i K_j^\top / \sqrt{d})$, check if the block's maximum probability is negligible relative to the running softmax maximum:

$$\text{skip } P_{ij}V_j \quad \text{if} \quad \max(\tilde{P}_{ij}) < e^{m_{\text{old}} - m_{\text{new}}} \cdot \epsilon_2$$

where $m_{\text{old}}, m_{\text{new}}$ are online softmax running maxima (computed via FlashAttention-style numerics).

7.8.2 Algorithmic Pseudocode

Algorithm 29 SpargeAttn: Two-Stage Block-Sparse Filtering

Require: Query blocks $\mathbf{Q}_i$ for $i = 1, \ldots, n_b$, Key blocks $\mathbf{K}_j$ for $j = 1, \ldots, n_b$, Value blocks $\mathbf{V}_j$
Require: Prediction threshold ϵ_1, softmax threshold ϵ_2, block size b_s
Ensure: Output $\mathbf{Y}$

Initialize: online softmax max $m_{\text{global}} \leftarrow -\infty$
for block row $i = 1$ to n_b **do**
  Initialize: block row output $\mathbf{Y}_i \leftarrow 0$, local max $m_i \leftarrow -\infty$
  for block column $j = 1$ to n_b **do**
    Stage 1: Sparse Prediction
    Compute proxy importance: $\hat{s}_{ij} \leftarrow f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j)$
    if $\hat{s}_{ij} < \epsilon_1$ **then**
      skip this block $[i, j]$ ▷ Early termination
      continue ▷ Move to next block
    end if
    Stage 2: Compute and Filter
    $\tilde{P}_{ij} \leftarrow \text{softmax}(\mathbf{Q}_i \mathbf{K}_j^\top / \sqrt{d})$
    $m_{\text{block}} \leftarrow \max(\tilde{P}_{ij})$
    if $m_{\text{block}} < e^{m_i - m_{\text{global}}} \cdot \epsilon_2$ **then**
      Block contributes negligibly; skip ▷ Softmax-aware pruning
      continue
    end if
    Update global max: $m_i \leftarrow \max(m_i, m_{\text{block}})$
    Accumulate: $\mathbf{Y}_i \leftarrow \mathbf{Y}_i + \tilde{P}_{ij}\mathbf{V}_j$
  end for
  $m_{\text{global}} \leftarrow \max(m_{\text{global}}, m_i)$
end forreturn Y

7.8.3 Key Characteristics

- **Complexity**: Empirical $\mathcal{O}(n^2 \cdot s)$ where s is the fraction of non-skipped blocks (typically 0.2–0.5).
- **Trainable**: No; plug-and-play acceleration for existing models.
- **Universality**: Works on language models, image diffusion models, and video generation.
- **Strength**: 2.5–5× speedup compared to dense or previous sparse methods; compatible with quantization (SageAttention integration).
- **Limitation**: Speedup depends on inherent sparsity; block-level granularity may miss finer patterns; threshold tuning required per model family.

7.9 MSA (MiniMax Sparse Attention): Blockwise Sparse Attention with Lightweight Index Branch

7.9.1 Mathematical Formulation

MSA [43] is a blockwise sparse attention mechanism built on grouped-query attention (GQA) that combines a lightweight **Index Branch** for block selection with a **Main Branch** for exact block-sparse softmax attention over the selected blocks. Let N be the sequence length, $B_k = 128$ the block size, and $k = 16$ the number of blocks selected per query (yielding a fixed per-query budget of $k \cdot B_k = 2048$ tokens regardless of N).

Index Branch : A low-dimensional projection produces index queries $\mathbf{q}_t^{\text{idx}} \in \mathbb{R}^{d_{\text{idx}}}$ and block representatives $\mathbf{r}_b \in \mathbb{R}^{d_{\text{idx}}}$ for each KV block b. The index scores per GQA group are:

$$s_{t,b}^{(g)} = \mathbf{q}_t^{\text{idx},(g)} \cdot \mathbf{r}_b, \qquad \mathcal{I}_t^{(g)} = \text{TopK}\left(\{s_{t,b}^{(g)}\}_{b=1}^{N/B_k},\ k\right)$$

The index input is **detached** (stop-gradient), so gradients do not flow back through the indexer's input; the Index Branch is trained instead by a KL alignment loss between its attention distribution and the Main Branch's distribution. Because softmax is order-preserving, the raw index scores feed directly into the top-k operator *without* an explicit softmax (**exp-free top-k**), reducing numerical cost.

Forced local block : The block containing the query position t is **always** included in $\mathcal{I}_t^{(g)}$, guaranteeing local context coverage.

Main Branch : Standard scaled dot-product attention is computed exactly over the union of selected blocks:

$$\mathbf{o}_t^{(h)} = \text{softmax}\left(\frac{\mathbf{q}_t^{(h)} \mathbf{K}_{\mathcal{I}_t}^{(h)\top}}{\sqrt{d_h}}\right) \mathbf{V}_{\mathcal{I}_t}^{(h)}$$

where $\mathbf{K}_{\mathcal{I}_t}, \mathbf{V}_{\mathcal{I}_t}$ gather the full-precision key/value tensors of the selected blocks. Selection is performed independently per GQA group, so different query groups within the same layer may attend to different block subsets.

7.9.2 Algorithmic Pseudocode

Algorithm 30 MSA: MiniMax Sparse Attention with Index + Main Branches

Require: Query $\mathbf{Q} \in \mathbb{R}^{N \times d}$, Key $\mathbf{K}$, Value $\mathbf{V}$, block size $B_k = 128$, top-k blocks $k = 16$
Require: Index projection W_{idx} (detached input), block representatives $\{\mathbf{r}_b\}$
Ensure: Output $\mathbf{Y} \in \mathbb{R}^{N \times d}$

Compute block reps: $\mathbf{r}_b \leftarrow \text{mean}(\mathbf{K}_{bB_k:(b+1)B_k})$ for $b = 1, \ldots, N/B_k$
for query position $t = 1, \ldots, N$ **do**
  for each GQA group g **do**
    $\mathbf{q}_t^{\text{idx}} \leftarrow \text{sg}(W_{\text{idx}} \mathbf{x}_t)$ ▷ detached index query
    Compute block scores: $s_b \leftarrow \mathbf{q}_t^{\text{idx}} \cdot \mathbf{r}_b$
    Identify local block: $b_{\text{local}} \leftarrow \lfloor t / B_k \rfloor$
    $\mathcal{I}_t^{(g)} \leftarrow \text{TopK}(\{s_b\}, k) \cup \{b_{\text{local}}\}$ ▷ exp-free top-k + forced local
  end for
  Gather selected KV: $\tilde{\mathbf{K}}_t \leftarrow \text{Gather}(\mathbf{K}, \mathcal{I}_t)$, $\tilde{\mathbf{V}}_t \leftarrow \text{Gather}(\mathbf{V}, \mathcal{I}_t)$
  Main branch SDPA: $\mathbf{y}_t \leftarrow \text{softmax}(\mathbf{q}_t \tilde{\mathbf{K}}_t^\top / \sqrt{d_h}) \tilde{\mathbf{V}}_t$
end for
Add KL alignment loss $\mathcal{L}_{\text{KL}} = \text{KL}(\text{index dist} \,\|\, \text{main dist})$ during training **return** $\mathbf{Y}$

7.9.3 Key Characteristics

- **Complexity**: Index branch $\mathcal{O}(H_{\text{kv}} \cdot d_{\text{idx}} \cdot N^2)$; Main branch $\mathcal{O}(H_q \cdot d_h \cdot N \cdot k \cdot B_k)$. Per-query budget fixed at $k \cdot B_k = 2048$ tokens regardless of N.
- **Trainable**: Yes; end-to-end (the Index Branch is trained via a KL alignment loss, not eval-only).
- **State**: $\mathcal{O}(N)$ KV cache (GQA-sized); no expansion of the recurrent/state dimension.
- **Strength**: Deployed on the 109B MiniMax-M3, MSA achieves a 28.4× per-token attention compute reduction at 1M context, with 14.2× prefill and 7.6× decode speedup on H800 GPUs.
- **Limitation**: Selection granularity is fixed at the block level ($B_k = 128$); the forced local block and per-group independent selection increase engineering complexity and require custom block-sparse kernels.

7.10 SparDA (Sparse Decoupled Attention): Forecast-Guided Lookahead Block Selection

7.10.1 Mathematical Formulation

SparDA [26] introduces a **decoupled** sparse attention scheme that augments the standard per-layer projections Q, K, V with a fourth projection, the **Forecast**, which predicts the KV blocks that will be needed by the *next* layer. This lookahead enables overlapping CPU-to-GPU prefetch of the next layer's selected blocks with the current layer's execution, hiding the offload latency that plagues sparse-attention deployments on memory-constrained hardware.

Forecast projection : For input $\mathbf{x} \in \mathbb{R}^{N \times d}$,

$$\mathbf{F} = \mathbf{x}\mathbf{W}_F \in \mathbb{R}^{N \times H_{\text{kv}} \times d_f}$$

with **one Forecast head per GQA group**, which reduces selection overhead relative to multi-head selectors used by prior work.

Block scoring and selection : For each query i and KV block b with representative $\mathbf{r}_b$:

$$S_{i,b} = \mathbf{F}_i \cdot \mathbf{r}_b, \qquad \mathcal{I}_i = \text{TopK}(\{S_{i,b}\}_b, k)$$

Block-sparse attention : Standard scaled dot-product attention is then computed over the selected blocks:

$$\mathbf{o}_i = \text{softmax}\left(\frac{\mathbf{q}_i \mathbf{K}_{\mathcal{I}_i}^\top}{\sqrt{d_h}}\right)\mathbf{V}_{\mathcal{I}_i}$$

Training : SparDA adds fewer than 0.5% additional parameters. Only the Forecast projections are trained, by matching the original (frozen) sparse selector's attention distribution—a lightweight distillation that preserves the base model's behaviour while enabling lookahead prefetch.

7.10.2 Algorithmic Pseudocode

Algorithm 31 SparDA: Sparse Decoupled Attention with Forecast Prefetch

Require: Input $\mathbf{x} \in \mathbb{R}^{N \times d}$, cached Key/Value blocks, top-k blocks k
Require: Projections W_q, W_k, W_v, W_F; frozen base selector π_{orig}
Ensure: Output $\mathbf{Y}$; prefetched blocks for next layer

Compute Forecast: $\mathbf{F} \leftarrow \mathbf{x}\mathbf{W}_F$ ▷ one head per GQA group
for query $i = 1, \ldots, N$ **do**
    Compute block scores: $S_{i,b} \leftarrow \mathbf{F}_i \cdot \mathbf{r}_b$ for each block b
    Select: $\mathcal{I}_i \leftarrow \text{TopK}(\{S_{i,b}\}, k)$
    Prefetch $\mathbf{K}_{\mathcal{I}_i}, \mathbf{V}_{\mathcal{I}_i}$ from CPU to GPU for the *next* layer ▷ overlaps with current layer
end for
Compute block-sparse SDPA over current-layer selected blocks: $\mathbf{Y} \leftarrow \text{BlockSparseSDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}, \{\mathcal{I}_i\})$
Training loss: $\mathcal{L} \leftarrow \text{KL}(\text{Forecast dist} \,\|\, \pi_{\text{orig}})$ ▷ only W_F is updated **return Y**

7.10.3 Key Characteristics

- **Complexity**: Forecast $\mathcal{O}(H_{\text{kv}} \cdot d_f \cdot N \cdot N_{\text{blocks}})$; Main attention $\mathcal{O}(H_q \cdot d_h \cdot N \cdot k \cdot B_k)$.
- **Trainable**: Yes; only the Forecast projections are trained ($< 0.5\%$ extra parameters).
- **Strength**: On two 8B sparse-pretrained models, SparDA achieves up to 1.25× prefill and 1.7× decode speedup over the sparse-attention offload baseline, and up to 5.3× decode throughput versus a non-offload sparse baseline.
- **Limitation**: Requires a CPU–GPU offload pipeline and a frozen base selector; the lookahead benefit is contingent on next-layer block demand being predictable from the current Forecast.

7.11 Design Space and Selection Criteria

The seven sparse attention methods occupy complementary positions in a multidimensional design space:

- **Geometric/fixed patterns** (Sparse Transformer, Longformer): Simple to implement and analyze, but patterns do not adapt to content.
- **Learned selection** (SparseK, NSA): Enable adaptation through trainable selection networks, at the cost of higher implementation complexity.
- **Training-free acceleration** (FASA, SpargeAttn): Enable drop-in speedup of existing models without retraining, suited for deployed systems.
- **Theoretical guarantees** (BigBird): Provide formal proofs of expressive completeness, important for understanding safety margins.

Selection guidance:

1. **New model training** where resources permit: NSA or SparseK for maximal inference speedup; BigBird for theoretical assurance.
2. **Long-context document understanding**: Longformer or FASA for practical simplicity; NSA for extreme scale.

3. **Accelerating existing models**: FASA for RoPE-based LLMs; SpargeAttn for any architecture.

4. **Constrained environments**: Sparse Transformer for simplicity and proven efficiency at moderate scale.

8 Gated Attention Families—Complete Literature Analysis

8.1 Executive Summary: Gating for Memory Control

The emerging field of *gated attention mechanisms* addresses a fundamental challenge in sequence modeling: how should information be retained, forgotten, and updated as the model processes new tokens? Traditional additive recurrent states accumulate all information without erasure, leading to memory saturation and inability to adapt to changing context. Softmax attention provides full expressiveness but uses $\mathcal{O}(Ld)$ KV cache during inference, limiting context window size. Gated mechanisms provide a middle ground: selective control over information survival.

The unifying principle across gated architectures is that gating controls *what information survives*. Gates operate at three distinct levels:

1. **Recurrent state decay** (GLA, HGRN2): Multiplicative gates on the memory matrix S_t control how much previous state persists into the next step.

2. **Write strength in recurrent updates** (DeltaNet, Gated DeltaNet, Gated DeltaNet-2): Gates control the magnitude and channel-wise direction of new information written into memory.

3. **Softmax logit biasing** (FoX): Gates inject token-level recency bias into attention logit computation.

4. **Post-attention sparsity** (Gated Softmax): Gates selectively scale SDPA output channels.

This appendix synthesizes eight major gated-attention architectures published 2023–2026, analyzing their mathematical foundations, hardware efficiency characteristics, and empirical trade-offs.

8.2 1. Gated Linear Attention (GLA)

8.2.1 Mathematical Foundation

Gated Linear Attention [85] augments vanilla linear attention (which uses a matrix-valued recurrent state) with data-dependent multiplicative decay. Standard linear attention reformulates the traditional softmax mechanism as a recurrence over hidden states:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t\mathbf{q}_t$$

where state $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ accumulates outer products. This purely additive formulation suffers from "memory overload": information accumulates monotonically and cannot be erased, degrading performance on tasks requiring context selection.

GLA introduces a data-dependent **diagonal gating matrix** $\mathbf{G}_t \in [0,1]^{d_k \times d_k}$:

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t\mathbf{q}_t$$

The gate $\mathbf{G}_t$ is parameterized with an outer-product structure:

$$\mathbf{G}_t = \mathbf{1}\boldsymbol{\alpha}_t^\top, \quad \boldsymbol{\alpha}_t = \text{logsigmoid}\left(\mathbf{W}_{gk}\mathbf{x}_t\right)/c$$

where $\mathbf{W}_{gk}$ is a low-rank projection ($\mathbb{R}^{d \to d_k}$) and $c \approx 16$ is a normalizer. The logsigmoid activation maps $\alpha_t \in \mathbb{R}$ to approximately $[-1/2, 0]$, and division by c further contracts this to near-identity for initialization. The resulting gate $G_t = 1 + \alpha_t \approx 1$ near initialization, then learns to decay ($\alpha_t \to -\infty$) historical information.

Algorithm 32 Gated Linear Attention (Recurrent Form)

Require: Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1}$
Require: Projections: W_q, W_k, W_v (standard); W_{gk} (gate projection)
Ensure: Output $\mathbf{o}_t$ and updated state $\mathbf{S}_t$

$\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$
$\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$
$\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
Compute gate: $\boldsymbol{\alpha}_t \leftarrow \text{logsigmoid}(W_{gk}\mathbf{x}_t)/16$ ▷ Per-key-dim decay
Apply outer-product gating: $\mathbf{G}_t \leftarrow \mathbf{1}\boldsymbol{\alpha}_t^\top$ ▷ diagonal outer product
Decay previous state: $\mathbf{S}_t^{\text{decay}} \leftarrow \mathbf{G}_t \odot \mathbf{S}_{t-1}$ ▷ element-wise product
Add new association: $\mathbf{S}_t \leftarrow \mathbf{S}_t^{\text{decay}} + \mathbf{v}_t \mathbf{k}_t^\top$
Retrieve via query: $\mathbf{o}_t^{\text{raw}} \leftarrow \mathbf{S}_t \mathbf{q}_t$
Apply output gate: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t^{\text{raw}}) \otimes \text{SiLU}(W_g \mathbf{x}_t)$ **return** $\mathbf{o}_t$, $\mathbf{S}_t$

8.2.2 Hardware-Efficient Training

For parallel training, GLA uses a chunkwise block-wise algorithm that groups tokens into chunks C and computes recurrent transitions in parallel:

$$\mathbf{O}_{[t]} = \overleftarrow{\mathbf{Q}}_{[t]} \mathbf{S}_{[t]}^\top + \left(\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \boldsymbol{\Gamma}_{[t]}\right) \mathbf{V}_{[t]}$$

where $\overleftarrow{\mathbf{Q}}_{[t]}$ are attending to the state at the chunk boundary (lookback), and $\boldsymbol{\Gamma}_{[t]}$ is the causal mask with decay-aware scaling:

$$\boldsymbol{\Gamma}_{[t],ij} = \begin{cases} \prod_{l=j}^{i-1} \alpha_l & \text{if } i > j \text{ (lookback)} \\ \prod_{l=1}^{i-j} \alpha_l & \text{if } i \leq j \text{ (in-chunk)} \end{cases}$$

This design maximizes matmul operations susceptible to tensor-core acceleration, achieving sub-quadratic total complexity $\mathcal{O}(nLd^2/C)$ where L is the number of attention heads and C is chunk size.

8.2.3 Strengths and Limitations

Aspect	Strengths	Limitations
Computational efficiency	Sub-quadratic training via chunkwise parallelism. Constant-memory inference $O(d^2)$.	Requires custom CUDA/Triton kernels; not available in all frameworks.
Context generalization	Extends 2K-token training to 20K+ tokens with negligible perplexity degradation.	Still underperforms softmax on retrieval-heavy benchmarks (needle-in-haystack).
Selectivity	Per-key-dimension data-dependent decay.	Gate structure limits per-key-value selectivity; no direct control over which specific associations to erase.

8.3 2. DeltaNet: Error-Correcting Linear Attention

8.3.1 Mathematical Foundation

DeltaNet [86] applies a classical **delta learning rule** to linear attention state updates. Instead of additive accumulation, DeltaNet computes the difference between the predicted value (retrieved from memory) and the target value, then performs an error-correcting update:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

where $\beta_t \in (0, 1)$ is a data-dependent **writing strength** scalar. This update can be decomposed as an erase-write operation:

$$\mathbf{v}_t^{\text{old}} = \mathbf{S}_{t-1}\mathbf{k}_t \quad \text{(current prediction)}, \quad \mathbf{v}_t^{\text{updated}} = \beta_t \mathbf{v}_t + (1 - \beta_t)\mathbf{v}_t^{\text{old}} \quad \text{(blend old/new)}$$

so that:

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \mathbf{v}_t^{\text{old}} \mathbf{k}_t^\top + \mathbf{v}_t^{\text{updated}} \mathbf{k}_t^\top$$

The key insight is that this update rule minimizes an online MSE loss at each timestep:

$$\mathcal{L}_t(\mathbf{S}) = \tfrac{1}{2}\|\mathbf{S}\mathbf{k}_t - \mathbf{v}_t\|^2 \Rightarrow \nabla_{\mathbf{S}} \mathcal{L}_t = (\mathbf{S}\mathbf{k}_t - \mathbf{v}_t)\mathbf{k}_t^\top$$

One step of SGD with learning rate β_t yields the delta rule update. This connection to test-time training (TTT) provides theoretical grounding: DeltaNet is directly optimizing for value prediction at inference time.

8.3.2 Efficient Parallel Training

DeltaNet's transition matrix $\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top$ is a generalized Householder matrix (rank-1 update to identity). For chunkwise training, DeltaNet uses the **WY representation**, which represents the product of m such rank-1 updates compactly:

$$\prod_{i=1}^{m} (\mathbf{I} - \beta_i \mathbf{k}_i \mathbf{k}_i^\top) = \mathbf{I} - \mathbf{W}\mathbf{Y}^\top$$

where $\mathbf{W}, \mathbf{Y} \in \mathbb{R}^{d \times m}$ are constructed recursively. This factorization enables matmul-rich parallelism without materializing the full product, keeping chunkwise training efficient.

Algorithm 33 DeltaNet State Update with WY Representation

Require: Input chunk $\mathbf{X}_{[t]}$; prior state $\mathbf{S}_{t-1}$
Require: L2-normalized queries $\hat{\mathbf{Q}}_t$, keys $\hat{\mathbf{K}}_t$, values $\mathbf{V}_t$
Ensure: Output $\mathbf{O}_{[t]}$ and updated state $\mathbf{S}_t$
Chunkwise phase:
for position $i = 1$ to chunk_size **do**
  Normalize: $\hat{\mathbf{k}}_i \leftarrow \hat{\mathbf{K}}_t[i]/\|\hat{\mathbf{K}}_t[i]\|$, $\hat{\mathbf{q}}_i \leftarrow \hat{\mathbf{Q}}_t[i]/\|\hat{\mathbf{Q}}_t[i]\|$
  Compute write strength: $\beta_i \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_i)$
  Prediction: $\hat{\mathbf{v}}^{\text{pred}} \leftarrow \mathbf{S}_{i-1}^{\text{in-chunk}} \hat{\mathbf{k}}_i$
  Error-aware blend: $\mathbf{v}_i^{\text{update}} \leftarrow \beta_i(\mathbf{v}_i - \hat{\mathbf{v}}^{\text{pred}})$
  Erase-write: $\mathbf{S}_i^{\text{in-chunk}} \leftarrow \mathbf{S}_{i-1}^{\text{in-chunk}} - \hat{\mathbf{v}}^{\text{pred}} \hat{\mathbf{k}}_i^\top + (\beta_i \mathbf{v}_i + (1 - \beta_i)\hat{\mathbf{v}}^{\text{pred}})\hat{\mathbf{k}}_i^\top$
  Output: $\mathbf{o}_i \leftarrow \mathbf{S}_i^{\text{in-chunk}} \hat{\mathbf{q}}_i$
end for
Inter-chunk phase: Use WY representation of transition products **return** $\mathbf{O}_{[t]}$, $\mathbf{S}_t$

8.3.3 Strengths and Limitations

Aspect	Strengths	Limitations
Associative recall	Perfect recall on Multi-Query Associative Recall (MQAR) benchmark; error-correcting semantics naturally fit retrieval patterns.	Without global forgetting, memory still crowds over extreme sequence lengths; requires periodic reset mechanisms for unbounded context.
Theory	Grounded in online MSE optimization; clear connection to test-time training paradigm.	Requires L2-normalized keys for numerical stability; double-width normalization adds overhead.
Throughput	WY representation enables efficient chunkwise training.	Slightly slower than Mamba2 per-token due to richer transition matrices (rank-1 instead of diagonal).

8.4 3. Gated DeltaNet: Synthesis of Gating and Error Correction

8.4.1 Mathematical Foundation

Gated DeltaNet [87] synthesizes the strengths of GLA and DeltaNet by combining a **decay gate** α_t (from GLA) with the **writing strength gate** β_t (from DeltaNet):

$$\mathbf{S}_t = \mathbf{S}_{t-1}\left(\alpha_t(\mathbf{I} - \beta_t \mathbf{k}_t\mathbf{k}_t^\top)\right) + \beta_t \mathbf{v}_t\mathbf{k}_t^\top$$

Rewriting for clarity:

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t\mathbf{k}_t^\top) + \beta_t \mathbf{v}_t\mathbf{k}_t^\top$$

The two gates are complementary:

- $\alpha_t \to 0$ rapidly erases historical state: $\mathbf{S}_t \approx \beta_t \mathbf{v}_t\mathbf{k}_t^\top$ (context reset).
- $\alpha_t \to 1$ recovers pure delta-rule behavior: $\mathbf{S}_t \approx \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t\mathbf{k}_t^\top) + \beta_t \mathbf{v}_t\mathbf{k}_t^\top$ (targeted updates).
- $\beta_t \to 0$ skips writing but decays: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1}$ (pure forgetting).
- $\beta_t \to 1$ replaces memory aggressively: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top$ (replacement).

From an online learning perspective, Gated DeltaNet corresponds to:

$$\min_{\mathbf{S}_t} \|\mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\|_F^2 - 2\langle \mathbf{S}_t\mathbf{k}_t, \beta_t(\mathbf{v}_t - \alpha_t \mathbf{S}_{t-1}\mathbf{k}_t)\rangle$$

which introduces an adaptive weight decay α_t into an SGD-like update—analogous to decoupled weight decay in deep learning optimization.

Algorithm 34 Gated DeltaNet Parallel Chunkwise Forward

Require: Input chunk $\mathbf{X}_{[i]}$; prior state $\mathbf{S}_{i-1}$; chunk size C
Require: Projections: $W_q, W_k, W_v, W_\alpha, W_\beta$
Ensure: Output $\mathbf{O}_{[i]}$ and state $\mathbf{S}_i$
In-chunk computation:
for $j = 1$ to C **do**
  $\mathbf{q}_j \leftarrow W_q\mathbf{x}_j$, $\mathbf{k}_j \leftarrow W_k\mathbf{x}_j$, $\mathbf{v}_j \leftarrow W_v\mathbf{x}_j$
  $\alpha_j \leftarrow \text{sigmoid}(W_\alpha\mathbf{x}_j)$, $\beta_j \leftarrow \text{sigmoid}(W_\beta\mathbf{x}_j)$
  Apply combined gate: $\mathbf{S}_j \leftarrow \alpha_j\mathbf{S}_{j-1}(\mathbf{I} - \beta_j\mathbf{k}_j\mathbf{k}_j^\top) + \beta_j\mathbf{v}_j\mathbf{k}_j^\top$
  $\mathbf{o}_j \leftarrow \mathbf{S}_j\mathbf{q}_j$
end for
Inter-chunk recurrence:
$\mathbf{S}_i \leftarrow \mathbf{S}_C$ from in-chunk; propagate across chunks via cumulative $\prod \alpha$ masks
Output gating: $\mathbf{O}_{[i]} \leftarrow \text{GroupNorm}(\mathbf{O}^{\text{raw}}) \otimes \text{SiLU}(W_g\mathbf{X}_{[i]})$ **return** $\mathbf{O}_{[i]}$, $\mathbf{S}_i$

8.4.2 Empirical Performance and Trade-offs

Gated DeltaNet has been integrated into Alibaba's Qwen3-Next and Qwen3.5 production models. Empirically:

Task	Gated DeltaNet	Mamba2	DeltaNet	GLA
Language Modeling	**1.0**×	1.08×	1.05×	1.12×
Associative Recall	**1.0**×	$--(noperfect)$	**1.0**×	0.75
Length Extrapolation	**1.0**×	0.98×	0.96×	0.94×
Throughput (tokens/sec)	0.85×	**1.0**×	0.82×	0.88×

Gated DeltaNet achieves the best balance across diverse tasks at the cost of slightly reduced throughput due to richer transition matrices.

8.5 4. HGRN2: Hierarchical Gating with Outer-Product Expansion

8.5.1 Mathematical Foundation

HGRN2 [64] uses an outer-product-based state expansion with **hierarchically lower-bounded forget gates**. The state update is:

$$\mathbf{S}_t = \text{diag}(\mathbf{g}_t) \cdot \mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t\mathbf{q}_t$$

where $\mathbf{g}_t \in [b_\ell, 1]^d$ is a lower-bounded forget gate for layer ℓ. The bounds satisfy:

$$0 \leq b_{\ell_{\text{shallow}}} < b_{\ell_{\text{middle}}} < b_{\ell_{\text{deep}}} \leq 1$$

i.e., bounds increase (become less restrictive) in the deeper layers. The gate itself is computed as:

$$\mathbf{g}_t = b_\ell + (1 - b_\ell) \cdot \sigma(W_g\mathbf{x}_t)$$

where σ is sigmoid. When W_g outputs weakly negative logits, $\mathbf{g}_t \approx b_\ell$ (forced retention at shallow layers). When outputs are strongly positive, $\mathbf{g}_t \approx 1$ (memory refresh at any layer).

The hierarchical structure encourages different layers to specialize to different timescales: shallow layers model local dependencies (high retention due to binding b_ℓ), while deeper layers capture long-range structure (flexible gating with high upper bound).

Algorithm 35 HGRN2 Forward with Hierarchical Bounds

Require: Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1}$; layer index $\ell \in [0, L-1]$
Require: Non-decreasing bounds: $0 \leq b_0 < b_1 < \cdots < b_{L-1} \leq 1$
Ensure: Output $\mathbf{o}_t$ and state $\mathbf{S}_t$
1: Layer-specific retain bound: $b \leftarrow b_\ell$
2: Project: $\mathbf{q}_t \leftarrow W_q\mathbf{x}_t$, $\mathbf{k}_t \leftarrow W_k\mathbf{x}_t$, $\mathbf{v}_t \leftarrow W_v\mathbf{x}_t$
3: Compute forget gate with binding: $\mathbf{g}_t \leftarrow b + (1-b) \cdot \sigma(W_g\mathbf{x}_t)$ ▷ Constrained to $[b, 1]$
4: Diagonal multiplication (vectorized): $\mathbf{S}_t \leftarrow \mathbf{S}_{t-1} \circ \text{diag}(\mathbf{g}_t) + \mathbf{v}_t\mathbf{k}_t^\top$
5: Retrieve: $\mathbf{o}_t \leftarrow \mathbf{S}_t\mathbf{q}_t$
6: Optional normalization: $\mathbf{o}_t \leftarrow \text{RMSNorm}(\mathbf{o}_t)$ **return** $\mathbf{o}_t$, $\mathbf{S}_t$

8.5.2 Scaling and Empirical Results

At 3B scale on 100B tokens, HGRN2 slightly outperforms Mamba2 and LLaMA-architecture Transformers on language modeling. The hierarchical gating provides multi-scale temporal modeling without explicit layer-wise architectural variants. However, vector-valued diagonal gating (as opposed to element-wise selection) provides less per-item flexibility than delta-rule variants.

8.6 5. Forgetting Transformer (FoX): Gating in Softmax Logit Space

8.6.1 Mathematical Foundation

Forgetting Transformer (FoX), proposed by Lin et al. [47], embeds a forget gate directly into softmax attention logits. Rather than replacing softmax with linear recurrence, FoX preserves full softmax expressiveness while adding recency control through a data-dependent logit bias.

For each token position t and all keys $j \leq t$, compute a scalar forget gate:

$$f_t = \sigma(w_f^\top \mathbf{x}_t + b_f)$$

The logit bias at position (i, j) is the cumulative log-forget product:

$$d_{ij} = \sum_{l=j+1}^{i} \log f_l = \log \prod_{l=j+1}^{i} f_l$$

The full attention output becomes:

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{D}\right)\mathbf{V}$$

where $\mathbf{D}_{ij} = d_{ij}$. Equivalently:

$$\text{Attention}(i, j) = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j/\sqrt{d_k} + d_{ij})}{\sum_{j'=1}^{i} \exp(\mathbf{q}_i^\top \mathbf{k}_{j'}/\sqrt{d_k} + d_{ij'})}$$

This weighting down-weights older tokens multiplicatively: a token from 10 steps ago is scaled by $\prod_{l=j+1}^{i} f_l$, which is typically much less than 1 if $f_t < 1$ on average.

The mechanism is mathematically equivalent to a **data-dependent learnable variant of ALiBi (Attention with Linear Biases)**, where earlier work used fixed $d_{ij} = -\infty \cdot |i-j|$ or $d_{ij} = -(i-j)$.

Algorithm 36 FoX: Forgetting Attention with Recency Bias

Require: Queries $\mathbf{Q}$, keys $\mathbf{K}$, values $\mathbf{V}$; input $\mathbf{X}$
Require: Forget gate parameters: $w_f \in \mathbb{R}^d$, $b_f \in \mathbb{R}$
Ensure: Output attention $\mathbf{O}$
Compute forget gates:
for $t = 1$ to T **do**
    $f_t \leftarrow \sigma(w_f^\top \mathbf{x}_t + b_f)$ ▷ Per-token forget probability
end for
Compute cumulative log-forget biases:
for $i = 1$ to T **do**
    for $j = 1$ to i **do**
        $d_{ij} \leftarrow \sum_{l=j+1}^{i} \log f_l$ ▷ Cumulative product in log space
    end for
end for
Standard SDPA with bias:
Compute scores: $\mathbf{S} \leftarrow \mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}$
Add bias: $\mathbf{S} \leftarrow \mathbf{S} + \mathbf{D}$
Apply softmax: $\mathbf{A} \leftarrow \text{softmax}(\mathbf{S}, \text{dim} = -1)$
Weight values: $\mathbf{O} \leftarrow \mathbf{AV}$ **return** $\mathbf{O}$

8.6.2 Integration with FlashAttention

The key advantage of FoX is compatibility with FlashAttention and existing optimized implementations. The forget biases are added in the softmax logit computation, which is already a core operation in FlashAttention's block-wise algorithm. The overhead is:

$$\text{Overhead} \approx 0.5 - -2\% \text{ wall-clock time}$$

since the bias addition is amortized across matmul operations.

8.6.3 Strengths and Limitations

FoX achieves state-of-the-art results on length extrapolation, near-perfect needle-in-haystack retrieval, and superior long-context understanding compared to Transformers. However, it remains $\mathcal{O}(L^2 d)$ at training and $\mathcal{O}(Ld)$ per step at inference with KV cache, limiting extreme sequence lengths.

8.7 6. Gated Attention (Post-SDPA Sigmoid Gating)

8.7.1 Mathematical Foundation

The NeurIPS 2025 Best Paper by the Qwen team proposes applying a sigmoid gate **after** scaled dot-product attention (SDPA):

$$\mathbf{Y} = \text{SDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

$$\mathbf{Y}' = \mathbf{Y} \odot \sigma\left(\mathbf{X}\mathbf{W}_\theta\right)$$

where the gate score $\sigma(\mathbf{X}\mathbf{W}_\theta)$ can have shape $\mathbb{R}^{n \times q \times d_k}$ (element-wise gating per query head) or $\mathbb{R}^{n \times q}$ (head-wise fixed gating). Across 30+ variants and 15B MoE + 1.7B dense models trained on 3.5T tokens, the team found:

- Headwise gating adds only $\sim 1.6M$ parameters to a 15B model (negligible overhead).

- Effective gates are **sparse**: mean activation ≈ 0.116 (i.e., 88% are zero or near-zero).
- Gates act as query-dependent filters, suppressing low-value channels while preserving high-value signal.
- Gates demonstrably eliminate the **attention sink** phenomenon, where the attention pattern becomes dominated by a single early token.

Algorithm 37 Gated Softmax Attention (Post-SDPA)

Require: Queries $\mathbf{Q}$, Keys $\mathbf{K}$, Values $\mathbf{V}$; input $\mathbf{X}$
Require: Gate projection matrix $W_g \in \mathbb{R}^{d \to d_{\text{gate}}}$ where $d_{\text{gate}} \in \{1, d_k\}$
Ensure: Gated output $\mathbf{Y}'$

Standard SDPA:
Compute attention: $\mathbf{A} \leftarrow \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k})$
Weight values: $\mathbf{Y} \leftarrow \mathbf{A}\mathbf{V}$
Compute gates:
Project input: $\mathbf{G} \leftarrow \mathbf{X}W_g$ ▷ shape: $[n, q, d_{\text{gate}}]$
Apply sigmoid: $\mathbf{G} \leftarrow \sigma(\mathbf{G})$ ▷ Element-wise gating
Apply gating:
if $d_{\text{gate}} = 1$ **then**
    Broadcast and scale: $\mathbf{Y}' \leftarrow \mathbf{Y} \cdot \mathbf{G}$ ▷ Head-wise scaling
else ▷ $d_{\text{gate}} = d_k$
    Element-wise modulation: $\mathbf{Y}' \leftarrow \mathbf{Y} \odot \mathbf{G}$ ▷ Per-dimension gating
end ifreturn $\mathbf{Y}'$

8.7.2 Key Findings

- **Attention-sink suppression**: Gating breaks the feedback loop where softmax concentrates on the first token, enabling more balanced attention.
- **Training stability**: Models with gating tolerate larger learning rates (+30% higher stable η_{max}).
- **Minimal overhead**: Latency impact is only 1.6% on H100 GPUs; throughput drops at most $2 - 3\%$.
- **Scaling law improvement**: Improves loss across model scales from 800M to 110B parameters consistently.

8.8 7. Gated DeltaNet-2: Decoupled Channel-wise Erase and Write

8.8.1 Mathematical Foundation

Gated DeltaNet-2 [34] addresses a shared limitation of Gated DeltaNet and Kimi Delta Attention (KDA): both use a single *scalar* delta gate β_t to simultaneously control (i) how much old content is erased along the key-side read direction and (ii) how much new value is written along the value axis. Gated DeltaNet-2 **decouples** these two roles into a channel-wise erase gate $\mathbf{b}_t$ (key axis) and a channel-wise write gate $\mathbf{w}_t$ (value axis), while inheriting the channel-wise decay $\boldsymbol{\alpha}_t$ from KDA. With state $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$, decay $\mathbf{D}_t = \text{Diag}(\boldsymbol{\alpha}_t)$, erase direction $\mathbf{e}_t = \mathbf{b}_t \odot \mathbf{k}_t$, and write target $\mathbf{z}_t = \mathbf{w}_t \odot \mathbf{v}_t$, the state update is:

$$\mathbf{S}_t = \left(\mathbf{I} - \mathbf{k}_t \mathbf{e}_t^\top\right) \mathbf{D}_t \, \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{z}_t^\top, \qquad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$$

or, expanding the gate products:

$$\mathbf{S}_t = \Big(\mathbf{I} - \mathbf{k}_t(\mathbf{b}_t \odot \mathbf{k}_t)^\top\Big)\,\mathbf{D}_t\,\mathbf{S}_{t-1} + \mathbf{k}_t(\mathbf{w}_t \odot \mathbf{v}_t)^\top$$

The two gates are produced by independent linear projections followed by sigmoid:

$$\mathbf{b}_t = \sigma(\mathbf{W}_b\mathbf{x}_t), \qquad \mathbf{w}_t = \sigma(\mathbf{W}_w\mathbf{x}_t)$$

and the channel-wise decay follows a log-decay parameterization (computed in fp32 to avoid precision loss in long cumulative products):

$$\mathbf{g}_t = \mathbf{a} - \mathrm{softplus}(\boldsymbol{\delta}), \qquad \boldsymbol{\alpha}_t = \exp(\mathbf{g}_t)$$

The structural roles of the two gates are complementary:

- $\mathbf{b}_t$ (key axis) makes the **read direction** channel-selective: it determines which key channels are removed before writing.
- $\mathbf{w}_t$ (value axis) makes the **write target** channel-selective: it determines which value channels are deposited into memory.
- $\boldsymbol{\alpha}_t$ (per key channel) provides global decay, as in KDA.

Reductions. Setting $\mathbf{b}_t = \beta_t\mathbf{1}_{d_k}$ and $\mathbf{w}_t = \beta_t\mathbf{1}_{d_v}$ recovers KDA; further collapsing $\boldsymbol{\alpha}_t$ to a scalar recovers Gated DeltaNet. From the fast-weight / online-learning perspective, the update is the minimizer of:

$$\mathcal{L}_t(\mathbf{S}) = \tfrac{1}{2}\|\mathbf{S} - \mathbf{D}_t\mathbf{S}_{t-1}\|_F^2 - 2\left\langle \mathbf{S}^\top\mathbf{k}_t,\ \mathbf{z}_t - (\mathbf{D}_t\mathbf{S}_{t-1})^\top\mathbf{e}_t \right\rangle$$

Algorithm 38 Gated DeltaNet-2 Recurrent Forward (Reference Form)

Require: Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1} \in \mathbb{R}^{d_k \times d_v}$
Require: Projections: W_q, W_k, W_v, W_b, W_w; log-decay parameters $\mathbf{a}, \boldsymbol{\delta}$
Ensure: Output $\mathbf{o}_t$ and state $\mathbf{S}_t$

$\mathbf{q}_t \leftarrow \mathrm{normalize}(W_q\mathbf{x}_t)$, $\mathbf{k}_t \leftarrow \mathrm{normalize}(W_k\mathbf{x}_t)$, $\mathbf{v}_t \leftarrow \mathrm{silu}(W_v\mathbf{x}_t)$
$\mathbf{b}_t \leftarrow \sigma(W_b\mathbf{x}_t)$ ▷ key-axis erase gate, $\mathbb{R}^{d_k}$
$\mathbf{w}_t \leftarrow \sigma(W_w\mathbf{x}_t)$ ▷ value-axis write gate, $\mathbb{R}^{d_v}$
$\boldsymbol{\alpha}_t \leftarrow \exp(\mathbf{a} - \mathrm{softplus}(\boldsymbol{\delta}))$ ▷ channel-wise decay, fp32
Erase direction: $\mathbf{e}_t \leftarrow \mathbf{b}_t \odot \mathbf{k}_t$; write target: $\mathbf{z}_t \leftarrow \mathbf{w}_t \odot \mathbf{v}_t$
Decay state: $\mathbf{S}_t \leftarrow \mathrm{Diag}(\boldsymbol{\alpha}_t)\,\mathbf{S}_{t-1}$
Read: $\mathbf{r}_t \leftarrow \mathbf{S}_t^\top\mathbf{e}_t$ ▷ sum over key axis
Delta-rule update: $\mathbf{S}_t \leftarrow \mathbf{S}_t - \mathbf{k}_t\mathbf{r}_t^\top + \mathbf{k}_t\mathbf{z}_t^\top$
Retrieve: $\mathbf{o}_t \leftarrow \mathbf{S}_t^\top\mathbf{q}_t$
Output gating: $\mathbf{o}_t \leftarrow \mathrm{GroupNorm}(\mathbf{o}_t) \odot \mathrm{silu}(W_g\mathbf{x}_t)$
return $\mathbf{o}_t$, $\mathbf{S}_t$

The chunkwise training algorithm extends KDA's WY representation by absorbing the channel-wise decay into an asymmetric delta recurrence on a decay-normalized state, enabling matmul-rich parallelism on tensor cores. Because $\mathbf{b}_t$ and $\mathbf{w}_t$ are per-channel diagonal operators varying per row, the scalar post-scaling shortcut of KDA breaks: the backward pass requires a **gate-aware WY** inversion that bakes the gates into each dot product accumulating the gradient.

8.8.2 Empirical Performance and Trade-offs

At 1.3B parameters trained on 100B FineWeb-Edu tokens, Gated DeltaNet-2 outperforms Mamba-2, Mamba-3, Gated DeltaNet, and KDA on language modeling, commonsense reasoning, and—most markedly—real-world and RULER long-context retrieval (especially multi-key needle-in-a-haystack). Its throughput on H100 stays near-flat (38.0 → 36.1 Kt/s) as sequence length grows from 2K to 16K, where a Transformer's drops sharply. Ablations show the erase gate $\mathbf{b}_t$ accounts for most of the gain; the write gate $\mathbf{w}_t$ contributes a smaller additional improvement. The hybrid variant (recurrent Gated DeltaNet-2 + sliding-window attention) further closes the gap with pure-attention models on tasks requiring local evidence aggregation.

Aspect	Strength	Limitation
Recall	Best-in-class among recurrent delta-rule models on multi-key NIAH.	Recurrent variant still gaps Transformer on NQ/DROP.
Throughput	Near-flat scaling 2K→16K; small overhead vs KDA.	Requires a new gate-aware WY Triton backward kernel.
Memory	Constant $\mathcal{O}(d^2)$ inference state.	Neg-eigenvalue erase range $[0, 2]$ gives no consistent gain at 1.3B scale.

Table 11: Gated DeltaNet-2 trade-offs at 1.3B / 100B-token scale [34].

8.9 8. Kimi Delta Attention (KDA): Channel-wise Decay Gating for the Delta Rule

8.9.1 Mathematical Foundation

Kimi Delta Attention [40] is the core module of Kimi Linear and extends Gated DeltaNet with a finer-grained **channel-wise decay gate**. Where Gated DeltaNet uses a single scalar per-head decay α_t alongside the scalar delta write gate β_t, KDA replaces the scalar decay with a **per-key-channel** decay $\boldsymbol{\alpha}_t \in \mathbb{R}^{d_k}$, parameterised through a log-decay formulation and applied as a diagonal matrix $\mathbf{D}_t = \mathrm{diag}(\boldsymbol{\alpha}_t)$ before the delta-rule update. The delta rule itself retains a scalar per-head write gate β_t:

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)\, \mathbf{D}_t\, \mathbf{S}_{t-1} + \beta_t \mathbf{v}_t \mathbf{k}_t^\top, \qquad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$$

The channel-wise decay follows a log-decay parameterisation (computed in fp32 for numerical stability over long cumulative products):

$$\mathbf{g} = \mathbf{a} - \mathrm{softplus}(\boldsymbol{\delta}), \qquad \boldsymbol{\alpha}_t = \exp(\mathbf{g})$$

where $\mathbf{a}$ and $\boldsymbol{\delta}$ are learned per-channel parameters. This gives each key channel an independent forgetting timescale, allowing the model to retain some channels longer than others within the same head.

Reduction to Gated DeltaNet : When $\boldsymbol{\alpha}_t$ collapses to a scalar (i.e., all channels share the same decay), KDA reduces exactly to Gated DeltaNet. The channel-wise decay is thus a strict generalisation that adds per-channel selectivity on top of the scalar decay/write gating of Gated DeltaNet.

Chunkwise training : A bespoke chunkwise algorithm uses a specialised **Diagonal-Plus-Low-Rank (DPLR)** variant of the transition matrix to remain hardware-efficient. The product of per-step transitions $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)\mathbf{D}_t$ admits a DPLR decomposition that allows matmul-rich parallelism on tensor cores while preserving the channel-wise decay structure.

Algorithm 39 Kimi Delta Attention Recurrent Forward (Reference Form)

Require: Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1} \in \mathbb{R}^{d_k \times d_v}$
Require: Projections: W_q, W_k, W_v, W_β; log-decay parameters $\mathbf{a}, \boldsymbol{\delta}$
Ensure: Output $\mathbf{o}_t$ and state $\mathbf{S}_t$

$\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$, $\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$, $\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
$\beta_t \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_t)$ ▷ scalar per-head write gate
$\boldsymbol{\alpha}_t \leftarrow \exp(\mathbf{a} - \text{softplus}(\boldsymbol{\delta}))$ ▷ channel-wise decay, fp32
$\mathbf{D}_t \leftarrow \text{diag}(\boldsymbol{\alpha}_t)$ ▷ diagonal decay matrix
Decay state: $\mathbf{S}_t \leftarrow \mathbf{D}_t \mathbf{S}_{t-1}$
Read: $\mathbf{r}_t \leftarrow \mathbf{S}_t^\top \mathbf{k}_t$ ▷ current prediction along key
Delta-rule update: $\mathbf{S}_t \leftarrow \mathbf{S}_t - \beta_t \mathbf{k}_t \mathbf{r}_t^\top + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$
Retrieve: $\mathbf{o}_t \leftarrow \mathbf{S}_t^\top \mathbf{q}_t$
Output gating: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t) \odot \text{silu}(W_g \mathbf{x}_t)$
return $\mathbf{o}_t$, $\mathbf{S}_t$

8.9.2 Empirical Performance and Trade-offs

KDA is deployed in Kimi Linear, a hybrid model (3B activated / 48B total parameters) combining KDA layers with Multi-head Latent Attention (MLA). Kimi Linear outperforms a full-MLA baseline across all evaluated tasks, reduces the KV cache by up to 75%, and achieves up to 6× decoding throughput for 1M-token context. The channel-wise decay is the principal driver of the gain: it allows fine-grained control over which key channels persist, improving long-context recall relative to scalar-decay Gated DeltaNet.

Aspect	Strength	Limitation
Recall	Channel-wise decay improves long-context recall over scalar-decay baselines; deployed in Kimi Linear (3B/48B hybrid).	Recurrent matrix state $\mathcal{O}(d^2)$ is heavier than diagonal-gated linear attention.
Throughput	Up to 6× decoding throughput at 1M context; 75% KV cache reduction vs full MLA.	Requires DPLR-aware chunkwise kernels for hardware efficiency.
Memory	Constant $\mathcal{O}(d^2)$ inference state (matrix recurrent).	Log-decay fp32 computation adds overhead; reduction to Gated DeltaNet loses channel selectivity.

Table 12: KDA trade-offs in Kimi Linear (3B activated / 48B total) [40].

8.10 Unified Gating Principle

All eight architectures share a common principle: **gating is a mechanism to control information flow and selective memory retention**. The specific design choices diverge along several dimensions:

1. **Location of gating**: Recurrent state updates (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2) versus softmax logit/output space (FoX, Gated Softmax, RetNet).

2. **Granularity of control**: Dimension-wise (GLA, HGRN2 diagonal), key-specific (DeltaNet), token-wise (FoX), head-wise (Gated Softmax), or channel-wise decoupled erase/write (Gated DeltaNet-2).

3. **Data dependence**: Fully data-dependent (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, FoX, Gated Softmax) versus fixed schedules (RetNet with exponential decay).

4. **Expressiveness trade-off**: Linear recurrent efficiency (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, RetNet) versus full softmax expressiveness (FoX, Gated Softmax).

8.11 Implementation and Practical Guidance

8.11.1 When to Use Each Architecture

1. **GLA**: Fast inference with moderate-to-long contexts (2K–20K tokens), when custom CUDA kernels are acceptable, and retrieval performance is not critical.
2. **DeltaNet**: Strong associative recall and in-context learning tasks; good for synthetic tasks (MQAR) and key-value association testing.
3. **Gated DeltaNet**: Production models requiring the best balance of recall, forgetting, and throughput (proven in Qwen3-Next; ICLR 2025).
4. **Gated DeltaNet-2**: Long-context retrieval workloads (multi-key NIAH) where decoupled channel-wise erase/write yields the strongest recall; when a gate-aware WY kernel is available.
5. **RetNet**: Extremely efficient inference without KV caching; suitable for edge/mobile deployments or toy models.
6. **HGRN2**: Multi-scale temporal hierarchies; when layer-specific forget bounds are desirable.
7. **FoX**: Length extrapolation and long-context understanding; when quadratic training is acceptable and replacing softmax is not desired.
8. **Gated Softmax**: Quick improvement to existing softmax models with minimal code changes; best for practitioners wanting immediate gains without major refactoring.

8.11.2 Hardware Considerations

Hardware Target	Recommended Architectures	Notes
GPU (H100/A100)	Gated Softmax, FoX, Gated DeltaNet, Gated DeltaNet-2	Mature softmax/linear implementations. GLA/DeltaNet require custom kernels.
TPU (high memory)	GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2	Hardware supports efficient matmuls; linear recurrent methods shine.
Mobile/Edge	RetNet, lightweight Gated Softmax	Constant-memory inference critical.

9 Normalization, Quantization, Depth Routing, and Runtime Algorithms

This annex collects the implementation-level detail for normalization variants, BitNet-style ternary quantization, adaptive depth routing (Mixture-of-Depths), conditional memory (Engram), factorized embeddings, and the runtime algorithms (training-step stability controls, SBERT inference router) that support the configuration-driven pipeline. The main sections defer to this annex for the mathematical formulations and pseudocode; the bibliography keys cited here resolve against `docs/bibliography/`.

9.1 Normalization Variants: LayerNorm, RMSNorm, pRMSNorm, Dynamic Tanh, Dynamic Erf, and FlashNorm

Normalization determines how activation scale is controlled across depth. In this repository, normalization is not only a modeling choice but also a schema compatibility question, because only certain values are currently accepted by `norm_type`. The schema contract is:

$$\texttt{norm\_type} \in \{\texttt{layer\_norm}, \texttt{dynamic\_tanh}, \texttt{derf}, \texttt{rms\_norm}, \texttt{prms\_norm}, \texttt{flash\_norm}\}$$

The formulations most relevant to this codebase are:

9.1.1 LayerNorm (Baseline)

LayerNorm [4] is the default normalizer; it centers and rescales each token by its per-token mean and variance:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i-\mu)^2, \quad y_i = \gamma_i \frac{x_i-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta_i.$$

9.1.2 RMSNorm

RMSNorm removes mean-centering and only rescales by root mean square magnitude [92]:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}, \qquad y_i = \gamma_i \frac{x_i}{\mathrm{RMS}(x)}.$$

Compared with LayerNorm, RMSNorm is computationally simpler (no subtraction of feature mean) and is often used when reducing normalization overhead is important (e.g., Llama, GPT-NeoX). It has a learnable per-dimension scale γ but no bias term.

9.1.3 Partial RMSNorm (pRMSNorm)

pRMSNorm [92] further reduces overhead by estimating the RMS from only the first $p\%$ of the hidden dimensions:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{k}\sum_{i=1}^{k} x_i^2 + \epsilon}, \qquad k = \lceil d \cdot p \rceil,$$

exploiting the assumption that neurons within a layer are approximately i.i.d. The same scale γ is then applied to all dimensions. The partial ratio p is controlled by the `prms_partial_ratio` config field (default 6.25%). The paper reports that competitive results are achievable with as little as 6.25% of the inputs, though gradient instability can arise with very small ratios.

9.1.4 Dynamic Tanh (DyT)

Dynamic Tanh proposes replacing explicit normalization with a bounded elementwise map [97]:

$$\mathrm{DyT}(x) = \tanh(\alpha x)$$

where α is learned. The core idea is that bounded nonlinear contraction can provide stable signal scaling without explicitly computing per-token normalization statistics.

9.1.5 Dynamic Erf (Derf)

Derf extends the same normalization-free direction by using an error-function based map [12]:

$$\mathrm{Derf}(x) = \mathrm{erf}(\alpha x + s)$$

with learnable scale/shift. Reported results in the cited work indicate stronger performance than DyT and common normalization baselines across multiple domains.

9.1.6 FlashNorm (Weightless RMSNorm)

FlashNorm [30] is not a new mathematical normalization; it is an *exact algebraic rewrite* of the ubiquitous "RMSNorm → Linear" pair that (i) eliminates the per-dimension learnable scale $\mathbf{g}$ by folding it into the subsequent linear layer's weights, and (ii) defers the scalar RMS reduction to the output of the matrix multiplication so that the matmul (matrix unit) and the RMS reduction (vector unit) execute in parallel. Three propositions formalize the rewrite.

Proposition 1 (Weight folding). Given activation $\mathbf{a} \in \mathbb{R}^n$, RMSNorm weight $\mathbf{g} \in \mathbb{R}^n$, and a subsequent linear $\mathbf{W} \in \mathbb{R}^{n \times k}$,

$$\mathbf{z} = \mathrm{RMSNorm}(\mathbf{a})\mathbf{W} = \frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^*, \qquad W^*_{i,j} = g_i \cdot W_{i,j} \;\Leftrightarrow\; \mathbf{W}^* = \mathrm{diag}(\mathbf{g})\mathbf{W}.$$

The per-dimension scale $\mathbf{g}$ becomes redundant and is folded into $\mathbf{W}$ once at load time. The standalone `FlashNorm` module in this repository applies this directly: it has *no learnable parameters* (the scale is meant to be absorbed by the downstream projection).

Proposition 2 (Deferred normalization). For a bias-free linear $\mathbf{W}^*$, scalar–matrix commutativity yields

$$\frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^* = (\mathbf{a}\mathbf{W}^*) \cdot \frac{1}{\mathrm{RMS}(\mathbf{a})}.$$

The matmul $\mathbf{a}\mathbf{W}^*$ and the RMS reduction are independent of each other; on hardware with distinct matrix and vector execution units they execute in parallel, and only a single vector–scalar multiply synchronizes them. The fused module `FlashNormLinear` implements this rewrite: on the bias-free path it computes the matmul on the *un-normalized* input and multiplies the result by the per-token reciprocal RMS. When the linear has a bias $\mathbf{c}$, the rewrite is no longer exact (Remark 1 of the paper): $(\mathbf{a}\mathbf{W}^* + \mathbf{c})\,/\mathrm{RMS}(\mathbf{a}) \neq \mathbf{a}\mathbf{W}^*/\mathrm{RMS}(\mathbf{a}) + \mathbf{c}$; the layer then applies the scalar to the input first and lets the linear add its bias as usual.

Proposition 3 (RMS cancellation). By RMS scale invariance, $\mathrm{RMS}(\alpha\mathbf{a}) = \alpha\,\mathrm{RMS}(\mathbf{a})$. Therefore an "RMSNorm → Linear → RMSNorm" pattern allows the *first* RMSNorm to be dropped entirely:

$$\mathrm{RMSNorm}\left(\frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^*\right) = \mathrm{RMSNorm}(\mathbf{a}\mathbf{W}^*)\,.$$

This cancellation is relevant for QKV-normalization (Gemma 4) and MLA latent normalization (DeepSeek-V2, Mistral Small 4); it is not auto-applied in this repository but is documented for completeness.

Partial-RMS composition. FlashNorm composes naturally with the pRMSNorm trick (§9, pRMSNorm subsection): when `flashnorm_partial_ratio` $= p > 0$, the RMS is estimated from the first $k = \lceil d \cdot p \rceil$ dimensions of the input only, reducing the cost of the vector-unit RMS reduction further.

BitNet interaction. Folding a floating-point $\mathbf{g}$ into BitNet's ternary weight tensor $\{-1, 0, +1\}$ [81] would destroy the quantization, and BitLinear applies its own internal LayerNorm before activation quantization which needs the normalized input. The fused module `FlashNormBitLinear` therefore falls back to the sequential composition FlashNorm → BitLinear: the matmul and the RMS reduction still execute in parallel, but the post-matmul scalar multiply of Prop. 2 is sacrificed. Folding f.p. $\mathbf{g}$ into a 1.58-bit weight tensor is left to future kernel-level work.

PyTorch pseudocode.

Input: activation $\mathbf{a} \in \mathbb{R}^{T\times n}$, RMSNorm weight $\mathbf{g} \in \mathbb{R}^n$, linear $\mathbf{W} \in \mathbb{R}^{n\times k}$, bias $\mathbf{c} \in \mathbb{R}^k$ or `None`
`// Prop. 1: weight folding at load time`
$\mathbf{W}^* \leftarrow \mathbf{g} \odot \mathbf{W}$ ▷ column-wise; `g.unsqueeze(0) * W` in PyTorch
$\mathbf{g} \leftarrow \mathbf{1}$ ▷ the standalone norm is now weightless
`// Prop. 2: forward pass with deferred RMS (bias-free path)`
$\mathbf{b} \leftarrow \mathbf{a}\mathbf{W}^{*\top}$ ▷ matrix unit / tensor cores
rms_inv $\leftarrow$ rsqrt(mean($\mathbf{a}^2$) + ε) ▷ vector unit, independent of $\mathbf{b}$
if $\mathbf{c}$ = `None` **then**
  return $\mathbf{b}$ · rms_inv ▷ single vector–scalar multiply
else
  `// Remark 1: bias path is sequential; matmul and RMS still parallelize`
  return ($\mathbf{a}$ · rms_inv)$\mathbf{W}^{*\top} + \mathbf{c}$
end if

Reported latency (NVIDIA T4 GPU). The paper reports the following speedups on the norm-then-project operation, with an adaptive dispatch that falls back to the sequential path in the memory-bound (decode) regime:

Scale	Tokens	Sequential (ms)	FlashNorm (ms)	Speedup	Notes
SmolLM2-135M	4096	0.704	0.468	+33.6%	compute-bound (prefill)
SmolLM2-135M	8192	0.929	0.599	+35.5%	compute-bound (prefill)
Llama-7B	1024	1.882	1.654	+12.1%	compute-bound (prefill)
Llama-7B	4096	7.628	6.570	+13.9%	compute-bound (prefill)

Zero-loss weight folding validation. Weight folding (Prop. 1) is mathematically exact; the paper verified bit-identical outputs on three open-weight checkpoints:

Model	Precision	Result
SmolLM2-135M	fp16	max diff = 0.0, cosine sim = 1.0, identical text
Llama-3.2-1B	fp32	bit-identical greedy generation
Llama-3.1-8B	fp32	bit-identical greedy generation

Quality preservation on `lm-evaluation-ha` (wikitext word_ppl, MMLU 5-shot, HellaSwag 0-shot) stays within benchmark noise (worst $\Delta = +0.0017$ word_ppl).

9.1.7 Comparison

Method	Formula	Stats Needed	Notes
LayerNorm	$\gamma_i(x_i - \mu)/\sqrt{\sigma^2 + \epsilon} + \beta_i$	mean + variance	Default; full centering and rescaling [4].
RMSNorm	$\gamma_i x_i / \sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}$	RMS only	Lower overhead; widely used baseline [92].
pRMSNorm	$\gamma_i x_i / \sqrt{\frac{1}{k}\sum_{j\le k} x_j^2 + \epsilon}$	Partial RMS	RMS from first p% of dims; `prms_partial_ratio` default 6.25% [92].
Dynamic Tanh	$\tanh(\alpha x)$	none	Normalization-free bounded transform; drop-in replacement [97].

Method	Formula	Stats Needed	Notes
Dynamic Erf	$\mathrm{erf}(\alpha x + s)$	none	Normalization-free alternative; improves over DyT [12].
FlashNorm	$x_i / \sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}$ (weightless)	RMS only	Exact rewrite of RMSNorm→Linear: folds $\mathbf{g}$ into $\mathbf{W}$ (Prop. 1), defers scalar RMS to matmul output (Prop. 2); 12–35% latency reduction on T4 GPU [30].

9.2 BitNet Ternary Quantization Path

When `use_bitnet=true`, BitLinear layers replace standard `nn.Linear` with ternary weight quantization $(-1, 0, +1)$ following BitNet [81]. Given weight tensor W, a practical scaling is:

$$s = \mathrm{mean}(|W|), \qquad \tilde{W} = \mathrm{clip}\left(\mathrm{round}\left(\frac{W}{s}\right), -1, 1\right).$$

The packed mapping uses two bits per weight symbol for storage efficiency. This enables significant compression ($\sim 32\times$ versus FP32) but is experimental and may affect model quality; it is best suited for inference deployment.

Activation quantization for the INT8 path is:

$$q = \mathrm{round}\left(x \cdot \frac{127}{\max(|x|) + \epsilon}\right), \qquad q \in [-128, 127]$$

with dequantization $x \approx q/\alpha$. For N parameters, size estimates before metadata and packing overhead are:

$$\text{FP32 size} \approx 4N, \quad \text{FP16 size} \approx 2N, \quad \text{1.58-bit size} \approx \frac{1.58}{8}N$$

which aligns with lightweight deployment goals [70, 11].

9.3 Mixture-of-Depths (MoD) Token Routing

When `use_mixture_of_depths=true`, each transformer block scores tokens with a lightweight router. Only the top `mixture_of_depths_capacity_ratio` subset is updated; skipped tokens pass through unchanged [98]. This allocates more depth to salient tokens, reducing per-layer compute.

Input: hidden states $H \in \mathbb{R}^{n \times d}$, router R, capacity ratio ρ
scores $\leftarrow R(H)$ ▷ per-token router score
$\mathcal{S} \leftarrow$ `top-k`$(\text{scores}, k = \lceil \rho \cdot n \rceil)$ ▷ select salient subset
$H_{\text{update}} \leftarrow$ `block`$(H[\mathcal{S}])$ ▷ apply transformer block to selected tokens
$H[\mathcal{S}] \leftarrow H_{\text{update}}$ ▷ write back; non-selected tokens bypass the block
Output: H ▷ auxiliary load-balancing loss added from `mixture_of_depths_router_aux_loss_weight`

9.3.1 Configuration

- `use_mixture_of_depths`: bool — enable MoD token routing.
- `mixture_of_depths_capacity_ratio`: float in $(0, 1]$ — fraction of tokens updated per layer.
- `mixture_of_depths_router_aux_loss_weight`: float ≥ 0 — auxiliary loss weight for router regularization.

9.4 Engram Conditional Memory

Engram (`engram_attn`) implements conditional memory via scalable N-gram lookup [14]. It builds hash-based lookup tables for N-gram contexts from size 2 up to `engram_max_ngram_size`, with independent hash heads per N-gram order. A causal depthwise convolution (`engram_kernel_size`) provides short-range context before the N-gram lookup.

Input: token stream $x_{1:n}$, N-gram orders $\{2, \ldots, N\}$, hash heads per order H
$h_{\text{conv}} \leftarrow \text{DepthwiseConv1d}(x_{1:n}, k = \texttt{engram\_kernel\_size})$ ▷ short-range context
for each N-gram order $k \in \{2, \ldots, N\}$ **do**
    $\text{ctx}_t^{(k)} \leftarrow x_{t-k+1:t}$ ▷ N-gram context window
    $\text{lookup}_t^{(k)} \leftarrow \text{HashTable}^{(k)}[\text{hash}(\text{ctx}_t^{(k)})]$ ▷ per-head retrieval, H heads per order
end for
$y_t \leftarrow \sum_k \text{lookup}_t^{(k)} + h_{\text{conv},t}$ ▷ combine conditional memories
Output: $y_{1:n}$

9.4.1 Configuration

- `engram_max_ngram_size`: int ≥ 2 — highest N-gram order (default 3).
- `engram_n_heads_per_ngram`: int ≥ 1 — hash heads per N-gram order (default 4).
- `engram_embed_dim_per_head`: int ≥ 1 — embedding dim per hash head (default 32).
- `engram_kernel_size`: int ≥ 1 — causal depthwise conv kernel width (default 4).
- `engram_seed`: int — hash seed for reproducibility (default 42).

Total Engram hidden size = (`max_ngram_size`-1)$\times$`n_heads_per_ngram`$\times$`embed_dim_per_head`.

9.5 Factorized Embeddings and Embedding Convolution

9.5.1 Factorized Embeddings

When `use_factorized_embedding=true`, the embedding matrix is factorized into two smaller matrices, reducing parameters from $O(V \times H)$ to $O(V \times R + R \times H)$ where $R =$ `factorized_embedding_dim`.

- `use_factorized_embedding`: bool — enable factorization.
- `factorized_embedding_dim`: int ≥ 1 — intermediate dimension (typical: 64–256).

9.5.2 Embedding Convolution

When `use_embedding_conv=true`, a depthwise Conv1d is applied over the embedding stream before the first transformer block, with kernel size `embedding_conv_kernel`. This provides a lightweight local-context filter on token embeddings.

9.6 Training-Step Stability Controls

The schema-driven training step applies gradient accumulation, global-norm clipping, post-clip explosion guards, and bounded NaN/Inf retries. The accumulated gradient is:

$$g_{\text{acc}} = \frac{1}{K} \sum_{i=1}^{K} g_i, \quad K = \texttt{gradient\_accumulation\_steps}$$

$$g_{\text{clip}} = g_{\text{acc}} \cdot \min\left(1, \frac{\tau}{\|g_{\text{acc}}\|_2 + \epsilon}\right), \quad \tau = \texttt{grad\_clip\_max\_norm}$$

then overflow guards use `inf_post_clip_threshold` and retry logic bounded by `max_nan_retries`.

Algorithm 40 Schema-Driven Training Step with Stability Controls

Require: Batch stream, config C

Initialize retry counter $r \leftarrow 0$
for each optimizer step **do**
  Accumulate gradients for $K = C.\texttt{gradient\_accumulation\_steps}$ micro-batches
  Apply global norm clipping with $\tau = C.\texttt{grad\_clip\_max\_norm}$
  if post-clip gradient exceeds $C.\texttt{inf\_post\_clip\_threshold}$ or NaN/Inf detected **then**
    if $r < C.\texttt{max\_nan\_retries}$ **then**
      restore safe state / skip step; $r \leftarrow r + 1$
      continue
    else
      stop training with failure state
    end if
  end if
  run optimizer step selected by `optimizer_class`
  update scheduler (`cosine`, `constant`, or `linear_warmup_then_constant`)
  if step mod `checkpoint_every_n_steps`$= 0$ **then**
    save rolling checkpoint and prune to `max_rolling_checkpoints`
  end if
  update best checkpoints up to `num_best_checkpoints`
  emit CSV + telemetry following `gradient_log_interval` and `telemetry_log_interval`
end for

Algorithm 41 SBERT Inference Mode Router

Require: mode m, model E, inputs X

if $m =$ `similarity` **then**
  return $\cos(E(x_1), E(x_2))$
else if $m =$ `search` **then**
  return top-k by dot-product/cosine against corpus embeddings
else if $m =$ `cluster` **then**
  return clustering labels over $E(X)$
else
  return serialized embeddings $E(X)$
end if

9.7 SBERT Inference Mode Router

The SBERT inference dispatcher selects among similarity scoring, corpus search, clustering, and persistent encoding modes over a single shared encoder [68]. The cosine similarity and regression loss are:

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\|\|e_2\|}, \qquad \mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

with $y \in [-1, 1]$ in this pipeline.

10 Activation Functions

This annex documents the complete activation-function library available in the model. The library provides **43 feed-forward activation functions**: 40 elementwise activations and 3 gated-FFN variants. Activations are grouped into five families, following the taxonomy of Dubey et al. [21] and the analytic overview of Lederer [44], augmented with the learnable Rational Activation Function (RAF) of Fang et al. [24].

The selection is driven by the `ffn_activation` configuration field (a string enum) and dispatched at runtime by a factory function that mirrors the normalization-layer dispatch (Annex 7).

$$\texttt{ffn\_activation} \in \{\texttt{silu},\ \texttt{gelu},\ \ldots,\ \texttt{raf},\ \texttt{swiglu},\ \texttt{geglu},\ \texttt{reglu}\}$$

10.1 Classical / Sigmoid–Tanh Family (12 activations)

The classical family comprises smooth, stateless activations rooted in logistic and algebraic functions [44, 21]. All share monotonicity or near-monotonicity and are continuously differentiable.

$$\text{Sigmoid}(x) = \frac{1}{1+e^{-x}} \qquad \text{Range: } (0,1),\ \text{monotonic, smooth} \quad [44] \tag{81}$$

$$\text{Tanh}(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}} = 2\,\sigma(2x) - 1 \qquad \text{Range: } (-1,1),\ \text{monotonic, smooth} \quad [44] \tag{82}$$

$$\text{Arctan}(x) = \arctan(x) \qquad \text{Range: } (-\tfrac{\pi}{2}, \tfrac{\pi}{2}),\ \text{monotonic, smooth} \quad [44] \tag{83}$$

$$\text{Softsign}(x) = \frac{x}{1+|x|} \qquad \text{Range: } (-1,1),\ \text{smooth, once-diff. at 0} \quad [44] \tag{84}$$

$$\text{Elliott}(x) = \frac{x}{1+|x|} \qquad \text{Alias of Softsign (survey “elliottsig” notation)} \quad [44] \tag{85}$$

$$\text{Identity}(x) = x \qquad \text{Range: } (-\infty,\infty),\ \text{linear, } C^{\infty} \quad \text{(internal)} \tag{86}$$

$$\text{Softplus}(x) = \log(1+e^{x}) \qquad \text{Range: } [0,\infty),\ \text{smooth, } C^{\infty} \tag{87}$$

$$\text{Mish}(x) = x\,\tanh\big(\text{softplus}(x)\big) = x\,\tanh\big(\log(1+e^{x})\big) \qquad \text{Range: } [\approx -0.31,\infty),\ \text{non-monotonic, smooth} \tag{88}$$

$$\text{GELU}(x) = x\,\Phi(x) = x\cdot\tfrac{1}{2}\Big(1+\operatorname{erf}\Big(\frac{x}{\sqrt{2}}\Big)\Big) \qquad \text{Range: } (\approx -0.17,\infty),\ \text{smooth, probabilistic} \tag{89}$$

$$\text{GELU-tanh}(x) = \tfrac{x}{2}\Big(1+\tanh\Big(\sqrt{\tfrac{2}{\pi}}(x+0.044715\,x^{3})\Big)\Big) \qquad \text{Tanh approximation of GELU (faster)} \tag{90}$$

$$\text{ReLU}(x) = \max(0,x) \qquad \text{Range: } [0,\infty),\ \text{piecewise-linear, non-smooth at } 0 \tag{91}$$

$$\text{SiLU}(x) = x\,\sigma(x) = \frac{x}{1+e^{-x}} \qquad \text{Range: } (\approx -0.278,\infty),\ \text{non-monotonic, smooth} \tag{92}$$

SiLU (Sigmoid Linear Unit) is the default activation (`ffn_activation=silu`). When $\beta = 1$, the parametric Swish reduces to SiLU. GELU and its tanh approximation are the standard in

BERT and GPT-2/3 [36]. Mish [57] is a self-regularized, non-monotonic composition of Softplus and Tanh, reported to outperform Swish on several benchmarks.

10.2 Rectified Family (12 activations)

The rectifier family extends ReLU with learnable slopes, bounded variants, and rectified-hyperbolic forms [59, 21].

$$\text{LeakyReLU}(x) = \begin{cases} x & x \geq 0 \\ a\,x & x < 0 \end{cases}, \quad a = 0.01 \qquad \text{Range: } ($$
(93)

$$\text{ReLU6}(x) = \min\big(\max(0, x),\, 6\big) \qquad \text{Range: } [0, 6],\ \text{bounded, fixed-point}$$
(94)

$$\text{HardSwish}(x) = x \cdot \frac{\text{ReLU6}(x+3)}{6} \qquad \text{Range: } (\approx -1.67, \infty),\ \text{cheap Swish}$$
(95)

$$\text{PReLU}(x) = \max(0, x) + \mathbf{p} \odot \min(0, x) \qquad \text{Learnable per-channel slope } \mathbf{p},$$
(96)

$$\text{AbsReLU}(x) = \max(0, x) - \max(0, -x) = x \text{ rectified to sign-ramp} \qquad \text{Range: } ($$
(97)

$$\text{NLReLU}(x) = \beta\,\log\big(1 + \max(0, x)\big) \qquad \text{Range: } [0, \infty),\ \beta = 1.0,\ \text{logarithmic con}$$
(98)

$$\text{BReLU}(x) = \min\big(\max(0, x),\, t\big), \quad t = 1.0 \qquad \text{Range: } [0, t],\ \text{bounded rect}$$
(99)

$$\text{VReLU}(x) = |x| \qquad \text{Range: } [0, \infty),\ \text{symmetric rect}$$
(100)

$$\text{Hexpo}(x) = a\,\max(0, x) - c\,\max(0, -x), \quad a{=}c{=}1.0 \qquad \text{Asymmetric two-sided rect}$$
(101)

$$\text{PenalizedTanh}(x) = \max\big(0,\, \tanh(x)\big) \qquad \text{Range: } [0, 1),\ \text{rectified tanh, smooth ne}$$
(102)

$$\text{DisReLU}(x) = \max(0,\, x - \delta), \quad \delta = 0.0 \qquad \text{Right-shifted ReLU h}$$
(103)

$$\text{LiSHT}(x) = x\,\tanh(x) \qquad \text{Range: } [0, \infty)\ (\text{magnitude}),\ \text{non-monotonic, smo}$$
(104)

PReLU [35] makes the negative slope a per-channel learnable parameter, enabling the network to learn the optimal leak per feature. **HardSwish** [37] is the cheap Swish approximation used in MobileNetV3. **ReLU6** [38] was introduced in MobileNetV1 for fixed-point quantization friendliness.

10.3 Exponential / ELU Family (12 activations)

The exponential-linear family provides smooth negative saturation and self-normalizing variants [16, 41, 21]. Several members introduce per-channel learnable parameters.

$$\mathrm{ELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}, \quad \alpha = 1.0 \qquad \text{Range: } (-\alpha, \infty) \quad [16] \quad (105)$$

$$\mathrm{SELU}(x) = \lambda\, \mathrm{ELU}_{\alpha'}(x) \qquad \alpha' \approx 1.6733,\ \lambda \approx 1.0507, \quad [41] \quad (106)$$
self-normalizing (mean 0, var 1)

$$\mathrm{CELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^{x/\alpha} - 1) & x < 0 \end{cases}, \quad \alpha = 1.0 \qquad \text{Once-diff. at 0 for any } \alpha \quad [5] \quad (107)$$

$$\mathrm{PELU}(x) = \begin{cases} \frac{\alpha}{\beta}\, x & x \geq 0 \\ \alpha(e^{x/\beta} - 1) & x < 0 \end{cases} \qquad \alpha, \beta \text{ learnable per-channel} \quad [78] \quad (108)$$

$$\mathrm{MPELU}(x) = \mathrm{ReLU}(x) + \beta \cdot \mathrm{ELU}_\alpha(x) \cdot \mathbb{1}_{x<0} \qquad \alpha, \beta \text{ learnable per-channel} \quad [21] \quad (109)$$

$$\mathrm{FELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases} \qquad \alpha \text{ learnable, clamped to } [0, 1] \quad [21]$$

(110)

$$\mathrm{EELU}(x) = \beta\, \mathrm{ReLU}(x) + \alpha(e^x - 1) \cdot \mathbb{1}_{x<0} \qquad \alpha, \beta \text{ learnable per-channel} \quad [21]$$
(111)

$$\mathrm{PDELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^{x/\alpha} - 1) + (1 - \alpha)\, x & x < 0 \end{cases} \qquad \alpha \text{ learnable, interpolates CELU–identity} \quad [21]$$

(112)

$$\mathrm{PREU}(x) = \beta\, \mathrm{ReLU}(x) + \alpha(e^x - 1) \cdot \mathbb{1}_{x<0}^{\leq 0} \qquad \alpha, \beta \text{ learnable per-channel} \quad [21]$$
(113)

$$\mathrm{SoftExp}(x) = \begin{cases} \frac{1}{\alpha}\big(e^{\alpha x} - 1\big) + \alpha & \alpha > 0 \\ x & \alpha = 0 \\ -\frac{1}{\alpha}\ln\big(1 - \alpha(x + \alpha)\big) & \alpha < 0 \end{cases} \qquad \alpha \text{ learnable; exp/linear/log interpolation} \quad [28]$$

(114)

$$\mathrm{ELiSH}(x) = \begin{cases} x\, \sigma(x) & x \geq 0 \\ (e^x - 1)\, \sigma(x) & x < 0 \end{cases} \qquad \text{Swish pos. branch, ELU-gated neg. branch} \quad [6]$$

(115)

$$\mathrm{HardELiSH}(x) = \begin{cases} x \cdot \mathrm{hard\_sigmoid}(x) & x \geq 0 \\ (e^x - 1) \cdot \mathrm{hard\_sigmoid}(x) & x < 0 \end{cases} \qquad \text{Hard-sigmoid variant of ELiSH} \quad [6]$$

(116)

where $\mathrm{hard\_sigmoid}(x) = \mathrm{ReLU6}(x + 3)/6$.

ELU [16] provides smooth negative saturation. **SELU** [41] induces self-normalization with fixed $\alpha' \approx 1.6733$, $\lambda \approx 1.0507$. **CELU** [5] is once-differentiable at 0 for any α (unlike ELU). **PELU** [78] makes both α, β learnable per-channel. The parametric variants (MPELU, FELU, EELU, PDELU, PREU) generalize ELU and PReLU by introducing learnable per-channel parameters for the negative and positive branches. **SoftExp** [28] continuously interpolates between logarithmic ($\alpha < 0$), linear ($\alpha = 0$), and exponential ($\alpha > 0$) regimes. **ELiSH** and **HardELiSH** [6] combine Swish and ELU behaviors via a piecewise definition.

10.4 Swish and Parametric Activations

Swish [66] is a family parameterized by a slope β:

$$\text{Swish}_\beta(x) = x\,\sigma(\beta x) = \frac{x}{1 + e^{-\beta x}}.$$

When $\beta = 1$ this is SiLU (the default). Large β approaches ReLU; $\beta \to 0$ approaches the linear function $x/2$.

- **Swish** (`swish`): fixed β (default 1.0, configured via `ffn_activation_config.swish_beta`). No learnable parameters.
- **SwishTrainable** (`swish_trainable`): β is a per-channel learnable parameter, initialized to 1.0. Equivalent to SiLU at initialization, then adapts during training.
- **Maxout** (`maxout`): computes k linear projections and takes the elementwise maximum: $\max_{i=0}^{k-1}(W_i x + b_i)$. A universal approximator for any continuous function given enough pieces. Parameter count increases by factor k (default $k = 2$, configured via `ffn_activation_config.maxout_pieces`).

References: Swish [66], Maxout [29].

10.5 Rational Activation Functions (RAF)

Following Fang et al. [24], a Rational Activation Function is a learnable ratio of two low-degree polynomials (a Padé approximant):

$$F(x) = \frac{P(x)}{Q(x)} = \frac{\sum_{j=0}^{m} a_j\, x^j}{1 + R(x)}$$

where $\mathbf{a} \in \mathbb{R}^{m+1}$ and $\mathbf{b} \in \mathbb{R}^n$ are *learnable*, and the denominator term $R(x)$ depends on the version. The default configuration matches the paper: degree $(m, n) = (5, 4)$, version "A" (safe), initialized by a least-squares fit to GELU on $[-3, 3]$.

The implementation supports five denominator variants:

- **Version A** (default, "safe"): $Q(x) = 1 + \sum_k |b_k\, x^{k+1}|$ (per-term absolute value). Guarantees $Q(x) \geq 1$, no division by zero.
- **Version B**: $Q(x) = 1 + \left|\sum_k b_k\, x^{k+1}\right|$ (absolute value of the whole sum). Also guarantees $Q(x) \geq 1$.
- **Version C**: $Q(x) = 0.1 + \left|\sum_k b_k\, x^k\right|$ with a 0.1 floor. Minimum denominator 0.1.
- **Version D**: like B, but injects uniform multiplicative noise $U(1-\varepsilon, 1+\varepsilon)$ on the denominator weights during *training only* ($\varepsilon = 0.1$). Regularization effect.
- **Version N** ("nunsafe"): $Q(x) = 1 + \sum_k b_k\, x^{k+1}$ without any absolute value. Can have poles (denominator zeros); use with care.

RAFT input scaling. Because a Padé approximant is a reliable universal approximator only on a bounded interval, the RAFT model preprocesses each token's pre-activations with a per-token min–max scaling into $[-3, 3]$:

$$\tilde{x} = \frac{x - \min(x)}{\max(x) - \min(x)} \cdot 6 - 3.$$

This is toggled by `ffn_activation_config.raf_input_scaling` (default `false`).

Initialization targets. The rational coefficients can be initialized by fitting to any of the following target functions: `gelu` (default), `relu`, `leaky_relu`, `leaky_relu_0.1`, `sigmoid`, `tanh`, `swish`/`silu`, or `identity`. The fit uses a least-squares solve on 2001 uniformly spaced samples over $[-3, 3]$.

Freezing. Setting `raf_trainable=false` freezes the rational parameters, useful for parameter-efficient fine-tuning. Frozen RAFs are typically paired with a separate (higher) learning rate for the remaining network parameters.

Note on naming. RAF stands for *Rational* Activation Function, not "Rectified"; it is the learnable polynomial ratio described above.

10.6 Gated FFN Variants: SwiGLU, GEGLU, ReGLU

SwiGLU, GEGLU, and ReGLU [71] are gated feed-forward units that replace the standard Linear → activation → Linear block. They own their own linear projections and are therefore implemented as FFN modules, *not* as elementwise activations:

$$\text{GatedFFN}(x) = \text{dropout}\Big(W_{\text{down}}\Big(\text{act}(x\,W_{\text{gate}}) \odot (x\,W_{\text{up}})\Big)\Big),$$

where $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d \times d_{\text{ff}}}$ and $W_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d}$. The gate function is:

- **SwiGLU** (`swiglu`): act = SiLU (the default, used in Llama, PaLM).
- **GEGLU** (`geglu`): act = GELU (used in some T5 variants).
- **ReGLU** (`reglu`): act = ReLU (cheapest option).

When `ffn_activation` is one of `swiglu`/`geglu`/`reglu`, the feed-forward layer swaps the standard dense and MoE FFN blocks for a gated FFN built with the same projection class (BitLinear under BitNet). Bias is disabled by default.

10.7 Configuration Schema

The `ffn_activation_config` object groups all learnable-activation parameters. It is optional and ignored for stateless activations.

- `raf_degrees`: $[m, n]$ Padé numerator/denominator degrees (ints ≥ 1). Default: $[5, 4]$.
- `raf_version`: denominator form, one of `A` (default), `B`, `C`, `D`, `N`.
- `raf_approx_func`: initialization target. One of `gelu` (default), `relu`, `leaky_relu`, `leaky_relu_0.1`, `sigmoid`, `tanh`, `swish`, `silu`, `identity`.
- `raf_trainable`: freeze the rational parameters when `false`. Default: `true`.
- `raf_input_scaling`: per-token min–max scaling to $[-3, 3]$ before the rational (RAFT preprocessing). Default: `false`.
- `prelu_init`: initial negative slope for PReLU. Default: 0.25.
- `elu_alpha`: ELU saturation constant. Default: 1.0.
- `celu_alpha`: CELU saturation constant. Default: 1.0.
- `swish_beta`: fixed β for Swish / initial β for SwishTrainable. Default: 1.0.
- `leaky_relu_slope`: negative slope for LeakyReLU. Default: 0.01.

- `maxout_pieces`: number of linear pieces k for Maxout (int ≥ 1). Default: 2.
- Additional ELU-parametric keys: `pelu_alpha`, `mpelu_alpha`, `mpelu_beta`, `felu_alpha`, `eelu_alpha`, `eelu_beta`, `pdelu_alpha`, `preu_alpha`, `preu_beta`, `softexp_alpha`. All default to 1.0 (except `softexp_alpha` which defaults to 0.0).

10.8 Selection Guide

The following decision tree aids activation choice:

- **Safe default** → `silu` (SiLU/Swish$_1$, used in Llama, PaLM).
- **Classic stability** → `gelu` or `relu`.
- **Smooth non-monotonic** → `mish` or `swish`.
- **Mobile-friendly** → `hardswish` or `relu6`.
- **Learnable per-task shape** → `raf`, `prelu`, `swish_trainable`, or `maxout`.
- **Self-normalizing networks** → `selu` (with AlphaDropout).
- **Gated FFN (own projections)** → `swiglu`, `geglu`, or `reglu`.

With BitNet quantization, `silu` may be more robust than smooth activations that rely on precise gradient computation. Learnable activations (`raf`, `prelu`, `swish_trainable`) add parameters but can adapt to the task distribution.